

claude-windows / b3f086c7-9143-451e-8d2c-de20114d21c6

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\b3f086c7-9143-451e-8d2c-de20114d21c6.jsonl`
- SHA-256: `9e666ece306e4f82aa3aa02e3fff5a32420a68d10d528e57a20b958c00637d8a`
- Source modified: `2026-07-07T17:43:54+00:00`
- Imported at: `2026-07-08T15:59:14+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `b3f086c7-9143-451e-8d2c-de20114d21c6`
```

Transcript

1. user (2026-07-07T16:57:05.533Z)

Add a validation-result cache keyed by video_id so a reupload does NOT re-run validation when the same content was already validated.

Problem (confirmed in backend/orchestration.py::execute_processing): the only skip path is a WHOLE-PIPELINE cache (~lines 732-749) that fires only when the video is status=="processed" AND has cached_segments (i.e., a prior SUCCESSFUL segmentation). So after a SEGMENTATION FAILURE (validation passed, detection failed ? no cached segments), a reupload of the SAME video_id re-runs the full validator suite from scratch (`registry.run\_all\_with\_context`, ~line 754) ? which on a heavy clip is minutes (e.g. ~6.5 min on the 5.3K 10-bit HEVC "Video 1"). Validation results ARE already persisted on the video record (add_video stores validation_results), but they're never used to short-circuit the validation STEP. This bit the owner repeatedly during the 2026-07-07 CUJ retries.

video_id = MD5 of the content (backend/utils/hashing.py::compute_video_id), so identical bytes ? identical id. Design a cache: when a request's video_id already has a stored validation verdict computed under the SAME validator configuration, skip re-running the validators and reuse the stored result (pass ? proceed to segmentation; fail ? return the stored failure). Key must include a validator-config fingerprint (e.g. hash of the relevant validation.* config: min_blur_threshold, min_fps, min_resolution, scene-cut/relevance thresholds, pts gap) so that a config change correctly invalidates the cache. Provide an override to force re-validation (req.force already bypasses the pipeline cache ? extend/mirror that). Store either on the video record (extend the existing validation_results with the config fingerprint + a schema/version) or a dedicated table.

Interplay: this composes with #513 (worker already skips re-validation on dispatch) and #512 (validators are faster) ? together, a post-failure reupload should reach segmentation in ~0 validation time. Guardrails: don't serve a stale PASS if the validator thresholds changed; keep the content-hash keying so a different video (even same filename) never reuses another's verdict.

Acceptance: a second /api/process for the same video_id (same validator config) skips validation (assert via logs/timing + a unit test in tests/); a config change forces re-validation; req.force bypasses. Repo pre-merge gate (full suite + ruff + mypy + coverage), branch from origin/master. Related: task 'Speed up video validation', #513.

2. assistant (2026-07-07T16:57:22.979Z)

I'll start by getting oriented ? reading the session handoff and key memories, then finding the actual code location.

3. user (2026-07-07T16:57:24.041Z)

```
1      ---
2      name: actual-repo-location
3      description: "The badminton-highlight-indexer code lives under khelsutra-guru/, not the
```

```

primary working dir stub"
4     metadata:
5         node_type: memory
6         type: project
7         originSessionId: df5d5fb2-534a-447a-8181-b222f94655f3
8     ---
9
10    The primary working directory `C:\Users\avidu\Projects\badminton-highlight-indexer` is a
near-empty
11    **stub** ? it contains only `AGENTS.md`, an empty `.claude/`, and
`output/nightly_regression/`
12    (a `LATEST_STATUS.txt`). The actual `khelsutra-guru/badminton-highlight-indexer`
codebase lives at:
13
14    `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer`
15
16    (git remote `github.com/Khelsutra/badminton-highlight-indexer`, default branch
`master`).
17
18    **How to apply:** when a task references this repo's code (backend/worker,
pipeline/segmenters,
19    config.json, run_all_tests.py, etc.), `cd` to the khelsutra-guru path and work there ?
the stub dir
20    has none of it. Confirmed 2026-07-06 while fixing the wf294 silent-detector-failure
issue.
21
22    Repo conventions verified there: CI = ruff 0.15.16 (`ruff check .`), mypy 2.1.0 (`mypy
backend
23    training`; `mypy.ini` quarantines several modules incl. `backend.worker.__main__`,
`geminis`,
24    `yolo_hybrid` via `ignore_errors=True`), full suite via `python run_all_tests.py`,
coverage floor
25    `--cov-fail-under=70` (actual ~86%). Segmenters use a `Success`/`Failure` Result
contract
26    ([[efficient-timeouts-and-retries]] is unrelated).
27

```

4. user (2026-07-07T16:57:24.517Z)

```

1     ---
2     name: session-handoff-gpu-resident
3     description: "Start here to pick up the GPU-resident WASB rollout (issue #506) ?
implementation PR #507 is open; golden gate next"
4     metadata:
5         type: project
6     ---
7
8     # Session Handoff ? GPU-resident WASB inference (rollout) . issue #506 . PR #507
MERGED
9
10    ## LATEST 2026-07-07 (end of session) ? all serving fixes merged + fresh redeploy
11    Every gap surfaced this session is merged to `origin/master @ ef41473`: **511**
(compute=local fails fast on the weightless control-plane) + **516** (SPA compute-picker
defaults to CLOUD on hosted) = the compute-picker footgun; **512** (validators share ONE decode
pass ? measured ~3.1x on the 5.3K clip: warm 179s?58s on a 16-core box; projects ~6.5min?~2min
on the 2-vCPU control-plane); **513** (worker skips redundant re-validation on dispatch ? also
fixes the default-blur-threshold reject); **515** (worker no-done-marker fastfail). **Clean
rebuild from `ef41473` ? redeploy control-plane + worker to the new digest (D3)** ? see the
DEPLOY_REPORT / [[gpu-vm-setup-gotchas]]. #509 forwarding was PROVEN live by the CUJ (worker
args carried `decode_backend=nvdec_resident`+batch64+prefetch4). Two follow-ups SPAWNED:
**task_5acf4b46** (clean 2?3 video validation-speedup closure test + DEPLOY_REPORT) and
**task_4585aa00** (cache validation results by video_id ? skip re-validation on reupload; the
missing cache that made CUJ retries re-pay validation). Validation is NOT cached across
reuploads today (only a whole-pipeline cache gated on a prior SUCCESSFUL segmentation).
12

```

```

13      ## SHIPPED 2026-07-07 ? GPU-resident NVDEC decode is LIVE in cloud-serving
14      Full arc done: PR #507 (feature) + PR #508 (cloud-serving preset default =
nvdec_resident) both merged to master (44a2f47, 143dbb9). The `rally-worker` Cloud Run Job
(asia-southeast1) runs the new image `rally-worker:latest` `sha256:465a9250?` (torch 2.6 +
PyNvVideoCodec 2.1.0 + baked NVDEC libs) with `WASB_DECODE_BACKEND=nvdec_resident` set ? **nvdec
is the live serving default**, validated on the real L4 (GX010128 served trajectory reproduced
the VM: 26479?26481 visible; cpu+nvdec+canary smokes all ?). Record:
`docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07.md`.
15      - **ROLLBACK levers:** `gcloud run jobs update rally-worker --region asia-southeast1
--remove-env-vars WASB_DECODE_BACKEND` (? torch-2.6 cpu path, same image); or `--image ?/rally-
worker@sha256:3bfefaf6?` (? prior torch-1.11 image).
16      - **PRs:** #507 (feature) + #508 (preset default) + #509 (dispatch forwarding + release
hardening) all merged. master @ 66756ad.
17      - **DONE ? control-plane + worker both on image D2 `sha256:09d2edf7` (built from master
66756ad = #507+#508+#509).** Redeployed 2026-07-07: `gcloud run services update rally-control-
plane --image ? :latest` ? rev **00017-qbd** (100% traffic, booted healthy: uvicorn up, cloud-
serving profile ACTIVE, edge-auth on); `gcloud run jobs update rally-worker --image ? :latest`.
So the #509 dispatch **forwarding** + #508 preset default are now LIVE in the control-plane
image. NB the FIRST control-plane deploy this session used the stale `465a9250` (built from
44a2f47, pre-#508/#509) ? rev 00016; superseded by 00017. Rollback: `gcloud run services update-
traffic rally-control-plane --to-revisions rally-control-plane-00015-kp9=100` (last pre-nvdec
rev) or `--to-revisions ?-00016-?`.
18      - **`WASB_DECODE_BACKEND=nvdec_resident` env is still set on the worker Job** ? now
REDUNDANT with the forwarded --set + preset (both say nvdec, env just wins), kept as belt-and-
suspenders + manual override. Safe to remove ONLY after a real /api/process dispatch confirms
the forwarding path live (needs a CF Access JWT ? not verifiable this session; forwarding is
deployed + unit-tested but not live-dispatch-verified).
19      - `DEPLOY_REPORT?gpu-resident-nvdec_2026-07-07.md` is current: PR #510 added the $5
"Update" (D2 redeploy of control-plane rev 00017 + worker) and corrected $6. All four PRs
merged: #507 #508 #509 #510; master @ 3be0935.
20      - **Wrinkles fixed (#509):** dispatch now forwards
indexing.wasb.{decode_backend,batch_size,prefetch_batches} (worker has no DEPLOYMENT_PROFILE ?
never applied the preset; batch tuning was silently lost too); added `.gcloudignore` (Cloud
Build had none ? relied on the .gitignore fallback to skip 8 GB output/scratch); added
`deploy/cloudrun/post_deploy_smoke.ps1`. See [[gpu-vm-setup-gotchas]].
21      - WASB-C3 weights-license gate still open (unchanged; pre-existing).
22
23      ## Live CUJ 2026-07-07 ? release PROVEN, 3 pre-existing gaps surfaced (all filed as
spawned tasks)
24      Owner ran a real /api/process CUJ ("Video 1 - Badminton.MP4", a 5312x2988 **10-bit
HEVC** 120Mbps clip). Outcomes:
25      - **#509 forwarding PROVEN live:** the cloud dispatch put `--set
indexing.wasb.decode_backend=nvdec_resident --set ?batch_size=64 --set ?prefetch_batches=4` into
the worker exec args. And a sharp-clip green run on the D2 worker (exec `rally-worker-ql56s`,
env-driven nvdec) = 19 rallies, success. Serving path is GREEN for valid footage.
26      - **Gap 1 ? slow validation (task_dcee8bf0):** ~125 random `cap.set(POS_FRAMES)` seeks
across scene_cut(100)/blur(15)/relevance(10), each their own VideoCapture, on 5.3K 10-bit HEVC
on the CPU-only control-plane ? ~6.5 min. Fix: single sequential/keyframe pass.
27      - **Gap 2 ? compute=local footgun (task_81572190):** SPA sent compute=local ? in-process
on the weightless control-plane ? "weights not found" fail. Guard/redirect it.
28      - **Gap 3 ? worker re-validates with DEFAULT thresholds (task_72d98f83):** control-plane
min_blur=8.0 (baked) NOT forwarded; worker re-validates at default 20.0 and REJECTED this soft
clip (sharpness 11.9) ? rc2. This is what failed the CUJ on the cloud path. Fix: skip worker re-
validation on dispatch (control-plane is authoritative) or forward validation config.
29      - **Validation-latency design (my rec):** don't move validation to GPU standalone. (1)
stop the double-validation, (2) kill the seeks (sequential/keyframe), (3) end-state = ffprobe
metadata pre-gate on control-plane + fold content checks (blur/court) INTO the worker's existing
GPU decode pass (decode once, validate+detect together). All 3 gaps are pre-existing, NOT caused
by the nvdec release.
30
31      ## (historical) release/rollout phase ? superseded by SHIPPED above
32      PR #507 squash-merged to master as **44a2f47** (branch deleted). Pre-merge gate
reproduced locally: full suite 1764 passed / coverage 85.02% / ruff+mypy clean; CI 11/11 green;
2 optional review follow-ups landed (PyNvVideoCodec==2.1.0 pin in both deploy files;

```

```

decode_backend persisted in native-runner detection_params). Real-L4 validation + golden gate
PASS + latency/resource A/B all done (see PR #507 comment + below). **Remaining = the serving
release (image rebuild + gated nvdec flip); decode_backend still defaults cpu so nothing in prod
has changed yet.**
33
34     ### Serving release plan (live infra: Job `rally-worker` + service `rally-control-
plane`, asia-southeast1; ONE image `rally-worker:latest` drives both ? gen_service_yaml.py:193)
35     - **Phase A (ship image, no behavior change):** `gcloud builds submit
--config=deploy/cloudrun/cloudbuild.yaml --project gen-lang-client-0306026826 .` ? new `rally-
worker:latest` (torch2.6+pynvc2.1.0+NVDEC libs). Push does NOT change prod (Job/service pin a
digest). Cut over via `gcloud run jobs update rally-worker --region asia-southeast1 --image
?:latest` + redeploy service (`gen_service_yaml.py` ? `services replace`). decode_backend
defaults cpu ? cpu path under torch2.6 (validated F1 0.576?0.582). Smoke 1 clip. Rollback: `jobs
update --image <old digest>`.
36     - **Phase B (enable nvdec, gated + reversible):** flip via WORKER env
`WASB_DECODE_BACKEND=nvdec_resident` (native runner reads env first ? no code/preset change,
instant rollback by removing it). ? THE untested risk: NVDEC on the real Cloud Run L4 ? the
image bakes libnvcuvid 580.159.03; Cloud Run's host-driver ABI is unverified there (only tested
on a self-managed L4 VM). MUST smoke 1 clip on the Job before trusting it. Then canary; then
durable default = add `decode_backend: nvdec_resident` to the cloud-serving preset (presets.py)
+ rebuild + Launch Report.
37     - Cost: build ~$0.3; each Job smoke ~$0.4?0.7 (L4). Rollback levers at every step.
38
39
40     ## You are here
41     The implementation is DONE and **PR #507 is open** (branch `feat/506-gpu-resident-
nvdec`, 2026-07-07): default-OFF `indexing.wasb.decode_backend = "cpu"| "nvdec_resident"`, the
validated NVDEC?affine-`grid_sample`?GPU-normalize path wired into
`backend/pipeline/detectors/wasb_infer.py` (`NvdecResidentFrontend` +
`run_video_nvdec_resident`, same resumable cache keyed by decode_backend), native-runner
threading + healthcheck gates (CUDA + PyNvVideoCodec), and the env bump (wasb env ? py3.10 +
torch 2.6.0+cu124 + PyNvVideoCodec; libnvcuvid/libnvidia-encode staged in
`deploy/cloudrun/Dockerfile` + `deploy/digitalocean/gpu_setup.sh`). Verified locally: 1764 tests
green, coverage 85% (floor 70), ruff/mypy/link/leak clean; a 14-agent adversarial review
confirmed 7 minor findings, all fixed (notably `padding_mode="zeros"` = cv2 BORDER_CONSTANT for
non-16:9 sources). Research evidence unchanged: [[gpu-resident-modernization]], issue #506
comments (F1 0.574 ? baseline 0.582 on GX010128 at SERVED, ~1.7x, GPU 47?79%).
42
43     ## Real-L4 validation ? DONE + golden gate PASS 2026-07-07 (this session, ~$3 of the
$100 budget; VM torn down)
44     Validated end-to-end on an L4 (`rally-506`, torn down clean, no orphan disks) ? full
evidence in the PR #507 comment. All GREEN: (1) `gpu_setup.sh` builds the env clean (py3.10 +
torch 2.6.0+cu124 cuda True + PyNvVideoCodec 2.1.0 + NVDEC lib extraction); (2) on-box **parity
gate PASS** (`pytest -k parity`, 480s ? needed test-name fix 9267255, the documented `-k parity`
selected 0 tests); (3) crash/resume byte-identical; (4) cpu?nvdec cache isolation correct; (5)
**Cloud Run image builds + `--gpus all` smoke PASS** (baked NVDEC libs decode; native
healthcheck ok with nvdec_resident); throughput 43.7 fps / GPU 82% on GX010128.
45     **Golden gate @ SERVED = PASS** (10 baseline-comparable clips, 445 rallies): mean
?f1@0.5 **+0.020**, mean ?tolF1 +0.006, worst ?0.009 (**GX010128 must-pass: 0.574 vs 0.582 ?**),
8/10 improved-or-flat. The other 5 golden clips are 4K-sourced (baselines are 1080p) so a vs-
baseline number would conflate resolution with decode ? the parity gate already covers nvdec?cpu
on identical pixels, so they add nothing to the verdict.
46     **Latency + resource A/B** (2nd L4 `rally-506b`, torn down; GX010128, batch 8, telemetry
in `gs://?nightly/dev/506/results/bench/`): nvdec **43.7 fps @ GPU 83% / CPU 30%** vs cpu-
stream(no-prefetch) 15.0 fps @ GPU 28% / CPU 49%; GPU mem +170 MiB (of 24GB), RAM ~2?3 GB flat.
The win is the **bottleneck flip (GPU?, CPU?)**; the 2.9x is vs the untuned streaming path ? vs
prefetch-tuned cpu the realistic end-to-end is ~1.6?1.8x (#506 spike). All in the PR #507
comment.
47
48     ## Staged assets (durable)
49     - **Golden video working set**: all 14 originals in `gs://khelsutra-rally-
corpus/videos/golden/<name>.mp4` + the 4 baseline-source 1080p proxies in `videos/golden_1080p/`
(mahadevpura_2, Badminton_BXH_2, Boxhill_Doubles, GX020094). Frame-count-verified against each
golden trajectory. (GOLDEN_VIDEOS.md could record this GCS location in a follow-up ? kept out of
PR #507 to not muddy its scope.)

```

```

50 - Gate trajectories: `gs://khelsutra-rally-nightly/dev/506/results/golden/*_nvdec.csv`.
Re-score with repo-root `score_golden_gate.py` (untracked dev tool, like `score_local.py`;
labels from `?corpus/labels/`, baselines from `?corpus/trajectories/`).
51
52 ## Next steps
53 1. **Merge PR #507** ? real-L4-validated + golden-gate PASS; CI green locally.
54 2. **Flip the default**: owner Launch Report ? set `decode_backend=nvdec_resident` in
the cloud-serving preset (`backend/config/presets.py`). The gate that gated this is now green.
55
56 ## Ramp-up kit
57 - PR #507 (full change description + verification record) and issue #506 (all comments ?
measured evidence).
58 - New tests: `tests/test_wasb_nvdec_resident.py` (grid/cv2 parity incl. non-16:9, cache
keying, CLI guards, the skippable on-box parity gate).
59 - Reference code still in `scratch/gpu_resident/` (`exp_t2_full.py`, `score_local.py`,
`t2setup_env.sh`, `telemetry.sh`).
60
61 ## Key decisions (don't relitigate)
62 - GPU resize MUST be affine `grid_sample` matching `cv2.warpAffine` ? NOT
`F.interpolate` (F1 0.58?0.07; only shows on hard clips). Locked by unit tests now.
63 - `padding_mode="zeros"` (cv2's BORDER_CONSTANT(0)), not "border" ? matters only for
non-16:9 sources; identical to the validated run on 16:9.
64 - PyNvVideoCodec (MIT) for GPU decode; TorchCodec couldn't load on the DL image.
65 - Score with SERVED preset (cap6+R4); raw (0.02,2.0,2.0) is degenerate.
66 - Default-OFF; nvdec/cpu detection caches never cross-replay (manifest keyed by
decode_backend).
67 - WASB weights/NYCU-dataset license (WASB-C3) is a separate launch gate, not a perf
blocker.
68

```

5. user (2026-07-07T16:57:25.535Z)

```

1 ---
2 name: pr513-worker-skip-revalidation
3 description: "PR #513 MERGED ? worker skips redundant re-validation on the cloud
dispatch path (fixes the 2026-07-07 blur-threshold CUJ failure)"
4 metadata:
5   node_type: memory
6   type: project
7   originSessionId: 902e27a6-e026-4b55-8c26-f2c47329cc44
8 ---
9
10 **PR #513 MERGED** 2026-07-07 (squash **23c1868** on master; CI 11/11 green; LGTM'd by
owner):
11 https://github.com/Khelsutra/badminton-highlight-indexer/pull/513 (branch deleted on the
remote).
12
13 **Bug:** on the cloud-serving path the control-plane validates + only dispatches on
PASS, but the
14 worker's `cmd_infer` RE-RAN validation with its OWN config (the control-plane's resolved
`validation.*`
15 keys aren't forwarded ? [[gpu-resident-modernization]]/#509 forwards only
`skip_ai_handoff` + `wasb.*`).
16 Live CUJ 2026-07-07: a 5312x2988 clip PASSED at `min_blur_threshold=8.0`, worker
REJECTED at default
17 20.0 ? successful GPU dispatch, then `status=failed`.
18
19 **Fix (skip-on-dispatch):** new `ComputeJobSpec.skip_validation` ? `CloudRunCompute`
emits a
20 `--skip-validation` worker flag ? `cmd_infer` honors it (skips `run_all_with_context`) ?
21 `_maybe_dispatch_remote` sets it True on every dispatch. Direct `worker infer` CLI still
validates.
22 Gate: 1780 passed / cov 85.42% / ruff + mypy clean.
23
24 **Rollout caveat (in the PR + README §2a):** `--skip-validation` is a NEW worker CLI arg

```

```

? a
25     control-plane-ahead-of-worker deploy makes an OLD worker image argparse-abort (rc 2)
BEFORE the
26     done-marker ? dispatcher polls the full `remote_poll_max_wait_sec` (~65 min) then fails
opaquely.
27     Deploy **worker image first**, together with the control plane (same coupling #507 has
for
28     `--decode-backend`). Follow-up (task_4e8bc546) ? make the dispatcher fail FAST on any
worker
29     crash-before-marker by consulting the Cloud Run execution status ? is **now running** in
the main
30     checkout on branch `fix/cloud-run-exec-status-fastfail`.
31
32     **? Concurrent-session hazard (why I used an isolated worktree):** a SIBLING session is
actively
33     editing the SHARED checkout `.../khelesutra-guru/badminton-highlight-indexer` on branch
34     `perf/validation-shared-decode-pass` ? a validators shared-decode-pass refactor (base.py
+
35     frame_sampler.py + blur/relevance/scene_cut) AND the "Guard compute=local on the hosted
control-plane"
36     task (orchestration.py `_WEIGHTS_BACKED_SEGMENTERS`). Its tree mutated under me between
commands. I did
37     #513 in a separate `git worktree` at `.../khelesutra-guru/bhi-wt-worker-reval` (still on
disk, branch
38     pushed) so nothing clobbered. **That sibling branch also touches `orchestration.py` +
39     `test_api_cloud_dispatch.py` (**PR #512**), as does #511. #513 LANDED FIRST on those
files, so
40     #511/#512 must now REBASE onto master (23c1868).** See \[\[actual-repo-location\]\] and
41     \[\[validation-shared-decode-pass\]\] / \[\[compute-local-hosted-guardrail\]\].
42
43     **Worktree cleanup:** the isolated worktree `.../khelesutra-guru/bhi-wt-worker-reval` was
de-registered
44     from git and its branch deleted, but the physical dir + `.git/worktrees/bhi-wt-worker-
reval` couldn't
45     be deleted (Windows file lock from the concurrent sibling git activity). Remove manually
when no other
46     session is active: `rm -rf .../bhi-wt-worker-reval && git -C <main> worktree prune`.
47

```

6. user (2026-07-07T16:57:25.886Z)

```

1     ---
2     name: validation-shared-decode-pass
3     description: "Validator frame sampling now shares ONE decode pass; pure grab()-through
is SLOWER than seeks, use forward-hybrid"
4     metadata:
5         node_type: memory
6         type: project
7         originSessionId: d3f3b3dc-c648-47b3-8dc4-5a6d3292c1a7
8     ---
9
10    The three frame-decoding validators (`scene_cut`, `blur`, `relevance` in
11    `backend/pipeline/validators/`) used to each open their OWN `cv2.VideoCapture`
12    and pull samples via per-sample `CAP_PROP_POS_FRAMES` seeks ? ~125 random seeks
13    across 3 decode opens. On long-GOP 4K GoPro clips on the CPU-only control-plane
14    this made "Running validations" take **6m47s** on a 1.41 GiB clip before
15    segmentation (observed live 2026-07-07, job f29d41d?).
16
17    **Fix (**PR #512 MERGED to master 2026-07-07** as squash `20862d7`; dev commit
18    d53851b, branch auto-deleted):** a shared single decode pass
19    `frame_sampler.run_frame_sampling` + a `FrameSamplingValidator` base
20    (`sample_indices`/`_init_state`/`_consume`/`_finalize`); the registry decodes
21    ALL sampled frames ONCE up front and streams them to each validator's online
22    accumulator (one frame held at a time ? do NOT cache ~125 frames; 4K = ~3 GB
23    OOM risk). Frames are byte-identical to the old seeks ? decisions unchanged.

```

```

24
25     **Non-obvious finding ? DON'T "simplify" it to pure `grab()`:** the ticket
26     suggested a pure forward `grab()` pass with no seeking. Measured on the 1.4 GiB /
27     1080p60 / 29175-frame reference clip (`scratch/boxhill_highlights_stage/Adarsh v
28     Vasant.MP4`), pure grab-through was **38s ? SLOWER than the 27?39s seek status
29     quo**, because `blur`/`relevance` sample right up to end-of-file so grabbing to
30     the last sample decodes the WHOLE video. The shipped **forward-hybrid** (grab
31     small gaps, ONE forward `POS_FRAMES` seek to skip big gaps) = **9.57s (~4x)**,
32     126 random seeks ? 17 monotonic-forward. `grab()` is NOT "free" ? it decodes.
33
34     **Relationship to [[pr513-worker-skip-revalidation]] (PR #513):** file-DISJOINT
35     (verified 2026-07-07 via `gh pr view --json files`) ? #512 = validators/ + CODE_MAP
36     + test_frame_sampler; #513 = compute/ + orchestration.py + worker +
test_api_cloud_dispatch
37     / test_compute_backend / test_worker. **No git merge conflict** either order; the
38     "conflict" wording in the index refers to dev-time shared-checkout contention, not a
39     diff conflict. The two are **complementary**: #512 speeds up validation that runs;
40     #513 skips redundant re-validation on the worker dispatch path.
41
42     The remaining floor is decoding `scene_cut`'s dense first--50s region (needed to
43     hit its exact sample indices). Getting to literal "a few seconds" on 4K would
44     need keyframe-only sampling (`-skip_frame nokey`) or a downscaled proxy ? both
45     change WHICH frames are sampled (and would break the same-indices regression in
46     `tests/test_frame_sampler.py`), so they're a separate, decision-affecting change.
47     **Tracked as issue #514** (follow-up to #512) ? owner flagged it important to revisit.
48

```

7. assistant (2026-07-07T16:57:31.908Z)

Excellent context. Now let me navigate to the real repo, check git state, and pull latest before reading code.

8. user (2026-07-07T16:57:35.101Z)

```

=== branch ===
master
=== status ===
?? score_golden_gate.py
?? score_local.py
=== last commit ===
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)

```

9. assistant (2026-07-07T16:57:38.615Z)

Clean tree on master (the two untracked files are known dev tools). Let me fetch and fast-forward.

10. user (2026-07-07T16:57:42.916Z)

```

=== after fetch ===
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
=== local vs origin ===
0      0

```

11. assistant (2026-07-07T16:57:47.560Z)

Up to date with origin/master at `ef41473`. Now let me study the key files. Let me read the orchestration code and supporting files.

12. user (2026-07-07T16:57:48.818Z)

```

1      """Canonical file-identity hashing for the indexer.

```

```

2
3     `video_id` is the MD5 of the *whole* video file. Historically each entrypoint
4     (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
5     `scripts/run_process_and_stitch.py`)
6     defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
7     The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
8     any future change to one copy would silently diverge the DB key.
9
10    This module is the single source of truth. Import `compute_video_id` everywhere a
11    video needs to be keyed.
12    """
13    from __future__ import annotations
14
15    import hashlib
16
17    # 64 KB matches the value used by the live pipeline (backend/main.py) before this
18    # was centralized. The chunk size does not affect the resulting digest ? it only
19    # bounds memory use while streaming large files.
20    _CHUNK_SIZE = 65536
21
22
23    def compute_video_id(filepath: str) -> str:
24        """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26        Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27        into memory at once.
28        """
29        hasher = hashlib.md5()
30        with open(filepath, "rb") as f:
31            for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
32                hasher.update(chunk)
33        return hasher.hexdigest()
34

```

13. user (2026-07-07T16:57:48.954Z)

```

1     """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
2     #390).
3
4     The full hash -> validate -> segment -> report pipeline (`_execute_processing`), the
5     Cloud Run dispatch + ingest path (`_maybe_dispatch_remote` /
6     `_ingest_remote_segments`),
7     and the shared compile resolution + worker handler (`_resolve_compile` /
8     `_execute_compile`).
9     Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
10    unchanged.
11
12    SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
13    remain
14    monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
15    ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
16    through
17    the ``backend.main`` module object at call time (`import backend.main as _main`),
18    never as a
19    bare local. ``backend.main`` re-exports every name below, so
20    ``monkeypatch.setattr(backend.main,
21    ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
22    unchanged.
23    This module must NOT import ``backend.main`` at module level (main imports us for the
24    re-export).
25    Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
26    ``db_instance``,
27    ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
28    Object-attribute patches (`m.storage.fetch`, `m.segmenter_registry.get`, ...) hit

```



```

the shared
18     objects and need no routing.
19     """
20
21     import csv
22     import json
23     import logging
24     import math
25     import os
26     import re
27     import tempfile
28     import threading
29     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30     from urllib.parse import quote
31
32     from backend import storage
33     from backend.config import AppConfig, StitchingConfig
34     from backend.pipeline.segmenters.base import segmenter_registry
35     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
36     from backend.pipeline.validators.base import registry
37     from backend.results import Failure
38     from backend.utils.paths import output_base, safe_output_filename
39
40     if (
41         TYPE_CHECKING
42     ): # runtime import would be circular (main imports us for the re-export)
43         from backend.main import ProcessRequest # noqa: F401
44
45     logger = logging.getLogger(__name__)
46
47     # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
48     # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
49     # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
50     # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
51     # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
52     # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
53     _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
54
55
56     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
57         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
58
59         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
60         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
61         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
62         """
63         if segments:
64             db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
65         else:
66             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
67
68
69     def _validation_failure_message(failures: List[Any]) -> str:
70         details = []
71         for failure in failures:
72             name = getattr(failure, "validator_name", None) or "validation"
73             message = (getattr(failure, "message", None) or "").strip()
74             details.append(f"{name}: {message}" if message else str(name))

```

```

75         if not details:
76             return "Video failed validation checks."
77         return "Video failed validation: " + "; ".join(details)
78
79
80     def _report_config_for_input(config: Any, raw_input: str) -> Any:
81         """Return report config that keeps remote-input reports out of disposable
82         scratch."""
83         storage_config = (
84             config.get("storage")
85             if isinstance(config, dict)
86             else getattr(config, "storage", None)
87         )
88         try:
89             if storage.input_is_local(raw_input, storage_config):
90                 return config
91         except ValueError:
92             return config
93         try:
94             cfg = (
95                 config.model_dump(mode="json")
96                 if hasattr(config, "model_dump")
97                 else dict(config or {})
98             )
99         except Exception:
100             cfg = {}
101         report = dict(cfg.get("report") or {})
102         if report.get("output_dir"):
103             return config
104         report["output_dir"] = os.path.join(output_base(config), "reports")
105         cfg["report"] = report
106         return cfg
107
108     def _ingest_remote_segments(
109         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110     ) -> int:
111         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
112         the bucket and
113         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
114         call ? so
115         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
116         were produced.
117         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
118         pattern (cloud
119         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
120         malformed row is
121         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
122         temp dir is
123         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
124         a job failure
125         and prunes any partial rows (destroy-AFTER-confirm)."""
126         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
127         ref = storage.parse_uri(uri)
128         n = 0
129         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
130             local = storage.fetch(
131                 uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
132             )
133             with open(local, newline="", encoding="utf-8") as f:
134                 for row in csv.DictReader(f):
135                     if n >= max_rows:
136                         logger.warning(
137                             "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
138                             rest.",

```

```

131         max_rows,
132         uri,
133     )
134     break
135     try:
136         start, end = float(row["start"]), float(row["end"])
137     except (KeyError, TypeError, ValueError):
138         continue
139     if not (math.isfinite(start) and math.isfinite(end)):
140         continue # reject inf/nan times rather than persist a junk rally
141     meta: Dict[str, Any] = {
142         "source": "cloud_run",
143         "ending_reason": (row.get("ending_reason") or "").strip(),
144     }
145     sc = (row.get("shot_count") or "").strip()
146     if sc:
147         try:
148             meta["shot_count"] = int(
149                 float(sc)
150             ) # OverflowError on 'inf'/'1e400'
151         except (ValueError, OverflowError):
152             meta["shot_count"] = sc
153     db.add_play_segment(video_id, start, end, metadata=meta)
154     n += 1
155     return n
156
157
158     # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
159     # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
160     # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
161     # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
162     # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
163     _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
164
165
166     def _local_detector_weights_present(cfg, seg_name: str) -> bool:
167         """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
168         on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
169         and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
170         otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
171         A non-weights segmenter returns True (nothing local to check)."""
172         name = (seg_name or "").strip().lower()
173         if name not in _WEIGHTS_BACKED_SEGMENTERS:
174             return True
175         idx = getattr(cfg, "indexing", None)
176         substrate = (
177             getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
178         ).strip().lower()
179         if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet")):
180             path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
181         else: # wasb_hybrid, or fusion on the (default) WASB substrate
182             path = (
183                 os.environ.get("WASB_WEIGHTS")
184                 or getattr(getattr(idx, "wasb", None), "weights_path", "")

```

```

185         or ""
186     )
187     return bool(path) and os.path.exists(os.path.expanduser(path))
188
189
190     def _cloud_available(cfg) -> bool:
191         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
192 3).
193         True when the control plane is wired for Cloud Run: either the configured default
194 target is
195         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
196 coordinates
197         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
198         (``storage.output_root``)
199         are present so a forced per-request override can actually dispatch + collect a
200 result.
201         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
202 would work."""
203         deployment = getattr(cfg, "deployment", None)
204         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
205             return True
206         has_job = bool(
207             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
208         )
209         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
210         return bool(has_job and out_root)
211
212
213     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
214         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
215 minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
216 comes
217         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
218 "starting"
219         (queue/cold-start), so the very first tick already shows movement."""
220         minutes = int(elapsed_sec) // 60
221         if minutes < 1:
222             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
223         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
224
225
226     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
227         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
228 gs://
229         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
230 keep
231         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
232 no
233         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
234         if getattr(input_ref, "backend", "") != "gcs":
235             return None
236         sc = _storage_settings(cfg)
237         if not sc:
238             return None
239         try:
240             st = storage.get_storage("gcs", config=sc).stat(input_ref)
241             return getattr(st, "md5", None) or None
242         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
243             logger.info(
244                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
245                 getattr(input_ref, "uri", input_ref),
246                 exc_info=True,
247             )

```

```

239         return None
240
241
242     def _cached_process_payload(
243         video_id: str, validation_results, segments, requested_compute
244     ) -> Dict[str, Any]:
245         """The /api/process result for a video whose detection already exists ? ONE shape
for
246         both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
247         return {
248             "video_id": video_id,
249             "status": "success",
250             "message": (
251                 f"Reused {len(segments)} cached play segments. "
252                 "Pass force=true to reprocess this video."
253             ),
254             "results": validation_results or [],
255             "play_segments": segments,
256             "cached": True,
257             "compute": {"path": "cached", "requested": requested_compute},
258         }
259
260
261     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
262         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
263         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
264         ref = storage.parse_uri(uri)
265         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
266             local = storage.fetch(
267                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
268             )
269             with open(local, encoding="utf-8") as f:
270                 loaded = json.load(f)
271             return loaded if isinstance(loaded, dict) else None
272
273
274     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
275     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
276     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
277     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
278     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
279     _REHYDRATE_LOCKS_GUARD = threading.Lock()
280
281
282     def _rehydrate_lock(video_id: str) -> threading.Lock:
283         with _REHYDRATE_LOCKS_GUARD:
284             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
285
286
287     def probe_process_cache(db, cfg, video_id: str) -> bool:
288         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
289
290         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
291         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
292         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
293         errors report False (? the normal dispatch path with its guards)."""
294         try:
295             existing = db.get_video(video_id)

```

```

296         if (
297             existing
298             and existing.get("status") == "processed"
299             and db.query_play_segments(video_id, {}))
300         ):
301             return True
302         sc = _storage_settings(cfg)
303         if not sc or (sc.get("backend") or "local") == "local":
304             return False
305         output_root = (sc.get("output_root") or "").rstrip("/")
306         if not output_root:
307             return False
308         report_uri = f"{output_root}/outputs/{video_id}/report.json"
309         if not storage.exists(report_uri, config=sc):
310             return False
311         report = _read_bucket_json(report_uri, sc)
312         return bool(report and report.get("status") == "success")
313     except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
314         logger.warning(
315             "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
316         )
317         return False
318
319
320     def cached_process_result(
321         db, cfg, video_id: str, video_path: str, requested_compute=None
322     ) -> Optional[Dict[str, Any]]:
323         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
324         the DB first, then the bucket (`outputs/<md5>/report.json` with `status=success`
325         re-ingests via `_ingest_remote_segments`): **the DB is a cache, the bucket is the
326         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). `None` ?
no
327         completed work found (or the check failed) ? the caller dispatches normally. The
bucket
328         branch mirrors the real cloud ingest exactly (watermarks + `_finalize_reingest`),
so a
329         partial earlier ingest is replaced, never double-counted.
330
331         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
332         per-`video_id` lock ? a concurrent caller blocks, then finds the winner's rows via
the
333         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
334         job before invoking (writes happen post-claim)."""
335         try:
336             with _rehydrate_lock(video_id):
337                 existing = db.get_video(video_id)
338                 if existing and existing.get("status") == "processed":
339                     segs = db.query_play_segments(video_id, {})
340                     if segs:
341                         return _cached_process_payload(
342                             video_id,
343                             existing.get("validation_results", []),
344                             segs,
345                             requested_compute,
346                         )
347                 sc = _storage_settings(cfg)
348                 if not sc or (sc.get("backend") or "local") == "local":
349                     return None
350                 output_root = (sc.get("output_root") or "").rstrip("/")
351                 if not output_root:
352                     return None
353                 outputs_uri = f"{output_root}/outputs/{video_id}"

```

```

354         report_uri = f"{outputs_uri}/report.json"
355         if not storage.exists(report_uri, config=sc):
356             return None
357         report = _read_bucket_json(report_uri, sc)
358         if not report or report.get("status") != "success":
359             return None
360         if not video_path:
361             # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
362             # carries the worker's --video-uri (P3); older reports fall back to the
363             # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
364             # recover, and the next same-md5 submit re-binds the truthful path via
365             # add_video's upsert.
366             video_path = str(report.get("video_uri") or "") or _first_object_under(
367                 f"{output_root}/videos/{video_id}/", sc
368             )
369             play_wm, cand_wm = db.segment_watermarks(video_id)
370             db.add_video(video_id, video_path, "processed", [])
371             n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
372             if n <= 0:
373                 return None
374             segments = [
375                 s
376                 for s in db.query_play_segments(video_id, {})
377                 if int(s.get("id", 0)) > play_wm
378             ]
379             _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
380             if not segments:
381                 return None
382             logger.info(
383                 "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
384                 n,
385                 video_id,
386                 outputs_uri,
387             )
388             return _cached_process_payload(video_id, [], segments, requested_compute)
389     except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
390         logger.warning(
391             "[API] submit-time cache check failed for %s ? dispatching normally",
392             video_id,
393             exc_info=True,
394         )
395     return None
396
397
398     def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
399         """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
400         try:
401             return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
402         except Exception: # noqa: BLE001 ? recovery nicety, never a failure
403             return ""
404
405
406     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
407
408
409     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
410         """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
411         nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/` ` ?
so
412         ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
DB
413         has the video's rows now (re-query). Non-md5 ids return False without touching the

```

```

414         bucket; every failure degrades to False (the read just reports what the DB has)."""
415         vid = (video_id or "").strip().lower()
416         if not _MD5_HEX_RE.fullmatch(vid):
417             return False
418         if not probe_process_cache(db, cfg, vid):
419             return False
420         return cached_process_result(db, cfg, vid, "") is not None
421
422
423     def _maybe_dispatch_remote(
424         cfg,
425         db,
426         req: "ProcessRequest",
427         video_id: str,
428         seg_name: str,
429         val_results,
430         play_wm: int,
431         cand_wm: int,
432         notify,
433     ) -> Optional[Tuple[Dict[str, Any], bool]]:
434         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
435         ``deployment.compute_target
436         == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
437         intervals into
438         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
439
440         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
441         the cloud job
442         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
443         observability report
444         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
445         process segmenter
446         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
447         non-``cloud_run`` target, so
448         the in-process path stays byte-identical)."""
449         from backend.compute import ComputeJobSpec, get_compute_backend
450
451     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
452         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
453         (id > play_wm);
454         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
455         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
456         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
457         only when the
458         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
459         rc / ingest
460         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
461         records the
462         # intended backend, `requested` the picker choice, `dispatched` whether the
463         remote job ran.
464         return (
465             {
466                 "video_id": video_id,
467                 "status": "failed",
468                 "message": msg,
469                 "results": [r.model_dump() for r in val_results],
470                 "play_segments": [],
471                 "compute": {
472                     "path": "cloud_run" if ran else "not_run",
473                     "requested": req.compute,
474                     "target": "cloud_run",
475                     "dispatched": ran,
476                 },
477             },
478             ran,
479         )

```



```

468         )
469
470         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
471         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
472         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
473         choice = (req.compute or "").strip().lower()
474         if choice and choice not in ("local", "cloud"):
475             logger.warning(
476                 "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
477                 req.compute,
478             )
479             choice = ""
480         if choice == "local":
481             # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
482             # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
483             # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
484             # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
485             # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
486             # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
487             # below exactly as before (the weights-present check is the escape hatch).
488             if (
489                 (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
490                 and _cloud_available(cfg)
491                 and not _local_detector_weights_present(cfg, seg_name)
492             ):
493                 return _failed(
494                     f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
495                     "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'." ,
496                     False,
497                 )
498             return None # explicit "run on my machine" ? in-process regardless of the
server's target
499         if choice == "cloud":
500             if not _cloud_available(cfg):
501                 return _failed(
502                     "cloud-serving was requested but this server isn't configured for it "
503                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
504                     False,
505                 )
506             compute = get_compute_backend(cfg, target_override="cloud_run")
507         else:
508             compute = get_compute_backend(
509                 cfg
510             ) # server default (default-OFF unless cloud_run)
511         if compute is None:
512             return None # default-OFF ? run the in-process segmenter below (unchanged)
513
514         storage_cfg = getattr(cfg, "storage", None)
515
516         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
517         try:

```

```

518         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
519     except ValueError as e:
520         return _failed(
521             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
522         )
523     if input_ref.backend == "local":
524         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
525         # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
526         # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
527         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
528         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
529         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
530         # default, while a bare/local-looking root can't stage no matter what
input_backend says
531         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
532         # gives us nothing to build the staged URI under).
533         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
534         try:
535             root_backend = (
536                 storage.parse_uri(input_root).backend if input_root else "local"
537             )
538         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
539             root_backend = "local"
540         if root_backend == "local":
541             return _failed(
542                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
543                 "stage local uploads into; a path local to this machine can't be read by
the Cloud "
544                 "Run job.",
545                 False,
546             )
547         local_src = input_ref.key # resolved local path of the upload
548         staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
549         try:
550             notify("staging upload to bucket")
551             # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
552             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
553             storage.put(
554                 local_src,
555                 staged_uri,
556                 config=(
557                     storage_cfg.model_dump() if storage_cfg is not None else None
558                 ),
559             )
560         except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
561             logger.error(
562                 "[API] cloud-serving: failed to stage %s -> %s: %s",
563                 local_src,
564                 staged_uri,
565                 e,
566                 exc_info=True,
567             )

```

```

568         return _failed(
569             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
570             False,
571         )
572     # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
573     # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
574     # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
575     req.video_path = staged_uri
576     input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
577     db.add_video(
578         video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
579     )
580     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
581     if not out_root:
582         return _failed(
583             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
584             "? that's where the Cloud Run job writes the result.",
585             False,
586         )
587
588     notify("dispatching (cloud_run)")
589     logger.info(
590         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
591         req.video_path,
592         out_root,
593     )
594     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
595     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
596     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
597     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
598     overrides = []
599     idx_cfg = getattr(cfg, "indexing", None)
600     sk = getattr(idx_cfg, "skip_ai_handoff", None)
601     if sk is not None:
602         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
603     # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
604     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
605     # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
606     # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
607     # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
608     # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
609     wasb_cfg = getattr(idx_cfg, "wasb", None)
610     if wasb_cfg is not None:
611         decode_backend = getattr(wasb_cfg, "decode_backend", None)
612         if decode_backend:
613             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
614         batch_size = getattr(wasb_cfg, "batch_size", None)
615         if batch_size:
616             overrides.append(f"indexing.wasb.batch_size={batch_size}")
617         prefetch = getattr(wasb_cfg, "prefetch_batches", None)

```

```

618         if prefetch:
619             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
620         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
621         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
622         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
623         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
624         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
625         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
626         spec = ComputeJobSpec(
627             video_uri=req.video_path,
628             out_uri=out_root,
629             segmenter=seg_name,
630             config_overrides=tuple(overrides),
631             skip_validation=True,
632         )
633         try:
634             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
635             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
636             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
637             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
638             result = compute.run_infer(
639                 spec,
640                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
641             )
642         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
643             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
644             return _failed(f"cloud dispatch error: {e}", True)
645         if result.returncode != 0:
646             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
647         if result.video_id and result.video_id != video_id:
648             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
649             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
650             logger.warning(
651                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
652                 "and the worker hash; ingesting under the local id.",
653                 result.video_id,
654                 video_id,
655             )
656
657         notify("ingesting (cloud_run)")
658         storage_dict = (
659             storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
660             if hasattr(storage_cfg, "model_dump")
661             else {}
662         )
663         try:
664             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
665         except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
666             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
667             return _failed(
668                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
669                 True,

```

```

670         )
671
672         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
673         segments = [
674             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
675         ]
676         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
677         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
678         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
679         provenance = {
680             "path": "cloud_run",
681             "requested": req.compute,
682             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
683             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
684             "execution": result.execution,
685             "outputs_uri": result.outputs_uri,
686         }
687         logger.info(
688             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
689             result.execution,
690             provenance["job"],
691             provenance["region"],
692             result.outputs_uri,
693             video_id,
694         )
695         return (
696             {
697                 "video_id": video_id,
698                 "status": "success",
699                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
700                 "results": [r.model_dump() for r in val_results],
701                 "play_segments": segments,
702                 "compute": provenance,
703             },
704             True,
705         )
706
707
708     def _execute_processing(
709         config=None, db=None, req=None, progress=None
710     ) -> Dict[str, Any]:
711         """The full hash ? validate ? segment ? report pipeline for one video.
712
713         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
714         module-level ``global_config``/``db_instance`` (backward-compatible with existing
715         tests that pass only ``req``). Route handlers always pass explicit params; older
716         test code that calls ``_execute_processing(req)`` still works.
717
718         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
719         Raises on unexpected internal errors ? the caller records those as a job failure
720         """
721         import backend.main as _main # call-time seam: main's globals/model stay patchable
722
723         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
724         # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
725         if config is not None and not isinstance(config, AppConfig):
726             # Old signature: _execute_processing(req, progress=None)
727             if req is None and db is not None and isinstance(db, _main.ProcessRequest):

```

```

728         req = db
729         db = None
730         elif req is None and isinstance(config, _main.ProcessRequest):
731             req = config
732             config = None
733     # Resolve config/db from module globals when not passed explicitly
734     if config is None:
735         config = _main.global_config
736     if db is None:
737         db = _main.db_instance
738
739     notify = progress or (lambda stage: None)
740
741     notify("hashing")
742     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
743     # (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
744     # original # req.video_path (the source URI) is kept for the DB record + report; only the file-
745     # reading # consumers (hash / validators / segmenter) use the resolved local path.
746     local_video = ""
747     try:
748         local_video = storage.acquire_local_input(
749             req.video_path, getattr(config, "storage", None)
750         )
751         video_id = _main.compute_video_id(local_video)
752     except Exception:
753         storage.cleanup_acquired_local_input(
754             req.video_path, local_video, getattr(config, "storage", None)
755         )
756         raise
757     try:
758         existing_video = db.get_video(video_id)
759     except Exception:
760         storage.cleanup_acquired_local_input(
761             req.video_path, local_video, getattr(config, "storage", None)
762         )
763         raise
764     was_reingest = existing_video is not None
765
766     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
767     default_seg = getattr(
768         getattr(config, "indexing", None), "default_segmenter", "motion"
769     )
770     seg_name = req.segmenter or default_seg
771     segmenter_actual = None
772     segmentation_ran = False
773     val_results: List[Any] = []
774     val_context: Dict[str, Any] = {}
775     # Compute-path PROVENANCE (the validation method): which backend actually ran the
776     # "cloud_run" by # "in_process" once the local segmenter is reached, set on the response as
777     # # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
778     # fail).
779     compute_actual: Optional[str] = None
780     response: Optional[Dict[str, Any]] = None
781     try:
782         if (
783             existing_video
784             and existing_video.get("status") == "processed"
785             and not req.force
786         ):
787             cached_segments = db.query_play_segments(video_id, {})

```

```

787         if cached_segments:
788             logger.info(
789                 "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
790                 video_id,
791             )
792             response = _cached_process_payload(
793                 video_id,
794                 existing_video.get("validation_results", []),
795                 cached_segments,
796                 getattr(req, "compute", None),
797             )
798             return response
799
800     # 1. Run pluggable validators (with-context variant surfaces fprobe_meta for
the report)
801     notify("validating")
802     logger.info(f"Running validations for {req.video_path}...")
803     val_results, val_context = registry.run_all_with_context(local_video, config)
804     failures = [r for r in val_results if not r.passed]
805
806     if failures:
807         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
808         # status; it no longer destroys the video's prior good segments.
809         db.add_video(
810             video_id,
811             req.video_path,
812             "failed",
813             [r.model_dump() for r in val_results],
814         )
815         response = {
816             "video_id": video_id,
817             "status": "failed",
818             "message": _validation_failure_message(failures),
819             "results": [r.model_dump() for r in val_results],
820         }
821         return response
822
823     # 2. Run segmenter & indexer
824     logger.info("[API] Video passed checks. Segmenting and indexing...")
825     db.add_video(
826         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
827     )
828
829     segmenter_cls = segmenter_registry.get(seg_name)
830     if not segmenter_cls:
831         # Submit-time validation already 400s unknown names; this is the defensive
832         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
833         available = list(segmenter_registry.list_available().keys())
834         response = {
835             "video_id": video_id,
836             "status": "failed",
837             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
838             "results": [r.model_dump() for r in val_results],
839             "play_segments": [],
840         }
841         return response
842
843     logger.info(f"[API] Using segmenter: {seg_name}")
844     notify(f"segmenting ({seg_name})")
845     segmenter_actual = seg_name
846     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only

```

```

847         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
848         play_wm, cand_wm = db.segment_watermarks(video_id)
849         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
850         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
851         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
852         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
853         # they're dropped on the cloud path in this slice ? a documented follow-up.)
854         remote = _maybe_dispatch_remote(
855             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
856         )
857         if remote is not None:
858             response, segmentation_ran = (
859                 remote # `ran` = did the cloud job actually execute (report)
860             )
861             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
862         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
863         compute_actual = "in_process"
864         segmenter = segmenter_cls(db, config)
865         try:
866             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
867             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
868             with _LOCAL_COMPUTE_LANE:
869                 result = segmenter.process_video(
870                     video_id, local_video, req.player_pool, req.max_frames
871                 )
872         except Exception:
873             # A raising segmenter must not leave half-written rows: restore the pre-run
874             # state (same destroy-after-confirm contract as the Failure branch), then
let
875             # the job worker record the failure.
876             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
877             raise
878         segmentation_ran = True
879
880         if isinstance(result, Failure):
881             logger.error(f"[API] Segmenter failed: {result.message}")
882             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
883             response = {
884                 "video_id": video_id,
885                 "status": "failed",
886                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
887                 "results": [r.model_dump() for r in val_results],
888                 "play_segments": [],
889             }
890             return response
891
892         segments = result.value if hasattr(result, "value") else result
893         notify("finalizing")
894         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
895
896         response = {
897             "video_id": video_id,
898             "status": "success",
899             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
.value

```



```

900         "results": [r.model_dump() for r in val_results],
901         "play_segments": segments,
902     }
903     return response
904 finally:
905     # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
906     # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
907     # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
908     # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
909     if isinstance(response, dict):
910         response.setdefault(
911             "compute",
912             {
913                 "path": compute_actual or "not_run",
914                 "requested": getattr(req, "compute", None),
915             },
916         )
917     prov = response.get("compute", {}) if isinstance(response, dict) else {}
918     logger.info(
919         "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
920         video_id,
921         getattr(req, "compute", None),
922         prov.get("path"),
923         prov.get("dispatched"),
924         (response or {}).get("status") if isinstance(response, dict) else None,
925     )
926     # Always emit the per-video observability report (next to the video, or to
927     # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
928     # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
929     # doesn't surface arbitrary run signals.)
930     paths = _main.emit_indexer_report(
931         db=db,
932         video_id=video_id,
933         # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
934         # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
935         # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
936         # still preserved in DB provenance via db.add_video.
937         video_path=local_video,
938         config=_report_config_for_input(config, req.video_path),
939         val_results=val_results,
940         val_context=val_context,
941         segmenter_requested=req.segmenter,
942         segmenter_actual=segmenter_actual,
943         was_reingest=was_reingest,
944         segmentation_ran=segmentation_ran,
945     )
946     if paths and isinstance(response, dict):
947         response["reports"] = paths
948     storage.cleanup_acquired_local_input(
949         req.video_path, local_video, getattr(config, "storage", None)
950     )
951
952
953 def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
954     """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
955

```

```

956         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
957         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
958         to a no-op rather than blocking a job it cannot measure."""
959         try:
960             sc = (
961                 storage_config.model_dump()
962                 if hasattr(storage_config, "model_dump")
963                 else (storage_config or {})
964             )
965             backend = storage.get_storage(input_ref.backend, config=sc)
966             return int(backend.stat(input_ref).size)
967         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
not block
968             logger.debug(
969                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
970             )
971             return None
972
973
974     class _CompileError(Exception):
975         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
976         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
977
978         def __init__(self, status_code: int, detail: str):
979             super().__init__(detail)
980             self.status_code = status_code
981             self.detail = detail
982
983
984     def _resolve_compile(
985         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
986     ):
987         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
988         row could change between submit and run). Verifies the video exists, the requested
segments match
989         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
990         so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
991         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
992         video = db.get_video(video_id)
993         if not video:
994             raise _CompileError(404, "Indexed video record not found.")
995         # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
996         try:
997             out_name = safe_output_filename(output_filename)
998         except ValueError as e:
999             raise _CompileError(400, str(e)) from e
1000
1001         all_segments = db.query_play_segments(video_id, {})
1002         matched = [s for s in all_segments if s["id"] in segment_ids]
1003         if not matched:
1004             raise _CompileError(400, "No matching play segments found to stitch.")
1005
1006         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1007         kept = []
1008         for s in matched:
1009             meta = s.get("metadata") or {}

```

```

1010         sc = meta.get("shot_count") if isinstance(meta, dict) else None
1011         try:
1012             if sc is not None and int(sc) < stitching.min_shot_count:
1013                 continue
1014         except (TypeError, ValueError):
1015             pass # unparseable count -> treat as unknown, keep
1016         kept.append(s)
1017     if len(kept) < len(matched):
1018         logger.info(
1019             "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
1020             floor.",
1021             stitching.min_shot_count,
1022             len(matched) - len(kept),
1023             len(matched),
1024         )
1025     if not kept:
1026         raise _CompileError(
1027             400,
1028             f"All {len(matched)} matching segments fall below "
1029             f"stitching.min_shot_count={stitching.min_shot_count}.",
1030         )
1031     return video, kept, out_name
1032
1033     def reel_object_uri(cfg, video_id: str, name: str) -> str:
1034         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
1035         (#463), or "" when
1036         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
1037         compile
1038         mirror-upload and the /api/highlights redirect:
1039         ``<output_root>/highlights/<video_id>/<name>``. """
1040         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1041         if not root or "://" not in root:
1042             return ""
1043         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1044
1045     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1046         """The active storage settings dict (per-backend config for the storage layer ? S3
1047         endpoint/region,
1048         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
1049         default backend
1050         isn't silently constructed as a fresh default. """
1051         sc = getattr(cfg, "storage", None)
1052         return sc.model_dump() if sc is not None else None
1053
1054     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1055         """A short-lived signed URL for a reel mirrored to the output bucket, so
1056         /api/highlights can
1057         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
1058         ephemeral local
1059         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
1060         config for BOTH
1061         the existence check and the signing (extracted here to keep backend/main.py lean ?
1062         P23/P24). """
1063         reel_uri = reel_object_uri(cfg, video_id, name)
1064         if not reel_uri:
1065             return None
1066         settings = _storage_settings(cfg)
1067         if storage.exists(reel_uri, config=settings):
1068             return storage.signed_url(reel_uri, config=settings, method="GET")
1069         return None

```

```

1065     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1066         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
1067         stitch the reel,
1068         and return the {status, filepath, download_url} result the SPA polls for. Returns a
1069         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
1070         stitch error)
1071         so the SPA always gets a clean verdict instead of a worker-crash error."""
1072         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
1073         backend.main
1074         notify = progress or (lambda stage: None)
1075         params = json.loads(job.get("params") or "{}")
1076         video_id = job.get("video_id") or ""
1077         segment_ids = params.get("segment_ids") or []
1078         locale = params.get("locale")
1079
1080         notify("resolving")
1081         try:
1082             video, kept, out_name = _resolve_compile(
1083                 db, cfg, video_id, segment_ids, params.get("output_filename", "")
1084             )
1085         except _CompileError as e:
1086             return {"status": "failed", "message": e.detail, "video_id": video_id}
1087
1088         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1089         output_dir = (
1090             getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1091         )
1092         # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
1093         the job);
1094         # unknown/absent locale keeps the configured default unchanged.
1095         localized_watermark = localize_watermark(stitching.watermark, locale)
1096         compiler = _main.VideoCompiler(
1097             output_dir,
1098             begin_padding=stitching.begin_padding,
1099             end_padding=stitching.end_padding,
1100             watermark=localized_watermark,
1101         )
1102         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1103
1104         notify("stitching")
1105         local_src = ""
1106         try:
1107             # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
1108             path). Resolve to
1109             # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
1110             scratch for a URI (the
1111             # same seam the process path uses); without it, compiling a bucket-ingested
1112             video can't open gs://.
1113             local_src = storage.acquire_local_input(
1114                 video["filepath"], getattr(cfg, "storage", None)
1115             )
1116         except Exception as e:
1117             # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
1118             (for remote
1119             # dispatch) doesn't run multiple compiles at once on the box (see
1120             _LOCAL_COMPUTE_LANE).
1121             with _LOCAL_COMPUTE_LANE:
1122                 out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1123             except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
1124                 FAILURE, not a crash
1125                 logger.error(
1126                     "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1127                 )
1128             return {
1129                 "status": "failed",

```

```

1120         "message": f"Highlight stitching failed: {e}",
1121         "video_id": video_id,
1122     }
1123     finally:
1124         if local_src:
1125             storage.cleanup_acquired_local_input(
1126                 video["filepath"], local_src, getattr(cfg, "storage", None)
1127             )
1128
1129     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1130     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1131     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1132     # still falls back to the local file on the same instance).
1133     reel_uri = reel_object_uri(cfg, video_id, out_name)
1134     if reel_uri:
1135         try:
1136             notify("uploading reel")
1137             # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1138             # project/client) is used, not a fresh default.
1139             storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1140         except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1141             logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1142
1143     # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1144     # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1145     download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1146     notify("finished")
1147     return {
1148         "status": "success",
1149         "filepath": out_path,
1150         "download_url": download_url,
1151         "video_id": video_id,
1152     }
1153

```

14. assistant (2026-07-07T16:58:00.114Z)

Now I understand the core flow. The whole-pipeline cache is at lines 781-798; validation runs at line 803. I need to insert a validation-result cache between them. Let me study the validators registry, config, DB layer, and the ValidationResult model.

15. user (2026-07-07T16:58:02.126Z)

```

1     import logging
2     from abc import ABC, abstractmethod
3     from typing import Any, List
4
5     from pydantic import BaseModel
6
7     from backend.pipeline.validators.frame_sampler import (
8         FRAME_META_KEY,
9         FRAME_STATES_KEY,
10        SampleMeta,
11        run_frame_sampling,
12    )
13
14    logger = logging.getLogger(__name__)
15
16

```

```

17 class ValidationResult(BaseModel):
18     validator_name: str
19     passed: bool
20     message: str
21     details: dict[str, Any] = {}
22
23
24 class VideoValidator(ABC):
25     @property
26     @abstractmethod
27     def name(self) -> str:
28         """Unique identifier for this validator."""
29         pass
30
31     @property
32     @abstractmethod
33     def description(self) -> str:
34         """Human-readable explanation of what this validator checks."""
35         pass
36
37     @abstractmethod
38     def validate(
39         self, video_path: str, config: Any, context: dict[str, Any]
40     ) -> ValidationResult:
41         """
42         Executes validation.
43         - video_path: Local path to the MP4 file.
44         - config: AppConfig (typed) ? callers pass the typed model; attribute access
45           preferred over .get() dict chains.
46         - context: Dynamic context dict shared across validators (stores shared values
47           like ffmpeg metadata).
48         """
49         pass
50
51
52 class FrameSamplingValidator(VideoValidator):
53     """A validator whose work is sampling decoded frames.
54
55     The three frame-decoding validators (scene_cut, blur, relevance) used to each
56     open their own ``cv2.VideoCapture`` and seek to their sample frames ? ~125
57     random ``POS_FRAMES`` seeks across three decode opens, which cost minutes on
58     long-GOP 4K clips (see ``frame_sampler``). Subclasses instead declare *which*
59     frame indices they want (:meth:`sample_indices`) and reduce those frames
60     online (:meth:`_init_state` / :meth:`_consume`), so the registry can decode
61     the whole set **once** in a single shared forward pass.
62
63     :meth:`validate` uses the shared pass' result when the registry ran it; when
64     a subclass is invoked standalone (direct ``.validate`` call, no shared
65     context) it decodes its own indices in one pass via the same engine. Either
66     way the frames ? and therefore the pass/fail decision ? are identical to the
67     old per-validator seeks.
68     """
69
70     @abstractmethod
71     def sample_indices(self, total_frames: int, fps: float) -> List[int]:
72         """Absolute frame indices this validator samples, in ascending order.
73
74         Pure and deterministic in ``(total_frames, fps)`` ? the shared pass unions
75         these across validators to drive one decode, and the regression suite pins
76         them to the legacy per-validator seek arithmetic.
77         """
78         ...
79
80     @abstractmethod
81     def _init_state(self, config: Any) -> Any:

```

```

82         """Fresh accumulator for one run (may pre-resolve config-derived params)."""
83         ...
84
85     @abstractmethod
86     def _consume(self, state: Any, frame_idx: int, frame: Any) -> None:
87         """Fold one sampled frame (BGR ndarray) into ``state``. Called in index
order."""
88         ...
89
90     @abstractmethod
91     def _finalize(
92         self, state: Any, meta: SampleMeta, config: Any, context: dict[str, Any]
93     ) -> ValidationResult:
94         """Render the ValidationResult from the accumulated ``state`` + decode meta."""
95         ...
96
97     def validate(
98         self, video_path: str, config: Any, context: dict[str, Any]
99     ) -> ValidationResult:
100         states = context.get(FRAME_STATES_KEY)
101         if states is not None and self.name in states:
102             # Shared single pass already decoded and reduced our frames.
103             meta = context[FRAME_META_KEY]
104             state = states[self.name]
105         else:
106             # Standalone: decode just this validator's indices in one pass.
107             meta, own_states = run_frame_sampling(video_path, [self], config)
108             state = own_states[self.name]
109         return self._finalize(state, meta, config, context)
110
111
112 class ValidationRegistry:
113     def __init__(self):
114         self._validators: List[VideoValidator] = []
115
116     def register(self, validator: VideoValidator):
117         """Registers a new validator."""
118         self._validators.append(validator)
119
120     def run_all_with_context(self, video_path: str, config: Any):
121         """Runs all validators and returns ``(results, context)``.
122
123         The per-run ``context`` accumulates shared values (notably
124         ``context['ffprobe_meta']`` produced by FormatValidator). ``run_all``
125         discards it; callers that want media metadata for an observability report
126         use this variant instead of re-probing.
127         """
128         results = []
129         context: dict[
130             str, Any
131         ] = {} # Shared context (e.g. for caching ffprobe outputs)
132         self._prepare_frame_samples(video_path, config, context)
133         for val in self._validators:
134             try:
135                 res = val.validate(video_path, config, context)
136                 results.append(res)
137             except Exception as e:
138                 results.append(
139                     ValidationResult(
140                         validator_name=val.name,
141                         passed=False,
142                         message=f"Validator threw an unexpected error: {str(e)}",
143                     )
144                 )
145         return results, context

```

```

146
147     def _prepare_frame_samples(
148         self, video_path: str, config: Any, context: dict[str, Any]
149     ) -> None:
150         """Decode every frame-sampling validator's frames in ONE shared pass.
151
152         Populates ``context`` with the shared decode meta + per-validator
153         accumulator state; each :class:`FrameSamplingValidator` then finalizes
154         from that instead of opening its own capture. Best-effort: any failure
155         here is swallowed so validators transparently fall back to a standalone
156         pass (the run must never crash on a decode hiccup)."""
157         samplers = [
158             v for v in self._validators if isinstance(v, FrameSamplingValidator)
159         ]
160         if not samplers:
161             return
162         try:
163             meta, states = run_frame_sampling(video_path, samplers, config)
164         except Exception:
165             logger.exception(
166                 "Shared frame-sampling pass failed for %s; validators will fall "
167                 "back to standalone decode.",
168                 video_path,
169             )
170         return
171         context[FRAME_META_KEY] = meta
172         context[FRAME_STATES_KEY] = states
173
174     def run_all(self, video_path: str, config: Any) -> List[ValidationResult]:
175         """Runs all registered validators in sequence, accumulating results."""
176         results, _ = self.run_all_with_context(video_path, config)
177         return results
178
179
180     # Shared global registry
181     registry = ValidationRegistry()
182

```

16. user (2026-07-07T16:58:04.711Z)

```

=== validators dir ===
__init__.py
__pycache__
base.py
blur.py
format.py
frame_sampler.py
pts_continuity.py
relevance.py
resolution_fps.py
scene_cut.py
=== config files ===
__init__.py
__pycache__
loader.py
models.py
presets.py
startup.py
__init__.py
__pycache__
loader.py
models.py
presets.py
startup.py
=== db layer ===

```



```
backend/billing.py
backend/flywheel.py
backend/main.py
backend/orchestration.py
backend/results.py
backend/server_runtime.py
```

17. assistant (2026-07-07T16:58:08.621Z)

Let me find the DB layer and read the config models plus the validators to understand which config keys drive validation.

18. user (2026-07-07T16:58:11.813Z)

```
=== find db module (add_video/get_video) ===
backend/infrastructure/database_media.py
=== validation_results storage ===
backend/infrastructure/database.py
backend/infrastructure/database_media.py
backend/infrastructure/__pycache__/database.cpython-313.pyc
backend/infrastructure/__pycache__/database_media.cpython-313.pyc
backend/main.py
backend/orchestration.py
backend/__pycache__/database.cpython-311.pyc
backend/__pycache__/database.cpython-313.pyc
backend/__pycache__/main.cpython-313.pyc
backend/__pycache__/orchestration.cpython-313.pyc
=== db dir ===
__init__.py
__pycache__
base.py
capabilities.py
gcs.py
gdrive.py
gdrive_api.py
local.py
ref.py
s3.py
```

19. user (2026-07-07T16:58:12.337Z)

```
1      """Pydantic models for the application configuration.
2
3      This module defines the canonical, validated, typed configuration for the
4      Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6      Unknown / typo'd config keys are rejected at load time via extra="forbid".
7      """
8
9      from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
```

```

24 class ValidationConfig(BaseModel):
25     min_resolution_width: int = 1280
26     min_resolution_height: int = 720
27     min_fps: float = 29.9
28     min_blur_threshold: float = 20.0
29     max_scene_cuts_per_minute: float = 0.5
30     pts_max_gap_seconds: float = Field(
31         default=10.0,
32         ge=0.0,
33         description=(
34             "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35             "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36             "a gap >10s suggests edited/missing content. Mobile phones, screen "
37             "recordings, and other encoders may use different GOP sizes ? tune "
38             "this threshold for your recording device."
39         ),
40     )
41     relevance: ValidationRelevanceConfig = Field(
42         default_factory=ValidationRelevanceConfig
43     )
44
45
46 class TrackNetConfig(BaseModel):
47     wsl_distro: str = "Ubuntu"
48     repo_dir: str = "~/models/TrackNetV4"
49     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50     conda_env: str = "TrackNetV4"
51     weights_path: str = ""
52     queue_length: int = 5
53     stage_in_wsl: bool = True
54     wsl_stage_dir: str = "~/clips"
55     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56     velocity_threshold: float = 0.02
57     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58     # decoded frame cache) are deleted automatically ? they are pure intermediates,
59     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61     keep_frames: bool = False
62     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
63     # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
64     # 0 = no extra floor (the headroom guard still applies).
65     min_free_gb: float = 0.0
66
67
68 class WasbIndexConfig(BaseModel):
69     """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72     these knobs actually take effect ? previously this whole block was silently dropped
73     because nested config sections defaulted to extra='ignore'.
74     """
75
76     wsl_distro: str = "Ubuntu"
77     repo_dir: str = "~/models/WASB-SBDT"
78     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79     conda_env: str = "wasb"
80     weights_path: str = (
81         "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82     )
83     sport: str = "badminton"
84     wsl_stage_dir: str = "~/clips"
85     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86     velocity_threshold: float = 0.02

```

```

87         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
88         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91         device: str = "cuda" # auto | cuda | mps | cpu
92         python_bin: str = (
93             "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
94         )
95         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98         keep_frames: bool = False
99         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100        # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101        # 0 = no extra floor (the headroom guard still applies).
102        min_free_gb: float = 0.0
103        # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104        # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105        # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106        # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107        # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108        # Tri-state:
109        #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110        #                     behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111        #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112        stream_video: Optional[bool] = None
113        # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114        # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115        # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
116        # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117        # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118        batch_size: Optional[int] = None
119        # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120        # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121        prefetch_batches: Optional[int] = None
122        # GPU-RESIDENT DECODE (#506) ? DEFAULT OFF. "nvdec_resident" moves
decode+resize+normalize
123        # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
`grid_sample`
124        # resize replicating WASB's cv2.warpAffine EXACTLY (the #506 lesson: a naive
F.interpolate
125        # silently over-detects ? F1 0.58?0.07 ? and only shows on hard clips) ? GPU
ImageNet
126        # normalize ? HRNet. Validated on a real L4 (GX010128, SERVED preset, 2026-07-06):
~1.7x
127        # fps (35.8?60.2), GPU util 47?79% / CPU 62?35%, F1 0.574 ? baseline 0.582. Native
(Linux)
128        # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env

```

```

(deploy/cloudrun/Dockerfile)
129     # with libnvcuvid/libnvidia-encode present, and device=cuda; the WSL runner ignores
it.
130     # Flip gate: the full 15-clip golden set at the SERVED preset (GX010128 must pass).
131     decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
132
133     @model_validator(mode="after")
134     def _nvdec_resident_requires_cuda(self) -> "WasbIndexConfig":
135         # NVDEC decode is CUDA-only; reject the contradictory combo at config load
instead of
136         # failing deep inside a paid GPU run. "auto" is allowed here ? the native
runner's
137         # healthcheck hard-fails if auto resolves to cpu with this backend on.
138         if self.decode_backend == "nvdec_resident" and self.device not in ("cuda",
"auto"):
139             raise ValueError(
140                 f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda'
or "
141                 f"'auto' (got {self.device!r}) ? NVDEC decode runs on the GPU."
142             )
143         return self
144
145     @field_validator("batch_size")
146     @classmethod
147     def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
148         # A DataLoader batch must be >=1; reject at config time so a typo can't reach
149         # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag
150         # when truthy, so 0 stays "off/parity-default").
151         if v is not None and v < 1:
152             raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
153         return v
154
155     @field_validator("prefetch_batches")
156     @classmethod
157     def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
158         # 0 = off (the default). Reject negatives: the runner passes any truthy value as
159         # `--prefetch-batches`, and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
160         # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
161         if v is not None and v < 0:
162             raise ValueError(
163                 f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
164             )
165         return v
166
167
168     class FusionConfig(BaseModel):
169         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
170
171         Every default reproduces control behaviour: the fusion segmenter persists the
172         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
173         true ? the person-cue features, but it does NOT gate the served handoff unless a
174         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
175         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
176         supplied ? off-court persons (spectators / adjacent courts) are excluded.
177         """
178
179         # Which trajectory substrate seeds the windows (shuttle stays primary).
180         substrate: Literal["wasb", "tracknet"] = "wasb"

```

```

181     # Windowing velocity threshold for the fusion family (the engine reads
182     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
183     # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
184     velocity_threshold: float = 0.02
185     # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
186     # enabled ? so the default path never imports torch / touches the GPU.
187     cue_flags: dict[str, bool] = Field(default_factory=dict)
188     # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
189     # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
190     min_crossings: int = Field(default=0, ge=0)
191     # Served Gate B: when the person cue is on, drop windows with median in-court
players
192     # < this. 0 = OFF.
193     min_players: int = Field(default=0, ge=0)
194     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
195     court_polygon: Optional[list[list[float]]] = None
196     # Net line for the person two-sided-motion split (person features only ? the shuttle
197     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
198     net_axis: Literal["x", "y"] = "y"
199     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
200
201
202     class IndexingConfig(BaseModel):
203         default_segmenter: str = "yolo_hybrid"
204         use_gpu: bool = True
205         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
206         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
207         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
208         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
209         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
210         detector_impl: str = "auto"
211         motion_threshold: float = 0.08
212         min_segment_duration: float = 3.0
213         dead_time_merge_gap: float = 2.0
214         chunk_duration: float = 90.0
215         chunk_overlap: float = 10.0
216         detector_model: str = "fasterrcnn_resnet50_fpn"
217         detector_score_threshold: float = 0.5
218         yolo_velocity_threshold: float = 0.15
219         yolo_frame_skip: int = 3
220         yolo_tracker: str = "bytetrack.yaml"
221         yolo_prefetch_workers: int = 2
222         min_action_window_duration: float = 2.0
223         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
224         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
225         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
226         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
227         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
228         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
229         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
230         max_window_duration: float = 6.0
231         window_min_split_gap: float = (
232             0.5 # min internal gap (s) that qualifies as a split point
233         )
234         # HARD-CAP fallback for W1: when True, an over-long window with NO internal

```

```

inactivity gap
235      # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
236      # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
237      # Only cuts (recall can't drop). OFF by default until measured/promoted.
238      window_hard_cap: bool = False
239      # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
240      # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
241      # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
242      # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
243      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
244      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
245      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
246      window_inactivity_end: float = 1.5
247      inactivity_rally_thresh: float = 0.20
248      inactivity_min_density: float = 0.30
249      # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
250      # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
251      # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
252      # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
253      # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
254      # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
255      # regression), |?end| 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),
256      # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
257      # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
258      inactivity_temporal_density: bool = True
259      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
260      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
261      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
262      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
263      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
264      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
265      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
266      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
267      window_blob_fallback: bool = False
268      # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
269      # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
270      # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
271      # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md

```

```

"R5".
272     window_blob_factor: float = 4.0
273     log_yolo_candidates: bool = True
274     store_yolo_candidates: bool = True
275     ai_handoff_padding: float = 2.0
276     ai_max_workers: int = 5
277     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
278     # chunks of a high-bitrate (4K) source blow past the provider upload cap
279     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
280     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
281     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
282     # Matches gemini_refine's --clip-scale-width default.
283     ai_chunk_scale_width: int = 1280
284     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
285     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
286     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
287     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
288     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
289     skip_ai_handoff: bool = False
290     # Optional debug knob: trim every video to the first N seconds before processing.
291     debug_duration: Optional[float] = None
292
293     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
294     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
295     fusion: FusionConfig = Field(default_factory=FusionConfig)
296
297     @field_validator("default_segmenter")
298     @classmethod
299     def segmenter_must_be_registered(cls, v: str) -> str:
300         # Lazy import: the segmenter registry imports modules that may import
301         # config, so we resolve it at validation time rather than at module load.
302         from backend.pipeline.segmenters.base import segmenter_registry
303
304         if segmenter_registry.get(v) is None:
305             available = sorted(segmenter_registry.list_available().keys())
306             raise ValueError(
307                 f"Segmenter '{v}' is not registered. Available: {' ', ' '.join(available)}"
308             )
309         return v
310
311     @model_validator(mode="after")
312     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
313         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
314         # step makes `while t < duration: t += step` loop forever, growing memory before
any
315         # GPU work. Reject the bad config at load time instead.
316         if self.chunk_overlap >= self.chunk_duration:
317             raise ValueError(
318                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
319                 f"({self.chunk_duration}); otherwise chunking never advances."
320             )
321         return self
322
323
324     class OutputConfig(BaseModel):
325         default_highlights_name: str = "highlights.mp4"
326         db_name: str = "sports_indexer.db"
327         # Directory where the DB, highlight reels, and processed clips are written.
328         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).

```

```

329     output_dir: str = "output"
330
331
332     class WatermarkConfig(BaseModel):
333         """Branding overlay burned into each highlight clip during the per-clip re-encode
334         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
335         (under `stitching.watermark` in config.json) to turn it off. The text is fully
336         configurable. The concat stage stays stream-copy (-c copy)."""
337
338         enabled: bool = True
339         text: str = "Generated by Khelsutra.guru"
340         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
341         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family below
342         font: str = "Sans" # fontconfig family used when font_path is empty
343         font_size_ratio: float = Field(
344             default=0.035, gt=0.0, le=0.5
345         ) # text height as a fraction of frame height
346         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
347         box: bool = True # faint backing box behind the text for legibility
348         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
349         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
350             "bottom-right"
351         )
352
353
354     class StitchingConfig(BaseModel):
355         begin_padding: float = 0.5
356         end_padding: float = 0.5
357         use_cache: bool = False
358         min_shot_count: int = 1
359         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
360         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
361         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
362         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
363         # local detector over-fires and its false positives are mostly SHORT (motion blips,
364         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
365         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
366         min_rally_duration: float = Field(default=0.0, ge=0.0)
367         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
368
369
370     class LoggingConfig(BaseModel):
371         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
372
373         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
374         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
375         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
376         # them to WARNING; set true to restore full chatter for request-flow debugging.
377         verbose_http: bool = False
378
379
380     class SecurityConfig(BaseModel):
381         # When set, /api/process video_path must resolve under this directory (hosted/tenant
382         # mode). Default None preserves the single-user local 'any local file' workflow.
383         allowed_video_root: Optional[str] = None
384
385         # --- Chunked upload (B2, PR-2) ---
386         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
387         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
388         # contain upload_dir or /api/process will reject the uploaded paths.

```



```

389     upload_dir: str = "output/uploads"
390     # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
391     # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
392     max_upload_mb: int = 4096
393     max_part_mb: int = 95
394     allowed_upload_extensions: List[str] = ["mp4", "mov"]
395
396     # --- M9 edge auth (Cloudflare Access, D9) ---
397     # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
398     # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
399     # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
400     # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
401     # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
402     # application's audience tag. NO secrets here ? these are public identifiers.
403     edge_auth_enabled: bool = False
404     cf_team_domain: str = (
405         "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
406     )
407     cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
408     # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
409     # backend/main.py so importing the app does no filesystem I/O; default [""].
Credentials stay
410     # OFF ? Cf-Access is a header, not a cookie.)
411
412
413     class StorageConfig(BaseModel):
414         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
415         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
416         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
417         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
418         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
419         (boto3/google auth chains supply credentials). See ``backend/storage/``. """
420
421         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
422         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
423         output_root: str = ""
424         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
425         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
426         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
427         # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"
428         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
429         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
430         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
431         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
432         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
433         input_root: str = ""
434         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---

```

```

435         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
436         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
437         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
438         # the legacy flat layout is used unchanged.
439         single_folder_per_video: bool = False
440         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
441         workspace_root: str = ""
442         # Cloud namespace prefix under the root so per-video folders sit tidily beside
443         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
444         workspace_prefix: str = "workspaces"
445         local: dict[str, Any] = Field(default_factory=dict)
446         s3: dict[str, Any] = Field(default_factory=dict)
447         gcs: dict[str, Any] = Field(default_factory=dict)
448         gdrive: dict[str, Any] = Field(default_factory=dict)
449         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
450         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
451         gdrive_api: dict[str, Any] = Field(default_factory=dict)
452
453
454     class DeploymentConfig(BaseModel):
455         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
456         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
457
458         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
459         behaves exactly as before this section existed. A profile is opt-in (``config.json``
460         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
461         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
462         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
463         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
464
465         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
466
467         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
468         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
469         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
470         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
471         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
472         # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
473         # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
474         # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
475         # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
476         # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
477         max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
478         # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD $4.3).
Must sit
479         # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays
the

```

```

480         # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
481         # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
482         # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
483         remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
484
485
486     class BillingConfig(BaseModel):
487         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
488
489         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
490         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
491         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
492         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
493         inserts a real, calibrated version and points ``pricebook_version`` at it.
494
495         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
496         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
497         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
498
499         enabled: bool = False
500         pricebook_version: str = (
501             "placeholder-v0" # the seeded $0 version; owner flips to a real one
502         )
503         fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore market)
504         margin: float = 0.0 # resale markup (?0); 0 = bill at cost
505
506
507     class FlywheelConfig(BaseModel):
508         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
509         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
510         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
511         the owned model climbs toward the D11 ship floor.
512
513         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
514         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
515         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
516         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-row).
517         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
518         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
519         owner/counsel/GPU scope. Pairs with M9-consent."""
520
521         enabled: bool = False
522         workspace_prefix: str = (
523             "flywheel" # per-video workspace prefix for captured artifacts
524         )
525         shadow_eval: bool = True # run the owned-vs-Gemini shadow comparison on capture

```

```

526         min_promote_n: int = 5 # promote_if_served_safe `min_n` (the served-safe gate)
527
528
529     class AppConfig(BaseModel):
530         """Root configuration object.
531
532         All runtime configuration should be accessed via an instance of this class
533         (or the loader) rather than raw dict lookups.
534         """
535
536         sport: Sport = "badminton"
537         ai_provider: str = "gemini"
538         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
539         # the alias never deprecates underneath us. Eval reports record the run date
540         # alongside.
541         ai_model: str = "gemini-flash-latest"
542
543         validation: ValidationConfig = Field(default_factory=ValidationConfig)
544         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
545         output: OutputConfig = Field(default_factory=OutputConfig)
546         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
547         security: SecurityConfig = Field(default_factory=SecurityConfig)
548         logging: LoggingConfig = Field(default_factory=LoggingConfig)
549         storage: StorageConfig = Field(default_factory=StorageConfig)
550         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
551         billing: BillingConfig = Field(default_factory=BillingConfig)
552         flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)
553
554         # Reject unknown keys ? typed config migration is complete (D3).
555         model_config = {
556             "extra": "forbid",
557             "validate_assignment": True,
558         }
559
560     # Convenience type alias for callers that want to annotate
561     Config = AppConfig
562

```

20. assistant (2026-07-07T16:58:26.179Z)

Now I have the `ValidationConfig` model ? that's the fingerprint domain. Let me confirm exactly which config each validator reads (to make sure the fingerprint covers everything, including anything outside `validation.\*`), and read the DB layer for `add\_video`/`get\_video`.

21. user (2026-07-07T16:58:27.615Z)

```

1     from typing import Any, List
2
3     import cv2
4
5     from backend.pipeline.validators.base import FrameSamplingValidator, ValidationResult
6     from backend.pipeline.validators.frame_sampler import SampleMeta
7
8     # Number of frames sampled evenly across the whole video.
9     _NUM_SAMPLES = 15
10
11
12     class BlurValidator(FrameSamplingValidator):
13         @property
14         def name(self) -> str:
15             return "blur_check"
16
17         @property

```

```

18         def description(self) -> str:
19             return "Samples frames across the video and checks if the footage is excessively
blurry or out of focus."
20
21         def sample_indices(self, total_frames: int, fps: float) -> List[int]:
22             # Sample _NUM_SAMPLES frames evenly distributed across the video.
23             step = max(1, total_frames // _NUM_SAMPLES)
24             return [i * step for i in range(_NUM_SAMPLES)]
25
26         def _init_state(self, config: Any) -> dict[str, Any]:
27             return {"variances": []}
28
29         def _consume(self, state: dict[str, Any], frame_idx: int, frame: Any) -> None:
30             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
31             # Calculate the Laplacian variance (standard focus metric)
32             laplacian_var = cv2.Laplacian(gray, cv2.CV_64F).var()
33             state["variances"].append(laplacian_var)
34
35         def _finalize(
36             self, state: dict[str, Any], meta: SampleMeta, config: Any, context: dict
37         ) -> ValidationResult:
38             if not meta.opened:
39                 return ValidationResult(
40                     validator_name=self.name,
41                     passed=False,
42                     message="Failed to open video file for blur analysis.",
43                 )
44
45             if meta.total_frames <= 0:
46                 return ValidationResult(
47                     validator_name=self.name, passed=False, message="Invalid frame count."
48                 )
49
50             variances = state["variances"]
51             if not variances:
52                 return ValidationResult(
53                     validator_name=self.name,
54                     passed=False,
55                     message="Could not read frames to perform blur verification.",
56                 )
57
58             avg_variance = sum(variances) / len(variances)
59
60             details = {
61                 "avg_laplacian_variance": round(avg_variance, 2),
62                 "min_laplacian_variance": round(min(variances), 2),
63                 "max_laplacian_variance": round(max(variances), 2),
64                 "sampled_frames_count": len(variances),
65             }
66
67             val_cfg = config.validation
68             min_blur_threshold = val_cfg.min_blur_threshold
69
70             if avg_variance < min_blur_threshold:
71                 return ValidationResult(
72                     validator_name=self.name,
73                     passed=False,
74                     message=f"Video quality rejected: Focus check failed. Average sharpness
score of {round(avg_variance, 1)} is below the minimum required threshold of
{min_blur_threshold}.",
75                     details=details,
76                 )
77
78             return ValidationResult(
79                 validator_name=self.name,

```

```

80         passed=True,
81         message="Video sharpness meets focus requirements.",
82         details=details,
83     )
84

```

22. user (2026-07-07T16:58:28.198Z)

```

1     from typing import Any, List
2
3     import cv2
4     import numpy as np
5
6     from backend.pipeline.validators.base import FrameSamplingValidator, ValidationResult
7     from backend.pipeline.validators.frame_sampler import SampleMeta
8
9     # Number of frames sampled evenly across the whole video.
10    _NUM_SAMPLES = 10
11
12
13    class RelevanceValidator(FrameSamplingValidator):
14        @property
15        def name(self) -> str:
16            return "relevance_check"
17
18        @property
19        def description(self) -> str:
20            return "Checks if the video content is relevant to badminton by scanning for
21            court color palettes and geometric features."
22
23        def sample_indices(self, total_frames: int, fps: float) -> List[int]:
24            # Sample _NUM_SAMPLES frames across the video.
25            step = max(1, total_frames // _NUM_SAMPLES)
26            return [i * step for i in range(_NUM_SAMPLES)]
27
28        def _init_state(self, config: Any) -> dict[str, Any]:
29            # Resolve the court-color bounds ONCE per run (config wins over the sport
30            # profile, Pydantic defaults as fallback) ? same precedence the old
31            # per-frame loop applied on every iteration.
32            sport_name = config.sport
33            from backend.sports import sport_registry
34
35            try:
36                sport_profile = sport_registry.get(sport_name)
37                sport_rel_cfg = sport_profile.get_relevance_config()
38            except KeyError:
39                sport_rel_cfg = {}
40
41            rel_cfg = config.validation.relevance
42
43            hsv_lower = np.array(
44                rel_cfg.court_green_lower
45                or sport_rel_cfg.get("court_green_lower")
46                or [35, 40, 40]
47            )
48            hsv_upper = np.array(
49                rel_cfg.court_green_upper
50                or sport_rel_cfg.get("court_green_upper")
51                or [85, 255, 255]
52            )
53            min_court_ratio = (
54                rel_cfg.court_pixels_min_ratio
55                if rel_cfg.court_pixels_min_ratio is not None
56                else sport_rel_cfg.get("court_pixels_min_ratio", 0.05)
57            )

```

```

57
58     return {
59         "hsv_lower": hsv_lower,
60         "hsv_upper": hsv_upper,
61         "min_court_ratio": min_court_ratio,
62         "green_ratios": [],
63     }
64
65     def _consume(self, state: dict[str, Any], frame_idx: int, frame: Any) -> None:
66         # Downscale frame for speed, then measure the court-green pixel ratio.
67         small_frame = cv2.resize(frame, (320, 240))
68         hsv = cv2.cvtColor(small_frame, cv2.COLOR_BGR2HSV)
69
70         mask = cv2.inRange(hsv, state["hsv_lower"], state["hsv_upper"])
71         green_pixels = cv2.countNonZero(mask)
72         total_pixels = mask.shape[0] * mask.shape[1]
73
74         state["green_ratios"].append(green_pixels / total_pixels)
75
76     def _finalize(
77         self, state: dict[str, Any], meta: SampleMeta, config: Any, context: dict
78     ) -> ValidationResult:
79         if not meta.opened:
80             return ValidationResult(
81                 validator_name=self.name,
82                 passed=False,
83                 message="Failed to open video file for relevance validation.",
84             )
85
86         if meta.total_frames <= 0:
87             return ValidationResult(
88                 validator_name=self.name, passed=False, message="Invalid frame count."
89             )
90
91         green_ratios = state["green_ratios"]
92         if not green_ratios:
93             return ValidationResult(
94                 validator_name=self.name,
95                 passed=False,
96                 message="Failed to extract frames for relevance checking.",
97             )
98
99         sport_name = config.sport
100         min_court_ratio = state["min_court_ratio"]
101         avg_green_ratio = sum(green_ratios) / len(green_ratios)
102
103         details = {
104             "avg_green_court_ratio": round(avg_green_ratio, 4),
105             "max_green_court_ratio": round(max(green_ratios), 4),
106             "min_green_court_ratio": round(min(green_ratios), 4),
107             "green_threshold": min_court_ratio,
108         }
109
110         # If a certain amount of court color is visible, consider it relevant
111         if avg_green_ratio < min_court_ratio:
112             return ValidationResult(
113                 validator_name=self.name,
114                 passed=False,
115                 message=f"Video rejected: Relevance check failed. The scene does not contain a recognizable {sport_name} court (court area found: {round(avg_green_ratio * 100, 2)}% vs min required {round(min_court_ratio * 100, 2)}%).",
116                 details=details,
117             )
118
119         return ValidationResult(

```

```

120         validator_name=self.name,
121         passed=True,
122         message=f"Video identified as relevant to {sport_name} ({sport_name} court
structures found).",
123         details=details,
124     )
125

```

23. user (2026-07-07T16:58:29.114Z)

```

1     from typing import Any, List
2
3     import cv2
4
5     from backend.pipeline.validators.base import FrameSamplingValidator, ValidationResult
6     from backend.pipeline.validators.frame_sampler import SampleMeta
7
8     # Correlation histogram threshold: 1.0 is a perfect match; < 0.6 flags a sudden
9     # massive scene change (a cut).
10    _CUT_THRESHOLD = 0.6
11    # Cap the number of samples so validation stays instant on long clips (the
12    # samples cover roughly the first 50 seconds at 2 fps).
13    _MAX_SAMPLES = 100
14
15
16    class SceneCutDensityValidator(FrameSamplingValidator):
17        @property
18        def name(self) -> str:
19            return "scene_cut_check"
20
21        @property
22        def description(self) -> str:
23            return "Analyzes visual frames to detect abrupt scene cuts, which indicate an
edited or compiled video."
24
25        def sample_indices(self, total_frames: int, fps: float) -> List[int]:
26            # Sample at 2 frames per second (fps/2 step), capped at _MAX_SAMPLES.
27            sample_interval = max(1, int(fps / 2))
28            indices: List[int] = []
29            frame_idx = 0
30            sampled_count = 0
31            while True:
32                indices.append(frame_idx)
33                sampled_count += 1
34                frame_idx += sample_interval
35                if frame_idx >= total_frames or sampled_count > _MAX_SAMPLES:
36                    break
37            return indices
38
39        def _init_state(self, config: Any) -> dict[str, Any]:
40            return {"cuts_detected": 0, "sampled_count": 0, "prev_hist": None}
41
42        def _consume(self, state: dict[str, Any], frame_idx: int, frame: Any) -> None:
43            # Calculate color histogram (hue/saturation) and compare to the previous
44            # sample; a big correlation drop is a cut.
45            hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
46            hist = cv2.calcHist([hsv], [0, 1], None, [50, 60], [0, 180, 0, 256])
47            cv2.normalize(hist, hist, alpha=0, beta=1, norm_type=cv2.NORM_MINMAX)
48
49            prev_hist = state["prev_hist"]
50            if prev_hist is not None:
51                score = cv2.compareHist(prev_hist, hist, cv2.HISTCMP_CORREL)
52                if score < _CUT_THRESHOLD:
53                    state["cuts_detected"] += 1
54

```



```

55         state["prev_hist"] = hist
56         state["sampled_count"] += 1
57
58     def _finalize(
59         self, state: dict[str, Any], meta: SampleMeta, config: Any, context: dict
60     ) -> ValidationResult:
61         if not meta.opened:
62             return ValidationResult(
63                 validator_name=self.name,
64                 passed=False,
65                 message="Failed to open video file using OpenCV.",
66             )
67
68         fps = meta.fps
69         total_frames = meta.total_frames
70         duration = total_frames / fps if fps > 0 else 0
71
72         if duration <= 0:
73             return ValidationResult(
74                 validator_name=self.name,
75                 passed=False,
76                 message="Invalid video duration.",
77             )
78
79         sample_interval = max(1, int(fps / 2))
80         cuts_detected = state["cuts_detected"]
81         sampled_count = state["sampled_count"]
82
83         # Compute cuts per minute
84         sampled_duration = (sampled_count * sample_interval) / fps if fps > 0 else 1
85         cuts_per_minute = (
86             (cuts_detected / sampled_duration) * 60 if sampled_duration > 0 else 0
87         )
88
89         details = {
90             "cuts_detected": cuts_detected,
91             "sampled_frames": sampled_count,
92             "cuts_per_minute": round(cuts_per_minute, 3),
93         }
94
95         val_cfg = config.validation
96         max_cuts_rate = val_cfg.max_scene_cuts_per_minute
97
98         if cuts_per_minute > max_cuts_rate:
99             return ValidationResult(
100                 validator_name=self.name,
101                 passed=False,
102                 message=f"Video splicing compromise: Detected {cuts_detected} cuts ({round(cuts_per_minute, 2)} cuts/min), which exceeds the continuous raw file limit of {max_cuts_rate} cuts/min.",
103                 details=details,
104             )
105
106         return ValidationResult(
107             validator_name=self.name,
108             passed=True,
109             message="Scene continuity check passed. No abnormal visual cuts detected.",
110             details=details,
111         )
112

```

24. user (2026-07-07T16:58:29.628Z)

```

1     import json
2     import logging

```

```

3     import subprocess
4     from typing import Any
5
6     from backend.pipeline.validators.base import ValidationResult, VideoValidator
7     from backend.utils.ffmpeg import ffprobe_bin, ffprobe_timeout_sec, timeout_label
8
9     logger = logging.getLogger(__name__)
10
11
12     def _probe_meta(video_path: str) -> dict | None:
13         """Re-probe ffprobe metadata as a fallback when format_check hasn't run first."""
14         try:
15             timeout = ffprobe_timeout_sec()
16             result = subprocess.run(
17                 [
18                     ffprobe_bin(),
19                     "-v",
20                     "error",
21                     "-select_streams",
22                     "v:0",
23                     "-show_format",
24                     "-show_streams",
25                     "-of",
26                     "json",
27                     video_path,
28                 ],
29                 capture_output=True,
30                 text=True,
31                 check=True,
32                 timeout=timeout,
33             )
34             return json.loads(result.stdout)
35         except subprocess.TimeoutExpired:
36             logger.warning(
37                 "resolution_fps_check: fallback ffprobe timed out after %s",
38                 timeout_label(ffprobe_timeout_sec()),
39             )
40             return None
41         except (subprocess.CalledProcessError, json.JSONDecodeError, OSError):
42             return None
43
44
45     class ResolutionFPSValidator(VideoValidator):
46         @property
47         def name(self) -> str:
48             return "resolution_fps_check"
49
50         @property
51         def description(self) -> str:
52             return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate (e.g. 30fps) requirements."
53
54         def validate(
55             self, video_path: str, config: Any, context: dict[str, Any]
56         ) -> ValidationResult:
57             meta = context.get("ffprobe_meta")
58             if not meta or not meta.get("streams"):
59                 # Fallback: re-probe instead of hard-failing (format_check may not
60                 # have run, e.g. if validator ordering changed).
61                 logger.warning(
62                     "resolution_fps_check: ffprobe_meta not in shared context ? "
63                     "re-probing. Ensure format_check runs before resolution_fps_check "
64                     "in the validator registry."
65                 )
66             meta = _probe_meta(video_path)

```

```

67         if not meta or not meta.get("streams"):
68             return ValidationResult(
69                 validator_name=self.name,
70                 passed=False,
71                 message="Video metadata is missing. Run format_check validator first.",
72             )
73
74         stream = meta["streams"][0]
75         width = int(stream.get("width", 0))
76         height = int(stream.get("height", 0))
77
78         # Parse frame rate (e.g. "30/1" or "30000/1001")
79         fps_str = stream.get("r_frame_rate", "0/1")
80         try:
81             num, den = map(int, fps_str.split("/"))
82             fps = num / den if den != 0 else 0
83         except ValueError:
84             fps = 0.0
85
86         details = {"width": width, "height": height, "fps": round(fps, 2)}
87
88         # Check bounds from typed config (AppConfig.validation)
89         val_cfg = config.validation
90         min_w = val_cfg.min_resolution_width
91         min_h = val_cfg.min_resolution_height
92         min_fps = val_cfg.min_fps
93
94         if width < min_w or height < min_h:
95             return ValidationResult(
96                 validator_name=self.name,
97                 passed=False,
98                 message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
99                 details=details,
100             )
101
102         if fps < min_fps:
103             return ValidationResult(
104                 validator_name=self.name,
105                 passed=False,
106                 message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
107                 details=details,
108             )
109
110         return ValidationResult(
111             validator_name=self.name,
112             passed=True,
113             message="Video resolution and frame rate meet minimum standards.",
114             details=details,
115         )
116

```

25. user (2026-07-07T16:58:30.632Z)

```

1     import subprocess
2     from typing import Any
3
4     from backend.pipeline.validators.base import ValidationResult, VideoValidator
5     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
6         ffprobe_bin,
7         ffprobe_timeout_sec,
8         timeout_label,
9     )
10

```

```

11
12 class PTSContinuityValidator(VideoValidator):
13     @property
14     def name(self) -> str:
15         return "pts_continuity_check"
16
17     @property
18     def description(self) -> str:
19         return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
20 timeline gaps, jumps, or editing splices."
21
22     def validate(
23         self, video_path: str, config: Any, context: dict[str, Any]
24     ) -> ValidationResult:
25         # Command to extract packet PTS and flags (extremely fast, works on H.264 &
26         # H.265)
27         cmd = [
28             ffprobe_bin(),
29             "-v",
30             "error",
31             "-select_streams",
32             "v:0",
33             "-show_entries",
34             "packet=pts_time,flags",
35             "-of",
36             "csv=p=0",
37             video_path,
38         ]
39
40         try:
41             timeout = ffprobe_timeout_sec()
42             result = subprocess.run(
43                 cmd, capture_output=True, text=True, check=True, timeout=timeout
44             )
45             output = result.stdout.strip()
46
47             if not output:
48                 return ValidationResult(
49                     validator_name=self.name,
50                     passed=False,
51                     message="No packet timestamps could be extracted from the video
52 stream.",
53                 )
54
55             timestamps = []
56             for line in output.split("\n"):
57                 line = line.strip()
58                 if line:
59                     parts = line.split(",")
60                     if len(parts) == 2:
61                         pts_str, flags = parts
62                         if "K" in flags: # Check if keyframe
63                             try:
64                                 timestamps.append(float(pts_str))
65                             except ValueError:
66                                 continue
67
68             if len(timestamps) < 2:
69                 return ValidationResult(
70                     validator_name=self.name,
71                     passed=True,
72                     message="Video is too short to perform a PTS continuity check.",
73                     details={"keyframes_count": len(timestamps)},
74                 )

```

```

73         # Analyze gaps
74         max_gap = 0.0
75         backward_jumps = 0
76         gaps = []
77
78         for i in range(1, len(timestamps)):
79             gap = timestamps[i] - timestamps[i - 1]
80             gaps.append(gap)
81             if gap < 0:
82                 backward_jumps += 1
83             if gap > max_gap:
84                 max_gap = gap
85
86         # A continuous GoPro recording usually has keyframe intervals (GOP size)
87         # of 1.0 to 4.0 seconds. A gap larger than the configured threshold
88         # (or a backward jump) indicates edited/spliced content.
89         details = {
90             "keyframes_count": len(timestamps),
91             "max_keyframe_gap_seconds": round(max_gap, 2),
92             "backward_jumps_detected": backward_jumps,
93             "first_keyframe": timestamps[0],
94             "last_keyframe": timestamps[-1],
95         }
96
97         if backward_jumps > 0:
98             return ValidationResult(
99                 validator_name=self.name,
100                 passed=False,
101                 message="Video integrity compromise: Backward timeline jumps
(splicing) detected.",
102                 details=details,
103             )
104
105         max_gap_threshold = config.validation.pts_max_gap_seconds
106         if max_gap > max_gap_threshold:
107             return ValidationResult(
108                 validator_name=self.name,
109                 passed=False,
110                 message=f"Video integrity compromise: Detected a timestamp gap of
{round(max_gap, 2)}s, indicating edited or missing content.",
111                 details=details,
112             )
113
114         return ValidationResult(
115             validator_name=self.name,
116             passed=True,
117             message="PTS timeline is continuous with no abnormal gaps or backward
jumps.",
118             details=details,
119         )
120
121     except subprocess.CalledProcessError as e:
122         return ValidationResult(
123             validator_name=self.name,
124             passed=False,
125             message="Failed to execute ffprobe PTS analysis.",
126             details={"error": e.stderr},
127         )
128     except subprocess.TimeoutExpired:
129         return ValidationResult(
130             validator_name=self.name,
131             passed=False,
132             message=f"ffprobe PTS analysis timed out after
{timeout_label(ffprobe_timeout_sec())}.",
133         )

```

```

134         except Exception as e:
135             return ValidationResult(
136                 validator_name=self.name,
137                 passed=False,
138                 message=f"Unexpected error in PTS validation: {str(e)}",
139             )
140

```

26. user (2026-07-07T16:58:30.921Z)

```

1     import json
2     import os
3     import subprocess
4     from typing import Any
5
6     from backend.pipeline.validators.base import ValidationResult, VideoValidator
7     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
8         ffprobe_bin,
9         ffprobe_timeout_sec,
10        timeout_label,
11    )
12
13
14    class FormatValidator(VideoValidator):
15        @property
16        def name(self) -> str:
17            return "format_check"
18
19        @property
20        def description(self) -> str:
21            return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
22
23        def validate(
24            self, video_path: str, config: Any, context: dict[str, Any]
25        ) -> ValidationResult:
26            if not os.path.exists(video_path):
27                return ValidationResult(
28                    validator_name=self.name,
29                    passed=False,
30                    message=f"File not found: {video_path}",
31                )
32
33            cmd = [
34                ffprobe_bin(),
35                "-v",
36                "error",
37                "-select_streams",
38                "v:0",
39                "-show_format",
40                "-show_streams",
41                "-of",
42                "json",
43                video_path,
44            ]
45
46            try:
47                timeout = ffprobe_timeout_sec()
48                result = subprocess.run(
49                    cmd, capture_output=True, text=True, check=True, timeout=timeout
50                )
51                info = json.loads(result.stdout)
52
53                if not info.get("streams") or not info.get("format"):
54                    return ValidationResult(

```

```

55         validator_name=self.name,
56         passed=False,
57         message="Failed to retrieve valid video streams from the file.",
58     )
59
60     # Store the metadata in context for subsequent validators to utilize
61     context["ffprobe_meta"] = info
62
63     stream = info["streams"][0]
64     fmt = info["format"]
65
66     details = {
67         "format_name": fmt.get("format_name"),
68         "duration": float(fmt.get("duration", 0)),
69         "size_bytes": int(fmt.get("size", 0)),
70         "codec_name": stream.get("codec_name"),
71     }
72
73     # Check if container name includes mp4
74     if "mp4" not in fmt.get("format_name", "").lower():
75         return ValidationResult(
76             validator_name=self.name,
77             passed=False,
78             message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
79             details=details,
80         )
81
82     return ValidationResult(
83         validator_name=self.name,
84         passed=True,
85         message="Video format is valid MP4 container.",
86         details=details,
87     )
88
89     except subprocess.CalledProcessError as e:
90         return ValidationResult(
91             validator_name=self.name,
92             passed=False,
93             message="Video stream is corrupted or unreadable by ffprobe.",
94             details={"error": e.stderr},
95         )
96     except subprocess.TimeoutExpired:
97         return ValidationResult(
98             validator_name=self.name,
99             passed=False,
100             message=f"ffprobe timed out after
{timeout_label(ffprobe_timeout_sec())}.",
101         )
102     except Exception as e:
103         return ValidationResult(
104             validator_name=self.name,
105             passed=False,
106             message=f"Unexpected error running ffprobe: {str(e)}",
107         )
108

```

27. assistant (2026-07-07T16:59:09.678Z)

Now I understand the full config surface each validator reads. Key finding: `relevance.py` reads `config.sport` (not just `validation.\*`) ? so the fingerprint must include `sport` too, not only the `validation` block. Let me examine the DB layer to decide storage (video record vs. dedicated table) and understand the migration machinery.

28. user (2026-07-07T16:59:09.835Z)

```

1     import json
2     import logging
3     from typing import Any, Dict, List, Optional, Tuple
4
5     logger = logging.getLogger(__name__)
6
7
8     class DatabaseMediaMixin:
9         def _connect(self): # pragma: no cover - implemented by Database facade
10             raise NotImplementedError
11
12         def _execute_write( # pragma: no cover - implemented by Database facade
13             self, sql: str, params: Tuple[Any, ...], error_msg: str
14         ) -> bool:
15             raise NotImplementedError
16
17         def add_video(
18             self,
19             video_id: str,
20             filepath: str,
21             status: str,
22             validation_results: List[Dict[str, Any]],
23         ) -> bool:
24             """Register or update a video row WITHOUT touching its child segments.
25
26             Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
27             with foreign_keys ON ? cascade-deletes every
28             play_segment/event/candidate_segment.
29             That made a status update (e.g. marking 'failed' on re-validation, or
30             'processed'
31             before the segmenter runs) silently destroy prior good results. UPSERT mutates
32             the
33             row in place; clearing segments is now an explicit, deliberate operation
34             (clear_video_data / prune_segments).
35             """
36             return self._execute_write(
37                 "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
38                 ?, ?) "
39                 "ON CONFLICT(id) DO UPDATE SET "
40                 "filepath=excluded.filepath, status=excluded.status, "
41                 "validation_results=excluded.validation_results",
42                 (video_id, filepath, status, json.dumps(validation_results)),
43                 "Failed to add video",
44             )
45
46         def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
47             """Return (max play_segment id, max candidate_segment id) for a video (0 if
48             none).
49
50             Captured before a (re)segmentation so the run's new rows (id > watermark) can be
51             told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
52             """
53             with self._connect() as conn:
54                 cur = conn.cursor()
55                 p = cur.execute(
56                     "SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE video_id = ?",
57                     (video_id, ),
58                 ).fetchone()[0]
59                 c = cur.execute(
60                     "SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE video_id =
61                     ?",
62                     (video_id, ),
63                 ).fetchone()[0]
64                 return int(p), int(c)

```



```

60     def prune_segments(
61         self,
62         video_id: str,
63         *,
64         play_upto: Optional[int] = None,
65         play_after: Optional[int] = None,
66         cand_upto: Optional[int] = None,
67         cand_after: Optional[int] = None,
68     ) -> None:
69         """Delete play/candidate segments by id range (events cascade from
play_segments).
70
71         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
72         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
73         and keep the OLD ones when the run failed/produced nothing (`*_after`).
74         """
75         with self._connect() as conn:
76             cur = conn.cursor()
77             if play_upto is not None:
78                 cur.execute(
79                     "DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
80                     (video_id, play_upto),
81                 )
82             if play_after is not None:
83                 cur.execute(
84                     "DELETE FROM play_segments WHERE video_id = ? AND id > ?",
85                     (video_id, play_after),
86                 )
87             if cand_upto is not None:
88                 cur.execute(
89                     "DELETE FROM candidate_segments WHERE video_id = ? AND id <= ?",
90                     (video_id, cand_upto),
91                 )
92             if cand_after is not None:
93                 cur.execute(
94                     "DELETE FROM candidate_segments WHERE video_id = ? AND id > ?",
95                     (video_id, cand_after),
96                 )
97             conn.commit()
98
99     def add_play_segment(
100         self,
101         video_id: str,
102         start_time: float,
103         end_time: float,
104         server: Optional[str] = None,
105         receiver: Optional[str] = None,
106         metadata: Optional[Dict[str, Any]] = None,
107     ) -> Optional[int]:
108         try:
109             with self._connect() as conn:
110                 cursor = conn.cursor()
111                 cursor.execute(
112                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
113                     (
114                         video_id,
115                         start_time,
116                         end_time,
117                         end_time - start_time,
118                         server,
119                         receiver,
120                         json.dumps(metadata or {}),
121                     ),

```

```

122         )
123         conn.commit()
124         return cursor.lastrowid
125     except Exception as e:
126         logger.error(f"Failed to add play segment: {e}")
127         return None
128
129     def add_event(
130         self,
131         segment_id: int,
132         timestamp: float,
133         event_type: str,
134         player: Optional[str] = None,
135         details: Optional[Dict[str, Any]] = None,
136     ) -> bool:
137         return self._execute_write(
138             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
139             (?, ?, ?, ?, ?)",
140             (segment_id, timestamp, event_type, player, json.dumps(details or {})),
141             "Failed to add event",
142         )
143
144     def add_candidate_segment(
145         self,
146         video_id: str,
147         detector: str,
148         start_time: float,
149         end_time: float,
150         chunk_index: Optional[int] = None,
151         confidence: Optional[float] = None,
152         stats: Optional[Dict[str, Any]] = None,
153         detection_params: Optional[Dict[str, Any]] = None,
154     ) -> Optional[int]:
155         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
156         detector-specific dicts stored as JSON. Returns the new candidate id, or
157         None."""
158         try:
159             with self._connect() as conn:
160                 cursor = conn.cursor()
161                 cursor.execute(
162                     """INSERT INTO candidate_segments
163                     (video_id, detector, chunk_index, start_time, end_time, duration,
164                     confidence, stats, detection_params)
165                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
166                     (
167                         video_id,
168                         detector,
169                         chunk_index,
170                         start_time,
171                         end_time,
172                         end_time - start_time,
173                         confidence,
174                         json.dumps(stats or {}),
175                         json.dumps(detection_params or {}),
176                     ),
177                 )
178                 conn.commit()
179                 return cursor.lastrowid
180         except Exception as e:
181             logger.error(f"Failed to add candidate segment: {e}")
182             return None
183
184     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
185         """Record that a candidate detection was confirmed as a given play_segment."""
186         return self._execute_write(

```

```

185         "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
186         (segment_id, candidate_id),
187         f"Failed to link candidate {candidate_id} to segment {segment_id}",
188     )
189
190     def query_candidate_segments(
191         self,
192         video_id: str,
193         detector: Optional[str] = None,
194         only_unlabeled: bool = False,
195     ) -> List[Dict[str, Any]]:
196         """Fetch candidate detections for the annotation / fine-tuning workflow.
197         `stats` and `detection_params` are deserialized from JSON."""
198         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
199         params: List[Any] = [video_id]
200
201         if detector:
202             query += " AND detector = ?"
203             params.append(detector)
204         if only_unlabeled:
205             query += " AND human_label IS NULL"
206
207         query += " ORDER BY start_time"
208
209         candidates = []
210         with self._connect() as conn:
211             rows = conn.cursor().execute(query, params).fetchall()
212             for row in rows:
213                 d = dict(row)
214                 d["stats"] = json.loads(d["stats"] or "{}")
215                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
216                 candidates.append(d)
217         return candidates
218
219     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
220         with self._connect() as conn:
221             row = (
222                 conn.cursor()
223                 .execute("SELECT * FROM videos WHERE id = ?", (video_id,))
224                 .fetchone()
225             )
226             if row:
227                 d = dict(row)
228                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
229                 return d
230         return None
231
232     def list_videos(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
233         """All indexed videos, most-recently-added first. Orders by rowid (a monotonic
234         insertion counter, since id is a TEXT PK) rather than the 1-second-granularity
235         processed_at, which would tie within a batch. Mirrors get_video's row shape (validation_results
236         parsed). Powers GET /api/videos for the operator console (PACKAGING_PLAN.md P1)."""
237         sql = "SELECT * FROM videos ORDER BY rowid DESC"
238         params: tuple = ()
239         if limit is not None:
240             sql += " LIMIT ?"
241             params = (int(limit),)
242         with self._connect() as conn:
243             rows = conn.cursor().execute(sql, params).fetchall()
244             out: List[Dict[str, Any]] = []
245             for row in rows:
246                 d = dict(row)

```

```

247         d["validation_results"] = json.loads(d["validation_results"] or "[]")
248     out.append(d)
249     return out
250
251 def query_play_segments(
252     self, video_id: str, filters: Dict[str, Any]
253 ) -> List[Dict[str, Any]]:
254     """
255     Dynamically queries play segments based on filters:
256     - duration_min: float
257     - server: string
258     - receiver: string
259     - event_type: string (e.g. smash, foul)
260     """
261     query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
262     params = [video_id]
263
264     if filters.get("duration_min"):
265         query += " AND r.duration >= ?"
266         params.append(filters["duration_min"])
267
268     if filters.get("server"):
269         query += " AND r.server = ?"
270         params.append(filters["server"])
271
272     if filters.get("receiver"):
273         query += " AND r.receiver = ?"
274         params.append(filters["receiver"])
275
276     if filters.get("event_type"):
277         # Check if there is an event of a specific type inside this segment
278         query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
279         params.append(filters["event_type"])
280
281     segments_list = []
282     with self._connect() as conn:
283         cursor = conn.cursor()
284         rows = cursor.execute(query, params).fetchall()
285
286         for row in rows:
287             r_dict = dict(row)
288             r_id = r_dict["id"]
289             r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
290
291             # Fetch associated events
292             event_rows = cursor.execute(
293                 "SELECT * FROM events WHERE segment_id = ?", (r_id,)
294             ).fetchall()
295             r_dict["events"] = []
296             for er in event_rows:
297                 e_dict = dict(er)
298                 e_dict["details"] = json.loads(e_dict["details"] or "{}")
299                 r_dict["events"].append(e_dict)
300
301             segments_list.append(r_dict)
302
303     return segments_list
304
305 def clear_video_data(self, video_id: str):
306     with self._connect() as conn:
307         conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
308         conn.commit()
309
310

```

29. assistant (2026-07-07T16:59:15.501Z)

Now the DB facade ? schema, migrations, and how `videos` is defined.

30. user (2026-07-07T16:59:16.557Z)

```
1     import logging
2     import os
3     import shutil
4     import sqlite3
5     from contextlib import contextmanager
6     from typing import Any, Tuple
7
8     from backend.infrastructure.database_jobs import DatabaseJobsMixin
9     from backend.infrastructure.database_media import DatabaseMediaMixin
10    from backend.infrastructure.database_user_models import DatabaseUserModelsMixin
11
12    logger = logging.getLogger(__name__)
13
14    # The highest schema version this binary knows how to migrate TO (the top of the
migration ladder
15    # in `__init_db`). PACKAGING_PLAN.md P2d uses it for the upgrade-safety contract on a
downloadable,
16    # auto-updating app: refuse to open a DB from a NEWER app (downgrade guard) and back the
DB up before
17    # an upgrade migration runs.
`tests/test_database.py::test_schema_version_matches_ladder` pins it to
18    # the ladder so adding a `_migrate_to_vN` without bumping this fails CI.
19    SCHEMA_VERSION = 9
20
21
22    class SchemaTooNewError(RuntimeError):
23        """The on-disk DB was created by a newer app version than this binary understands
(P2d)."""
24
25
26    class Database(DatabaseMediaMixin, DatabaseJobsMixin, DatabaseUserModelsMixin):
27        def __init__(self, db_path: str):
28            self.db_path = db_path
29            self._init_db()
30
31        @contextmanager
32        def _connect(self):
33            """Transaction scope that also CLOSES the connection.
34
35            ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
36            never closes, so every call leaked its connection to GC (lingering file
37            handles on the WAL sidecars under Windows). Internal methods use this.
38            """
39            conn = self._get_connection()
40            try:
41                with conn:
42                    yield conn
43            finally:
44                conn.close()
45
46        def _get_connection(self):
47            # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
48            # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
49            conn = sqlite3.connect(self.db_path, timeout=30.0)
50            conn.row_factory = sqlite3.Row
51            # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
52            conn.execute("PRAGMA foreign_keys = ON")
53            # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
```

```

54         conn.execute("PRAGMA busy_timeout = 30000")
55     try:
56         conn.execute("PRAGMA journal_mode = WAL")
57     except sqlite3.OperationalError:
58         pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
59     return conn
60
61 def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
62     """Run ONE bool-returning single-statement write inside a connection scope.
63
64     Captures the connect/commit/except-logger.error/return-False boilerplate shared
65     by
66     the bool-returning single-statement writers. ``error_msg`` is the caller's exact
67     per-method message (already formatted with its own ids) ? logged verbatim on a
68     swallowed write fault so each writer keeps its specific log text. Returns True
69     on
70     success, False (after logging ``error_msg``) on any exception."""
71     try:
72         with self._connect() as conn:
73             conn.cursor().execute(sql, params)
74             conn.commit()
75         return True
76     except Exception as e:
77         logger.error(f"{error_msg}: {e}")
78         return False
79
80 @staticmethod
81 def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
82     """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
83
84     A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
85     transaction, so an interrupted v3/v4 block can leave the column added while
86     ``user_version`` stays un-bumped ? and the next open would re-run the same
87     ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
88     future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
89     Swallow
90     ONLY that specific error so the ladder stays crash-safe, matching the re-
91     runnable
92     ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
93     """
94     try:
95         cursor.execute(ddl)
96     except sqlite3.OperationalError as exc:
97         if "duplicate column name" not in str(exc).lower():
98             raise
99
100 def _init_db(self):
101     """Initializes tables for videos, play_segments, and events."""
102     with self._connect() as conn:
103         cursor = conn.cursor()
104
105         self._create_base_tables(cursor)
106
107         # Versioned migrations live here. PRAGMA user_version is the schema
108         # contract ? bump it for every change and add an ordered `if version < N`
109         # block below. Tests assert the expected version, so accidental drift fails
110         CI.
111         version = cursor.execute("PRAGMA user_version").fetchone()[0]
112
113         # P2d upgrade-safety (no-op for a fresh DB or one already at
114         SCHEMA_VERSION):
115         # (1) DOWNGRADE GUARD ? a DB from a NEWER app (version > what we know) has
116         schema this
117         # binary can't reason about; opening it could corrupt data, so refuse
118         loudly.

```

```

111         if version > SCHEMA_VERSION:
112             raise SchemaTooNewError(
113                 f"This database (schema v{version}) was created by a newer version
of the app "
114                 f"than this one (knows up to v{SCHEMA_VERSION}). Upgrade the app, or
point it at "
115                 f"a different data directory (RALLY_APP_HOME). DB: {self.db_path}"
116             )
117         # (2) BACKUP-BEFORE-MIGRATE ? an EXISTING older DB is about to be upgraded
118         #     failed migration could corrupt it, so snapshot it first (best-effort:
WARN, never
119         #     block the upgrade). version 0 = a brand-new DB ? nothing to back up.
120         if 0 < version < SCHEMA_VERSION:
121             self._backup_before_migrate(conn, version)
122
123         if version < 1:
124             self._migrate_to_v1(cursor)
125
126         if version < 2:
127             self._migrate_to_v2(cursor)
128
129         if version < 3:
130             self._migrate_to_v3(cursor)
131
132         if version < 4:
133             self._migrate_to_v4(cursor)
134
135         if version < 5:
136             self._migrate_to_v5(cursor)
137
138         if version < 6:
139             self._migrate_to_v6(cursor)
140
141         if version < 7:
142             self._migrate_to_v7(cursor)
143
144         if version < 8:
145             self._migrate_to_v8(cursor)
146
147         if version < 9:
148             self._migrate_to_v9(cursor)
149         # future: if version < 10: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 10
150
151         conn.commit()
152
153     def _backup_before_migrate(
154         self, conn: sqlite3.Connection, from_version: int
155     ) -> None:
156         """Snapshot the DB to ``<db>.bak-v<from_version>`` before an upgrade migration
(P2d).
157
158         Checkpoints the WAL into the main file first so the copy is complete. Best-
effort: a backup
159         failure WARNS but does NOT block the upgrade (the ladder is itself crash-safe;
refusing to
160         start because a backup couldn't be written would be worse than proceeding
without one)."""
161         backup = f"{self.db_path}.bak-v{from_version}"
162         try:
163             conn.execute("PRAGMA wal_checkpoint(FULL)")
164             shutil.copy2(self.db_path, backup)
165             logger.info(
166                 "[DB] backed up v%s database before upgrading ? %s",

```

```

167         from_version,
168         backup,
169     )
170     except Exception as exc: # noqa: BLE001 ? the backup is a best-effort safety
net; its failure
171         # must NEVER block the in-place upgrade (the migration ladder is itself
crash-safe). Log loud.
172         logger.warning(
173             "[DB] could not back up the database before migrating from v%s (%s); "
174             "proceeding with the in-place upgrade.",
175             from_version,
176             exc,
177         )
178
179     def backup_database(self, dest: str) -> str:
180         """Write a consistent copy of the DB to ``dest`` (an 'export my data'
affordance, P2d).
181         Checkpoints the WAL first so ``dest`` is self-contained. Returns ``dest``."""
182         parent = os.path.dirname(os.path.abspath(dest))
183         if parent:
184             os.makedirs(parent, exist_ok=True)
185         with self._connect() as conn:
186             conn.execute("PRAGMA wal_checkpoint(FULL)")
187         shutil.copy2(self.db_path, dest)
188         return dest
189
190     def _create_base_tables(self, cursor):
191         """Create the videos / play_segments / events base tables (idempotent)."""
192         # Videos Table
193         cursor.execute("""
194             CREATE TABLE IF NOT EXISTS videos (
195                 id TEXT PRIMARY KEY,
196                 filepath TEXT NOT NULL,
197                 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
198                 status TEXT NOT NULL,
199                 validation_results TEXT
200             )
201         """)
202
203         # Play Segments Table (Extensible metadata JSON)
204         cursor.execute("""
205             CREATE TABLE IF NOT EXISTS play_segments (
206                 id INTEGER PRIMARY KEY AUTOINCREMENT,
207                 video_id TEXT NOT NULL,
208                 start_time REAL NOT NULL,
209                 end_time REAL NOT NULL,
210                 duration REAL NOT NULL,
211                 server TEXT,
212                 receiver TEXT,
213                 metadata TEXT,
214                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
215             )
216         """)
217
218         # Events Table
219         cursor.execute("""
220             CREATE TABLE IF NOT EXISTS events (
221                 id INTEGER PRIMARY KEY AUTOINCREMENT,
222                 segment_id INTEGER NOT NULL,
223                 timestamp REAL NOT NULL,
224                 type TEXT NOT NULL,
225                 player TEXT,
226                 details TEXT,
227                 FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
228             )

```



```

229         """)
230
231     def _migrate_to_v1(self, cursor):
232         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
233         # Common fields are columns; detector-specific metrics go in `stats` JSON,
234         # so a new segmenter reuses this table by setting its own `detector`/`stats`.
235         cursor.execute("""
236             CREATE TABLE IF NOT EXISTS candidate_segments (
237                 id INTEGER PRIMARY KEY AUTOINCREMENT,
238                 video_id TEXT NOT NULL,
239                 detector TEXT NOT NULL,
240                 chunk_index INTEGER,
241                 start_time REAL NOT NULL,
242                 end_time REAL NOT NULL,
243                 duration REAL NOT NULL,
244                 confidence REAL,
245                 stats TEXT,
246                 detection_params TEXT,
247                 confirmed_segment_id INTEGER,
248                 human_label TEXT,
249                 notes TEXT,
250                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
251                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
252                 FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
253             )
254         """)
255         cursor.execute("""
256             CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
257             ON candidate_segments(video_id, detector)
258         """)
259         cursor.execute("PRAGMA user_version = 1")
260
261     def _migrate_to_v2(self, cursor):
262         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
263         # /api/process request. `status` is the EXECUTION state of the job
264         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
265         # that failed validation) lives inside `result` JSON as result["status"].
266         # `result` is the full sync-era response dict (incl. report paths), so the
267         # observability report is the job's artifact, per the plan.
268         cursor.execute("""
269             CREATE TABLE IF NOT EXISTS jobs (
270                 id TEXT PRIMARY KEY,
271                 video_path TEXT NOT NULL,
272                 video_id TEXT,
273                 segmenter TEXT,
274                 player_pool TEXT,
275                 max_frames INTEGER,
276                 status TEXT NOT NULL DEFAULT 'queued',
277                 progress TEXT,
278                 error TEXT,
279                 result TEXT,
280                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
281                 started_at TIMESTAMP,
282                 finished_at TIMESTAMP
283             )
284         """)
285         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
286         cursor.execute("PRAGMA user_version = 2")
287
288     def _migrate_to_v3(self, cursor):
289         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
290         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
291         # due
292         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker

```

```

292         # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
293         # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
294         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
them.
295         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
296         self._add_column_idempotent(
297             cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP"
298         )
299         cursor.execute(
300             "CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status, next_poll_at)"
301         )
302         cursor.execute("PRAGMA user_version = 3")
303
304     def _migrate_to_v4(self, cursor):
305         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
306         # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
307         # path can scope rows to an owner. **NULL = local install = byte-identical to
308         # today** ? every existing row stays NULL and every existing query (which never
309         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
310         # the ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
them.
311         # Additive ONLY: no served-behavior change rides on this slice.
312         self._add_column_idempotent(
313             cursor, "ALTER TABLE videos ADD COLUMN user_id TEXT"
314         )
315         self._add_column_idempotent(
316             cursor, "ALTER TABLE candidate_segments ADD COLUMN user_id TEXT"
317         )
318         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
319         # of index churn while making the eventual per-user lookups cheap.
320         cursor.execute(
321             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
322             "WHERE user_id IS NOT NULL"
323         )
324         cursor.execute(
325             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
326             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL"
327         )
328         cursor.execute("PRAGMA user_version = 4")
329
330     def _migrate_to_v5(self, cursor):
331         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
332         # version): the resolved per-user `fusion_config` overlay + an INLINE
333         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
334         # and provenance that earned it. `is_active` pins the one row the serving bridge
335         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice
336         # only persists/loads it ? NOTHING reads it on the served path yet.
337         #
338         # user_id          ? owner of the model (NULL allowed = the local install)
339         # version          ? monotonic per-user model version
340         # baseline_version ? the shared baseline this was fit against (upgrade
switch)
341         # feature_schema_hash ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
342         # fusion_config    ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
343         # classifier_json  ? JSON LogisticRegression.to_dict() (None = config-only
model)
344         # promotion_evidence ? JSON per_user_gate PromotionDecision (why it was
promoted)
345         # n_videos_at_fit  ? held-out-user corpus size when fit (min_n trust floor
input)
346         # is_active       ? the one resolved row for this user (partial-unique

```

```

below)
347         cursor.execute("""
348             CREATE TABLE IF NOT EXISTS user_models (
349                 id INTEGER PRIMARY KEY AUTOINCREMENT,
350                 user_id TEXT,
351                 version INTEGER NOT NULL DEFAULT 1,
352                 baseline_version TEXT,
353                 feature_schema_hash TEXT,
354                 fusion_config TEXT,
355                 classifier_json TEXT,
356                 promotion_evidence TEXT,
357                 n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
358                 is_active INTEGER NOT NULL DEFAULT 0,
359                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
360             )
361         """)
362         # One model version per user (NULLs compare distinct in SQLite, so the local
363         # install can still carry multiple versioned rows; the active-row guard below
364         # is what enforces a single served model).
365         cursor.execute("""
366             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
367                 ON user_models(user_id, version)
368         """)
369         # At most ONE active model per user: a partial-unique index over the active
370         # (SQLite treats NULL user_id rows as distinct, so the local install's single
371         # active row is still guarded ? there is one local user.)
372         cursor.execute("""
373             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
374                 ON user_models(user_id) WHERE is_active = 1
375         """)
376         cursor.execute("PRAGMA user_version = 5")
377
378     def _migrate_to_v6(self, cursor):
379         # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost
380         # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records
381         # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is
382         # USD (matches GCP billing = one source of truth); INR is a display conversion
383         # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD
384         # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no
385         # served change.
386         for col, typ in (
387             ("owner_id", "TEXT"), # who pays (the non-owner user / tenant)
388             ("compute_backend", "TEXT"), # local_gpu | cloud_run | ...
389             ("gpu_type", "TEXT"), # L4 | T4 | ... (selects the per-second GPU rate)
390             ("gpu_active_seconds", "REAL"),
391             ("wall_seconds", "REAL"),
392             ("bytes_out", "INTEGER"), # egress
393             ("gemini_cost_usd", "REAL"), # provider-reported, already USD
394             ("pricebook_version", "TEXT"),
395             ("cost_usd", "REAL"), # computed total (USD = source of truth)
396             (
397                 "cost_breakdown_json",
398                 "TEXT",
399             ), # {gpu_seconds: .., egress_gb: .., gemini_usd: ..}
400         ):
401             self._add_column_idempotent(
402                 cursor, f"ALTER TABLE jobs ADD COLUMN {col} {typ}"
403             )
404         # Partial index keeps the NULL=no-cost fast path churn-free while making per-

```

```

owner billing cheap.
404         cursor.execute(
405             "CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
406             "WHERE owner_id IS NOT NULL"
407         )
408         # Versioned pricebook: one row per (version, resource, gpu_type). Rates change +
differ by
409         # gpu_type ? re-version (never mutate a shipped version, so a recorded cost
stays auditable).
410         cursor.execute("""
411             CREATE TABLE IF NOT EXISTS pricebook (
412                 version TEXT NOT NULL,
413                 resource TEXT NOT NULL,
414                 gpu_type TEXT NOT NULL DEFAULT '',
415                 unit_price_usd REAL NOT NULL DEFAULT 0.0,
416                 currency TEXT NOT NULL DEFAULT 'USD',
417                 effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
418             )
419         """)
420         cursor.execute(
421             "CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
422             "ON pricebook(version, resource, gpu_type)"
423         )
424         # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0
(nothing
425         # user-facing changes). The owner INSERTs a real, calibrated version (real GCP
$/GPU-s,
426         # egress $/GB) and flips billing.pricebook_version to it ? never overwrite this
seed.
427         for resource, gpu_type in (
428             ("gpu_seconds", "L4"),
429             ("gpu_seconds", "T4"),
430             ("wall_seconds", ""),
431             ("egress_gb", ""),
432         ):
433             cursor.execute(
434                 "INSERT OR IGNORE INTO pricebook (version, resource, gpu_type,
unit_price_usd, "
435                 "currency) VALUES ('placeholder-v0', ?, ?, 0.0, 'USD')",
436                 (resource, gpu_type),
437             )
438             cursor.execute("PRAGMA user_version = 6")
439
440         def _migrate_to_v7(self, cursor):
441             # v7: record the UI LOCALE a job was submitted in (the language the SPA was
rendered in,
442             # e.g. "kn", "hi-IN"). NULLABLE ? NULL = unrecorded = byte-identical to today
(CLI / older
443             # clients / pre-v7 rows). Enables PMF analytics ("which language markets use the
tool",
444             # count_jobs_by_locale) and pairs with the localized reel watermark (the SPA
sends the same
445             # locale to /api/compile). Additive-only ALTER; idempotent so an interrupted
upgrade re-runs.
446             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN locale TEXT")
447             cursor.execute("PRAGMA user_version = 7")
448
449         def _migrate_to_v8(self, cursor):
450             # v8: record the per-request COMPUTE choice the SPA compute-picker sent ("local"
forces the
451             # in-process segmenter; "cloud" forces a Cloud Run dispatch). It must survive
the async
452             # submit?worker boundary ? the worker reconstructs the ProcessRequest from this
row, so a
453             # NULL/unset value (CLI / older clients / pre-v8 rows) means "use the server

```

```

default", i.e.
454         # byte-identical to today. Additive-only ALTER; idempotent so an interrupted
upgrade re-runs.
455         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN compute TEXT")
456         cursor.execute("PRAGMA user_version = 8")
457
458     def _migrate_to_v9(self, cursor):
459         # v9: the jobs table now holds TWO kinds of work ? `kind`='process' (the
original, NULL ?
460         # process for pre-v9 rows) and `kind`='compile' (the async reel stitch, #329;
/api/compile
461         # is now 202+poll like /api/process so a slow ffmpeg stitch can't trip the edge
524). `params`
462         # is the kind-specific JSON payload a compile job carries
(segment_ids/output_filename/locale)
463         # since those don't map onto the process columns. Additive-only ALTERs;
idempotent.
464         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN kind TEXT")
465         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN params TEXT")
466         cursor.execute("PRAGMA user_version = 9")
467
468
469         # Domain CRUD methods live in mixins so Database remains the public facade while
470         # schema/migration/connection logic stays centralized in this module.
471

```

31. assistant (2026-07-07T17:02:40.981Z)

The DB layer is clear. Now let me check how the observability report consumes `val\_context` (since a cache hit that skips validation won't populate `ffprobe\_meta`), and find the existing test patterns for `\_execute\_processing`.

32. user (2026-07-07T17:02:44.311Z)

```

=== emit_indexer_report def ===
backend/reporting/__init__.py
=== val_context / ffprobe_meta usage in report ===
backend/main.py
backend/orchestration.py
backend/pipeline/validators/base.py
backend/pipeline/validators/format.py
backend/pipeline/validators/resolution_fps.py
backend/reporting/harvest.py
backend/reporting/__init__.py
backend/worker/__main__.py
=== test files touching _execute_processing / validation cache / add_video ===
tests/test_api_cloud_dispatch.py
tests/test_async_jobs.py
tests/test_base.py
tests/test_database.py
tests/test_job_worker_loop.py
tests/test_job_worker_module_seam.py
tests/test_main_module_budget.py
tests/__pycache__/test_api_cloud_dispatch.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_async_jobs.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_base.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_database.cpython-311-pytest-9.0.3.pyc
tests/__pycache__/test_database.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_database.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_job_worker_loop.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_job_worker_module_seam.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_main_module_budget.cpython-313-pytest-9.0.3.pyc

```

33. user (2026-07-07T17:02:49.605Z)

=== emit_indexer_report signature + val_context handling ===

```
29: def emit_indexer_report(  
36:     val_context: Optional[Dict[str, Any]] = None,  
65:     val_context=val_context,
```

34. user (2026-07-07T17:02:50.520Z)

```
33:     val_context: Optional[Dict[str, Any]], video_path: Optional[str]  
35:     """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source  
44:     ffmpegeta = (val_context or {}).get("ffprobe_meta") or {}  
209:     val_context: Optional[Dict[str, Any]],  
231:     vm = video_meta_from_context(val_context, video_path)
```

35. user (2026-07-07T17:02:55.705Z)

```
30  
31     # --- media metadata -----  
32     def video_meta_from_context(  
33         val_context: Optional[Dict[str, Any]], video_path: Optional[str]  
34     ) -> Dict[str, Any]:  
35         """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source  
36         distinguishes a real probe from the absence of one (the fallback footgun)."""  
37         meta: Dict[str, Any] = {"fps_source": "unknown"}  
38         if video_path and os.path.exists(video_path):  
39             try:  
40                 meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)  
41             except OSError:  
42                 pass  
43  
44         ffmpegeta = (val_context or {}).get("ffprobe_meta") or {}  
45         streams = ffmpegeta.get("streams") or []  
46         fmt = ffmpegeta.get("format") or {}  
47         if streams:  
48             s = streams[0]  
49             w, h = s.get("width"), s.get("height")  
50             meta["width"], meta["height"] = w, h  
51             if w and h:  
52                 meta["resolution"] = f"{w}x{h}"  
53             fps_str = s.get("r_frame_rate", "0/1")  
54             try:  
55                 num, den = (int(x) for x in fps_str.split("/"))  
56                 if den:  
57                     meta["fps"] = round(num / den, 2)  
58                     meta["fps_source"] = "probed"  
59             except (ValueError, ZeroDivisionError):  
60                 pass  
61             if s.get("codec_name"):  
62                 meta["container"] = fmt.get("format_name") or s.get("codec_name")  
63         if fmt.get("duration"):  
64             try:  
65                 meta["duration_s"] = round(float(fmt["duration"]), 1)  
66             except (ValueError, TypeError):  
67                 pass  
68         if fmt.get("format_name"):  
69             meta["container"] = fmt.get("format_name")  
70  
71         # Audio readiness signal (for Input Quality UX extension per review item 2).  
72         # Guidelines make "record with audio on, mic unobstructed" a first-class requirement  
73         # (ADR-007) because shuttle thwacks are a strong rally cue for recall.  
74         # We only record presence here (cheap, from ffprobe already in context); the actual  
75         # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /  
calibration  
76         # tool for now.  
77         audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]  
78         meta["audio_present"] = len(audio_streams) > 0
```

```

79         return meta
80
81
82     # --- stage builders -----
83     def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
84         with rec.stage("validation") as st:
85             total = len(val_results)
86             passed = sum(1 for r in val_results if getattr(r, "passed", False))
87             st.add_metric("checks_passed", passed)
88             st.add_metric("checks_total", total)
89             failures = [r for r in val_results if not getattr(r, "passed", True)]
90             for r in failures:
91                 st.messages.append(
92                     f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r, 'message',
93                                     '')}"
94                 )
95             st.status = "error" if failures else ("ok" if total else "not_run")
96
97     def _segmentation_stage(
98         rec: "obsreport.RunRecorder",
99         play_segments: List[Dict[str, Any]],
100         candidates: List[Dict[str, Any]],
101         segmenter_requested: Optional[str],
102         segmenter_actual: Optional[str],
103         config_default: Optional[str],
104         was_reingest: bool,
105         ran: bool,
106     ) -> None:
107         with rec.stage("segmentation") as st:
108             if not ran:
109                 st.status = "not_run"

```

36. assistant (2026-07-07T17:03:43.932Z)

`video\_meta\_from\_context` degrades gracefully when `ffprobe\_meta` is absent (sets `fps\_source="unknown"`, keeps `size\_mb`) ? so a cache hit that leaves `val\_context={}` yields a slightly degraded but crash-free report, and keeps the path truly ~0-cost (no re-probe). Good.

Now let me study the test patterns for `\_execute\_processing` ? how tests stub validators, segmenters, storage, db.

37. user (2026-07-07T17:03:47.042Z)

```

=== test files mentioning _execute_processing ===
tests/test_api_cloud_dispatch.py
tests/test_async_jobs.py
tests/test_base.py
tests/test_job_worker_module_seam.py
tests/test_main_module_budget.py
=== how tests build config/db + patch registry.run_all ===

```

38. user (2026-07-07T17:03:51.979Z)

```

=== test_base.py: _execute_processing usages with context ===
25: def test_registry():
53: def test_registry_safe_name_discovery_with_complex_ctor():
132: def test_result_contract_failure_vs_empty_success():
135:     Exercise the exact duck-typing patterns used in main.py (both the _execute_processing
161: def test_result_contract_success_path_and_unwrap():
175: def test_raising_segmenter_triggers_restore_contract():
181:     This allows the except blocks in main.py (_execute_processing:193-198 and the

```

39. user (2026-07-07T17:03:59.616Z)

Exit code 2

=== how tests invoke the process pipeline & stub validators/segmenters ===

```
101: db = Database(str(tmp_path / "t.db")) # noqa: F841
122:     m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
151: out = m._execute_processing(
170: m._execute_processing(
196: m._execute_processing(
213: m._execute_processing(
230: m._execute_processing(
241: (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on the
worker is
249: m._execute_processing(
262: out = m._execute_processing(
281: out = m._execute_processing(
307: out = m._execute_processing(
352: out = m._execute_processing(
376: out = m._execute_processing(
387: db = Database(str(tmp_path / "t.db")) # noqa: F841
395:     m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
416: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
417: out = m._execute_processing(
446: out = m._execute_processing(
460:     out = m._execute_processing(
499: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
501: out = m._execute_processing(
519: db = Database(str(tmp_path / "t.db")) # noqa: F841
537:     m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
544: out = m._execute_processing(
561: db = Database(str(tmp_path / "t.db")) # noqa: F841
568:     m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
575: out = m._execute_processing(
593:     out = m._execute_processing(
620: db = Database(str(tmp_path / "t.db"))
666: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
694: db = Database(str(tmp_path / "t.db")) # noqa: F841
712:     m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
756: out = m._execute_processing(
782: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
786: out = m._execute_processing(
803: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
807: out = m._execute_processing(
823: db = Database(str(tmp_path / "t.db")) # noqa: F841
828:     m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
834: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
838: out = m._execute_processing(
857: out = m._execute_processing(
884: out = m._execute_processing(
905: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
909: out = m._execute_processing(
929: out = m._execute_processing(
949: monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
951: out = m._execute_processing(
963: db = Database(str(tmp_path / "t.db")) # noqa: F841
```

=== conftest fixtures ===

40. assistant (2026-07-07T17:04:05.821Z)

This is exactly the harness I need. Let me read the test file's helpers and a couple of representative tests.

41. user (2026-07-07T17:04:07.545Z)

```
1      """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
```



```

3     Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
the rally
4     intervals back into the API DB, so /api/query + /api/compile (local stitch) are
unchanged ? all
5     GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6     done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7     dispatches; every other target runs the in-process segmenter byte-identically.
8     ""
9
10    import csv
11    import json
12    import os
13
14    import pytest
15
16    pytest.importorskip("httpx")
17    from fastapi.testclient import TestClient
18
19    import backend.compute as compute_pkg
20    import backend.main as m
21    from backend.compute.cloud_run import CloudRunJobClient
22    from backend.config.models import AppConfig
23    from backend.infrastructure.database import Database
24    from backend.infrastructure.execution_policy import ExecutionPolicy
25    from backend.pipeline.validators.base import ValidationResult
26
27    _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
28
29
30    def _pass():
31        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
32
33
34    class _InlineExecutor:
35        def submit(self, fn, *a, **k):
36            fn(*a, **k)
37
38
39    class _FakeCloudClient(CloudRunJobClient):
40        """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
41        into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""
42
43        def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
44            self.calls: list[list[str]] = []
45            self.video_id, self.returncode = video_id, returncode
46            self.rows = (
47                rows
48                if rows is not None
49                else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
50            )
51
52        def run_job(self, job_name, argv, *, region, project=None):
53            self.calls.append(list(argv))
54            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
55            done_uri = argv[argv.index("--done-uri") + 1]
56            if self.returncode == 0:
57                outputs = os.path.join(out_uri, "outputs", self.video_id)
58                os.makedirs(outputs, exist_ok=True)
59                with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
60                    json.dump(
61                        {

```

```

62         "video_id": self.video_id,
63         "segment_count": len(self.rows),
64         "status": "success",
65     },
66     f,
67 )
68     with open(
69         os.path.join(outputs, "intervals.csv"),
70         "w",
71         newline="",
72         encoding="utf-8",
73     ) as f:
74         w = csv.writer(f)
75         w.writerow(["start", "end", "shot_count", "ending_reason"])
76         for s, e, sc, er in self.rows:
77             w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
78     os.makedirs(os.path.dirname(done_uri), exist_ok=True)
79     with open(done_uri, "w", encoding="utf-8") as f:
80         json.dump(
81             {
82                 "video_id": self.video_id,
83                 "returncode": self.returncode,
84                 "segment_count": len(self.rows),
85                 "status": "success",
86             },
87             f,
88         )
89     return "exec-1"
90
91     def cancel_execution(self, execution):
92         raise AssertionError(
93             "cancel must never fire on the happy path"
94         ) # pragma: no cover
95
96
97     @pytest.fixture
98     def cloud_env(tmp_path, monkeypatch):
99         """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
100         bucket read an
101         identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
102         hash+validate."""
103         db = Database(str(tmp_path / "t.db")) # noqa: F841
104         monkeypatch.setattr(m, "db_instance", db)
105         monkeypatch.setattr(m, "db_instance", db)
106         bucket = tmp_path / "bucket"
107         bucket.mkdir()
108         cfg = AppConfig.model_validate(
109             { # noqa: F841
110                 "deployment": {"compute_target": "cloud_run"},
111                 "storage": {"backend": "local", "output_root": str(bucket)},
112             }
113         )
114         monkeypatch.setattr(m, "global_config", cfg)
115         monkeypatch.setattr(m, "global_config", cfg)
116         monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
117         # A real gs:// input would be fetched to hash+validate; return a local fake so that
118         # path is offline.
119         fake_local = tmp_path / "fetched.mp4"
120         fake_local.write_bytes(b"FAKEVIDEOBYTES")
121         monkeypatch.setattr(
122             m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
123         )
124         monkeypatch.setattr(
125             m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
126         )

```

```

124         return db, bucket, monkeypatch
125
126
127     def _inject_fake(monkeypatch, fake):
128         """Make backend.compute.get_compute_backend (re-imported inside
129         _maybe_dispatch_remote) build a
130         real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
131         token. ``**kw``
132         forwards the per-request ``target_override`` (item 3) so the cloud-picker path
133         resolves too."""
134         real = compute_pkg.get_compute_backend
135         monkeypatch.setattr(
136             compute_pkg,
137             "get_compute_backend",
138             lambda config, **kw: real(
139                 config, run_client=fake, policy=_FAST, token="tok", **kw
140             ),
141         )
142
143     def _meta(row):
144         md = row.get("metadata")
145         return json.loads(md) if isinstance(md, str) else (md or {})
146
147     def test_cloud_dispatch_ingests_segments(cloud_env):
148         db, _bucket, monkeypatch = cloud_env
149         fake = _FakeCloudClient()
150         _inject_fake(monkeypatch, fake)
151
152         out = m._execute_processing(
153             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
154         )
155
156         assert out["status"] == "success"
157         assert len(out["play_segments"]) == 2
158         rows = db.query_play_segments(out["video_id"], {})
159         assert len(rows) == 2
160         metas = [_meta(r) for r in rows]
161         assert all(md["source"] == "cloud_run" for md in metas)
162         assert {md.get("shot_count") for md in metas} == {5, 3}
163         assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
164
165     def test_cloud_dispatch_builds_correct_spec(cloud_env):
166         _db, bucket, monkeypatch = cloud_env
167         fake = _FakeCloudClient()
168         _inject_fake(monkeypatch, fake)
169
170         m._execute_processing(
171             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
172         )
173
174         assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
175         argv = fake.calls[0]
176         assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
177         assert argv[argv.index("--out-uri") + 1] == str(bucket)
178         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
179         joined = " ".join(argv)
180         assert (
181             "deployment.compute_target=local_gpu" in joined
182         ) # container runs INLINE (never re-dispatch)
183         assert "indexing.detector_impl=native" in joined
184         assert (
185             "indexing.skip_ai_handoff=false" in joined

```

```

186         ) # the engine's AI-handoff stance is forwarded
187
188
189     def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
190         """When the engine runs no-AI (skip_ai_handoff=True), the cloud job must too ? else
it tries the
191         Gemini handoff (which the cloud job has no key for) and yields 0 rallies. The flag
is forwarded."""
192         _db, _bucket, monkeypatch = cloud_env
193         m.global_config.indexing.skip_ai_handoff = True
194         fake = _FakeCloudClient()
195         _inject_fake(monkeypatch, fake)
196         m._execute_processing(
197             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
198         )
199         assert "indexing.skip_ai_handoff=true" in " ".join(fake.calls[0])
200
201
202     def test_cloud_dispatch_forwards_wasb_serving_knobs(cloud_env):
203         """The worker Job has no DEPLOYMENT_PROFILE, so it never applies the cloud-serving
preset ?
204         the control-plane MUST forward its resolved WASB serving knobs (decode_backend +
GPU-feed
205         tuning) or the worker silently reverts to the model defaults (cpu decode, batch 8),
making the
206         preset flip (#506/#507) a no-op on the real dispatch path. Regression guard for that
wiring."""
207         _db, _bucket, monkeypatch = cloud_env
208         m.global_config.indexing.wasb.decode_backend = "nvdec_resident"
209         m.global_config.indexing.wasb.batch_size = 64
210         m.global_config.indexing.wasb.prefetch_batches = 4
211         fake = _FakeCloudClient()
212         _inject_fake(monkeypatch, fake)
213         m._execute_processing(
214             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
215         )
216         joined = " ".join(fake.calls[0])
217         assert "indexing.wasb.decode_backend=nvdec_resident" in joined
218         assert "indexing.wasb.batch_size=64" in joined
219         assert "indexing.wasb.prefetch_batches=4" in joined
220
221
222     def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
223         """decode_backend defaults to cpu (always forwarded ? harmless, keeps worker ==
control-plane),
224         but batch_size/prefetch_batches default to None and must NOT be forwarded (None =
the worker's
225         parity default; forwarding an empty value would reach argparse as a bad token)."""
226         _db, _bucket, monkeypatch = cloud_env
227         # defaults: decode_backend="cpu", batch_size=None, prefetch_batches=None
228         fake = _FakeCloudClient()
229         _inject_fake(monkeypatch, fake)
230         m._execute_processing(

```

42. assistant (2026-07-07T17:04:15.771Z)

Let me see the in-process segmenter test setup (the `\_InProcSeg` stub and a segmentation-failure test) ? that's the exact scenario the cache targets.

43. user (2026-07-07T17:04:17.596Z)

```

380         assert "cloud input" in out["message"]
381         assert fake.calls == [] and called == [] # neither staged nor dispatched
382
383

```

```

384 def test_default_off_runs_in_process(tmp_path, monkeypatch):
385     """compute_target=' ' ? get_compute_backend returns None ? the in-process segmenter
runs
386     byte-identically (the gate is inert). No Cloud Run dispatch."""
387     db = Database(str(tmp_path / "t.db")) # noqa: F841
388     monkeypatch.setattr(m, "db_instance", db)
389     monkeypatch.setattr(m, "db_instance", db)
390     cfg = AppConfig() # compute_target defaults to "" # noqa: F841
391     monkeypatch.setattr(m, "global_config", cfg)
392     monkeypatch.setattr(m, "global_config", cfg)
393     monkeypatch.setattr(m, "global_config", cfg)
394     monkeypatch.setattr(
395         m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
396     )
397     video = tmp_path / "v.mp4"
398     video.write_bytes(b"hashable-bytes")
399
400     class _InProcSeg:
401         def __init__(self, db_, cfg_):
402             self.db = db_
403
404         def process_video(self, vid, path, *a, **k):
405             sid = self.db.add_play_segment(vid, 0.0, 5.0)
406             return [
407                 {
408                     "id": sid,
409                     "start_time": 0.0,
410                     "end_time": 5.0,
411                     "duration": 5.0,
412                     "metadata": {},
413                 }
414             ]
415
416     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
417     out = m._execute_processing(
418         m.ProcessRequest(video_path=str(video), segmenter="motion")
419     )
420     assert out["status"] == "success"
421     assert len(out["play_segments"]) == 1
422     assert "cloud_run" not in out["message"]
423
424
425 def test_cloud_ingest_failure_marks_failed_no_leak(cloud_env):
426     """A rc=0 marker but a MISSING intervals.csv (a partial push) must fail cleanly +
leak no rows
427     (destroy-AFTER-confirm), not 500 or hang."""
428     db, _bucket, monkeypatch = cloud_env
429
430     class _NoCsvClient(_FakeCloudClient):
431         def run_job(self, job_name, argv, *, region, project=None):
432             self.calls.append(list(argv))
433             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
434             done_uri = argv[argv.index("--done-uri") + 1]
435             outputs = os.path.join(out_uri, "outputs", self.video_id)
436             os.makedirs(outputs, exist_ok=True)
437             with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
438                 json.dump({"video_id": self.video_id, "status": "success"}, f)
439             # deliberately DO NOT write intervals.csv
440             os.makedirs(os.path.dirname(done_uri), exist_ok=True)
441             with open(done_uri, "w", encoding="utf-8") as f:
442                 json.dump({"video_id": self.video_id, "returncode": 0}, f)
443             return "exec"
444
445     _inject_fake(monkeypatch, _NoCsvClient())
446     out = m._execute_processing(

```

```

447         m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
448     )
449     assert out["status"] == "failed"
450     assert "could not read the result" in out["message"]
451     assert db.query_play_segments(out["video_id"], {}) == []
452
453
454     def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
455         """If the worker re-hashes different bytes (result.video_id != local id), ingest
456         still proceeds
457         under the LOCAL id but logs a WARNING so the orphaned-report condition is
458         diagnosable."""
459         db, _bucket, monkeypatch = cloud_env
460         _inject_fake(monkeypatch, _FakeCloudClient(video_id="DIVERGED"))
461         with caplog.at_level("WARNING", logger="backend.main"):
462             out = m._execute_processing(
463                 m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
464             )
465             assert out["status"] == "success"
466             assert len(db.query_play_segments(out["video_id"], {})) == 2
467             assert any(
468                 "DIVERGED" in r.getMessage() and "!= local" in r.getMessage()
469                 for r in caplog.records
470             )
471
472     class _InProcSeg:
473         """Minimal in-process segmenter (writes one play row) ? proves the local path
474         actually ran."""
475
476         def __init__(self, db_, cfg_):
477             self.db = db_
478
479         def process_video(self, vid, path, *a, **k):
480             sid = self.db.add_play_segment(vid, 0.0, 5.0)
481             return [
482                 {
483                     "id": sid,
484                     "start_time": 0.0,
485                     "end_time": 5.0,
486                     "duration": 5.0,
487                     "metadata": {},
488                 }
489             ]
490
491     # --- item 3: SPA compute-picker (per-request compute override + cloud_available
492     # capability) ---
493
494     def test_per_request_local_forces_in_process(cloud_env):
495         """Server configured for cloud (compute_target=cloud_run), but a per-request
496         compute='local'
497         forces the in-process segmenter and NEVER dispatches to Cloud Run."""
498         db, _bucket, monkeypatch = cloud_env
499         fake = _FakeCloudClient()
500         _inject_fake(monkeypatch, fake)
501         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
502
503         out = m._execute_processing(
504             m.global_config,
505             m.db_instance,
506             m.ProcessRequest(
507                 video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
508             ),

```

```

507         )
508         assert out["status"] == "success"
509         assert len(out["play_segments"]) == 1
510         assert "cloud_run" not in out["message"]
511         assert (
512             fake.calls == []
513         ) # explicit "run on my machine" ? no dispatch despite a cloud server
514
515
516     def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
517         """Server default is LOCAL (compute_target=''), but a per-request compute='cloud'
518         forces a Cloud
519         Run dispatch when the control plane is wired (CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT +
520         a bucket)."""
521         db = Database(str(tmp_path / "t.db")) # noqa: F841

```

44. assistant (2026-07-07T17:04:34.626Z)

I have the full harness. Let me check the schema-version ladder test (I'll bump to v10) and how DB tests are structured.

45. user (2026-07-07T17:04:36.615Z)

```

=== schema version ladder test ===
107: def test_schema_version_is_current(tmp_path):
111:     version = conn.execute("PRAGMA user_version").fetchone()[0]
140:     version = conn.execute("PRAGMA user_version").fetchone()[0]
279:     conn.execute("PRAGMA user_version = 3")
284:     # Re-open: the migration ladder must carry the v3 DB up to the head (v9) without data
    loss.
287:     assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
311:     on its own, so a crash after the ADD COLUMNS but before the ``PRAGMA user_version``
313:     bare ALTERS. The ladder must tolerate that (re-runnable, like the v1/v2/v5 blocks)
321:     # but the process died before the version bump, so user_version is stuck at 2.
324:     conn.execute("PRAGMA user_version = 2")
334:     assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
426: def test_upsert_is_idempotent_on_user_version(tmp_path):

```

46. user (2026-07-07T17:04:45.278Z)

```

100         # NOT NULL / PK flags on the load-bearing columns (cid, name, type, notnull, dflt,
    pk)
101         by_name = {c[1]: c for c in cols}
102         assert by_name["id"][5] == 1 # primary key
103         for col in ("video_id", "detector", "start_time", "end_time", "duration"):
104             assert by_name[col][3] == 1, f"{col} should be NOT NULL"
105
106
107     def test_schema_version_is_current(tmp_path):
108         db_path = tmp_path / "test.db"
109         Database(str(db_path))
110         conn = sqlite3.connect(str(db_path))
111         version = conn.execute("PRAGMA user_version").fetchone()[0]
112         conn.close()
113         # v9 = async compile job (kind/params); v8 = compute-picker; v7 = job UI locale; v6
    = cost ledger.
114         # Bump with each migration.
115         assert version == 9
116
117
118     def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
119         db_path = tmp_path / "legacy.db"
120         # Simulate a pre-v1 database that only has the old tables and no version.
121         conn = sqlite3.connect(str(db_path))

```

```

122     conn.execute(
123         "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status TEXT)"
124     )
125     conn.commit()
126     conn.close()
127
128     # Opening with the new Database should add candidate_segments without error...
129     Database(str(db_path))
130     # ...and running init again must be a no-op (idempotent).
131     Database(str(db_path))
132
133     conn = sqlite3.connect(str(db_path))
134     tables = [
135         r[0]
136         for r in conn.execute(
137             "SELECT name FROM sqlite_master WHERE type='table'"
138         ).fetchall()
139     ]
140     version = conn.execute("PRAGMA user_version").fetchone()[0]
141     conn.close()
142     assert "candidate_segments" in tables
143     assert "jobs" in tables # v2 (async job model, #16)
144     assert "user_models" in tables # v5 (per-user model store)
145     assert "pricebook" in tables # v6 (per-job cost ledger, M8)
146     assert (
147         version == 9
148     ) # v9 = async compile (kind/params); v8 = compute-picker; v7 = locale
149
150
151     def test_candidate_round_trip(tmp_path):
152         db = Database(str(tmp_path / "test.db"))
153         db.add_video("vid1", "vid1.mp4", "processed", [])
154
155         cid = db.add_candidate_segment(
156             "vid1",
157             "yolo_hybrid",
158             10.0,
159             25.0,
160             chunk_index=0,
161             confidence=0.42,
162             stats={"peak_velocity": 0.8, "track_id_count": 2},
163             detection_params={"frame_skip": 3},
164         )
165         assert cid is not None
166
167         cands = db.query_candidate_segments("vid1")
168         assert len(cands) == 1
169         c = cands[0]

```

47. user (2026-07-07T17:04:49.447Z)

```

270     )
271     conn = sqlite3.connect(str(db_path))
272     conn.execute("DROP INDEX IF EXISTS idx_videos_user")
273     conn.execute("DROP INDEX IF EXISTS idx_candidates_user")
274     conn.execute("DROP TABLE IF EXISTS user_models")
275     # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind
version
276     # and drop the v4/v5 artifacts that a true v3 DB wouldn't have).
277     conn.execute("ALTER TABLE videos DROP COLUMN user_id")
278     conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
279     conn.execute("PRAGMA user_version = 3")
280     conn.commit()
281     conn.close()
282     assert "user_id" not in _table_cols(db_path, "videos")

```



```

283
284 # Re-open: the migration ladder must carry the v3 DB up to the head (v9) without
data loss.
285 db2 = Database(str(db_path))
286 conn = sqlite3.connect(str(db_path))
287 assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
288 tables = {
289     r[0]
290     for r in conn.execute(
291         "SELECT name FROM sqlite_master WHERE type='table'"
292     ).fetchall()
293 }
294 assert "user_models" in tables # v5
295 assert "pricebook" in tables # v6 (cost ledger)
296 conn.close()
297 assert "user_id" in _table_cols(db_path, "videos")
298 assert "cost_usd" in _table_cols(
299     db_path, "jobs"
300 ) # v6 cost columns back-filled (NULL)
301 assert "user_id" in _table_cols(db_path, "candidate_segments")
302
303 v = db2.get_video("vidA")
304 assert v is not None and v["validation_results"] == [{"v": 1}]
305 cands = db2.query_candidate_segments("vidA")
306 assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3
307
308
309 def test_migration_idempotent_on_interrupted_upgrade(tmp_path):
310     """An INTERRUPTED v3/v4 upgrade must not brick the DB. A DDL ``ALTER`` auto-commits
311     on its own, so a crash after the ADD COLUMNS but before the ``PRAGMA user_version``
312     bump leaves the columns present while the version is stale ? re-opening re-runs the
313     bare ALTERS. The ladder must tolerate that (re-runnable, like the v1/v2/v5 blocks)
314     instead of raising ``OperationalError: duplicate column name``."""
315     db_path = tmp_path / "interrupted.db"
316     Database(
317         str(db_path)
318     ) # fully migrate to the current version (v3/v4 columns all present)
319
320 # Simulate the interruption: the v3/v4 ADD COLUMNS already landed and auto-
committed,
321 # but the process died before the version bump, so user_version is stuck at 2.
322 conn = sqlite3.connect(str(db_path))
323 conn.execute("DROP TABLE IF EXISTS user_models") # a true v2 DB lacks the v5 table
324 conn.execute("PRAGMA user_version = 2")
325 conn.commit()
326 conn.close()
327
328 # Re-opening re-runs v3 (jobs.policy/next_poll_at), v4
(videos/candidate_segments.user_id),
329 # v5 (user_models), v6 (jobs cost columns + pricebook), v7 (jobs.locale), v8
(jobs.compute) and
330 # v9 (jobs.kind/params) over artifacts that ALREADY exist ? must NOT raise (re-
runnable ALTERS /
331 # INSERT OR IGNORE), and must reach v9.
332 Database(str(db_path))
333 conn = sqlite3.connect(str(db_path))
334 assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
335 conn.close()
336 # Columns/tables are all present after the clean re-apply.
337 assert "policy" in _table_cols(db_path, "jobs")
338 assert "cost_usd" in _table_cols(db_path, "jobs") # v6 re-applied cleanly
339 assert "user_id" in _table_cols(db_path, "videos")
340 assert "user_id" in _table_cols(db_path, "candidate_segments")
341 conn = sqlite3.connect(str(db_path))
342 assert "user_models" in {

```

```
343         r[0]
344         for r in conn.execute(
```

48. assistant (2026-07-07T17:06:36.032Z)

I have a complete picture. Let me verify a couple of final details before implementing: the `ProcessRequest` model (does it have `force`?) and any other test that hardcodes the schema version or enumerates tables.

49. user (2026-07-07T17:06:39.118Z)

```
=== ProcessRequest fields ===
59:     enforce_startup_checks,
66: from backend.pipeline.segmenters.base import segmenter_registry
94: from backend.utils.hashing import compute_video_id # canonical file-identity hashing
111:     ``request.app.state`` ? eliminating the Optional module-level globals that forced
258: class ProcessRequest(BaseModel):
259:     video_path: str
262:     segmenter: Optional[str] = None
266:     # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
267:     # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268:     # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
269:     compute: Optional[str] = None
271:     # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
272:     force: bool = False
289:     # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content hash.
329:     (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
337:     "default_segmenter": getattr(indexing, "default_segmenter", None),
366:     """Honest probe of what THIS box can do ? the first-run wizard renders the compute step
from
387:     "segmenters": sorted(segmenter_registry.list_available().keys()),
388:     "default_segmenter": getattr(indexing, "default_segmenter", None),
401:     "compute_target": getattr(deployment, "compute_target", "") or "",

=== other schema-version / table-set assertions in tests ===
tests/test_async_jobs.py:106:         assert conn.execute("PRAGMA user_version").fetchone()[0] ==
9
tests/test_async_jobs.py:146:         assert c.execute("PRAGMA user_version").fetchone()[0] == 9
tests/test_async_jobs.py:148:         "SELECT name FROM sqlite_master WHERE type='table' AND
name='jobs'"
tests/test_async_jobs.py:155:         "SELECT name FROM sqlite_master WHERE type='table'"
tests/test_database.py:24:     cursor.execute("SELECT name FROM sqlite_master WHERE
type='table'")
tests/test_database.py:115:     assert version == 9
tests/test_database.py:137:         "SELECT name FROM sqlite_master WHERE type='table'"
tests/test_database.py:147:         version == 9
tests/test_database.py:287:     assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
tests/test_database.py:291:         "SELECT name FROM sqlite_master WHERE type='table'"
tests/test_database.py:334:     assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
tests/test_database.py:345:         "SELECT name FROM sqlite_master WHERE type='table'"
tests/test_db_upgrade_safety.py:8: from backend.infrastructure.database import SCHEMA_VERSION,
Database, SchemaTooNewError
tests/test_db_upgrade_safety.py:29:     """Ties the SCHEMA_VERSION constant to the migration
ladder: a freshly-built DB lands exactly at
tests/test_db_upgrade_safety.py:30:     SCHEMA_VERSION. Adding a _migrate_to_vN without bumping
the constant fails here."""
tests/test_db_upgrade_safety.py:33:     assert _user_version(p) == SCHEMA_VERSION
tests/test_db_upgrade_safety.py:45:     Database(p) # ? SCHEMA_VERSION
tests/test_db_upgrade_safety.py:53:     _set_user_version(p, SCHEMA_VERSION + 5) # a DB from a
hypothetical newer app
tests/test_db_upgrade_safety.py:67:     assert _user_version(p) == SCHEMA_VERSION # upgraded
```

```

tests/test_db_upgrade_safety.py:81:    assert _user_version(dest) == SCHEMA_VERSION
tests/test_eval_harness.py:72:    assert len(report.stages["final"].pr_curve) == 9
tests/test_eval_metrics.py:151:    assert len(curve) == 9
tests/test_import_smoke.py:122:    "assert rs.FEATURE_SCHEMA['version'] ==
rs.FEATURE_SCHEMA_VERSION\n"
tests/test_job_worker_loop.py:341:    assert conn.execute("PRAGMA
user_version").fetchone()[0] == 9
tests/test_job_worker_loop.py:359:    assert c.execute("PRAGMA user_version").fetchone()[0]
== 9
tests/test_locale_analytics.py:34:    assert _user_version(path) == 9 # head: v7 added locale,
v8 compute, v9 compile job
tests/test_locale_analytics.py:51:    assert _user_version(path) == 9
tests/test_training_gen0.py:112:    from backend.features.rally_seq import FEAT_NAMES,
FEATURE_SCHEMA_VERSION
tests/test_training_gen0.py:115:    assert payload["feature_schema_version"] ==
FEATURE_SCHEMA_VERSION

```

50. assistant (2026-07-07T17:06:47.693Z)

`ProcessRequest.force` exists (default `False`). Four test files hardcode `user\_version == 9` (async_jobs, database, job_worker_loop, locale_analytics); `test\_db\_upgrade\_safety.py` uses the `SCHEMA\_VERSION` constant so it auto-tracks. I have everything I need.

Let me set up a task list and mark a chapter, then implement.

51. user (2026-07-07T17:06:52.478Z)

Chapter marked: "Validation-result cache". Continue your current work.

52. user (2026-07-07T17:06:54.483Z)

Task #1 created successfully: Add config_fingerprint to ValidationRegistry

53. user (2026-07-07T17:06:56.088Z)

Task #2 created successfully: Add validation_cache table (migration v10) + DB methods

54. user (2026-07-07T17:06:57.483Z)

Task #3 created successfully: Wire cache into _execute_processing

55. user (2026-07-07T17:06:59.483Z)

Task #4 created successfully: Tests: cache reuse, config-change, force, DB round-trip, schema bump

56. user (2026-07-07T17:07:00.915Z)

Task #5 created successfully: Pre-merge gate: full suite + ruff + mypy + coverage

57. assistant (2026-07-07T17:07:07.599Z)

Per branching-safety, let me cut a feature branch from `origin/master` before editing.

58. user (2026-07-07T17:07:09.755Z)

branch 'feat/validation-result-cache' set up to track 'origin/master'.

Switched to a new branch 'feat/validation-result-cache'

=== now on ===

feat/validation-result-cache

ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker (#515)

59. assistant (2026-07-07T17:07:11.855Z)

Now the implementation. Task #1 ? the fingerprint on `ValidationRegistry`:

60. user (2026-07-07T17:07:12.365Z)

Updated task #1 status

61. user (2026-07-07T17:07:21.597Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\validators\base.py has been updated successfully. (file state is current in your context ? no need to Read it back)

62. assistant (2026-07-07T17:07:24.001Z)

Now add the `config\_fingerprint` method to `ValidationRegistry`:

63. user (2026-07-07T17:07:35.607Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\validators\base.py has been updated successfully. (file state is current in your context ? no need to Read it back)

64. user (2026-07-07T17:07:38.572Z)

Updated task #1 status

65. user (2026-07-07T17:07:38.614Z)

Updated task #2 status

66. assistant (2026-07-07T17:07:41.528Z)

Task #2 ? the migration and DB methods. First the schema bump + ladder + migration in `database.py`:

67. user (2026-07-07T17:07:42.900Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\infrastructure\database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

68. user (2026-07-07T17:07:48.850Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\infrastructure\database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

69. user (2026-07-07T17:08:07.099Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\infrastructure\database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

70. assistant (2026-07-07T17:08:10.021Z)

Now the DB methods in `database\_media.py` (`set/get\_validation\_cache`) and the `clear\_video\_data` cleanup:

71. user (2026-07-07T17:08:21.068Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-

indexer\backend\infrastructure\database_media.py has been updated successfully. (file state is current in your context ? no need to Read it back)

72. user (2026-07-07T17:08:27.240Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\infrastructure\database_media.py has been updated successfully. (file state is current in your context ? no need to Read it back)

73. user (2026-07-07T17:08:30.200Z)

Updated task #2 status

74. user (2026-07-07T17:08:30.431Z)

Updated task #3 status

75. assistant (2026-07-07T17:08:32.937Z)

Task #3 ? wire the cache into `\_execute\_processing`. First the import:

76. user (2026-07-07T17:08:34.160Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

77. assistant (2026-07-07T17:08:36.526Z)

Now the core change ? the validation-cache short-circuit:

78. user (2026-07-07T17:08:50.942Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

79. assistant (2026-07-07T17:08:56.693Z)

Let me verify the edited region reads correctly in full context:

80. user (2026-07-07T17:08:57.600Z)

```
780         try:
781             if (
782                 existing_video
783                 and existing_video.get("status") == "processed"
784                 and not req.force
785             ):
786                 cached_segments = db.query_play_segments(video_id, {})
787                 if cached_segments:
788                     logger.info(
789                         "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
790                         video_id,
791                     )
792                 response = _cached_process_payload(
793                     video_id,
794                     existing_video.get("validation_results", []),
795                     cached_segments,
796                     getattr(req, "compute", None),
797                 )
798                 return response
```

```

799
800      # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
801      #      UNLESS this exact content already has a cached verdict under the current
validator config.
802      notify("validating")
803      # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
804      # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
805      # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
806      # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
807      # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
808      # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
809      # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
810      # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
811      # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
812      # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
813      # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
814      val_fingerprint = registry.config_fingerprint(config)
815      cached_validation = None
816      if not req.force:
817          try:
818              cached_validation = db.get_validation_cache(video_id)
819          except Exception: # noqa: BLE001 ? a cache-read fault must degrade to a
real validation
820              logger.warning(
821                  "[API] validation-cache read failed for video_id=%s ? re-
validating.",
822                      video_id,
823                      exc_info=True,
824                  )
825      reused_validation = bool(
826          cached_validation
827          and cached_validation.get("fingerprint") == val_fingerprint
828      )
829      if reused_validation:
830          # Replay the stored ValidationResult list; val_context stays {} (no fresh
ffprobe_meta ?
831          # the report's media block degrades gracefully to size-only, which is the
deliberate cost
832          # of skipping the decode). Segmentation reads neither, so it is unaffected.
833          val_results = [
834              ValidationResult(**r) for r in cached_validation.get("results", [])
835          ]
836          logger.info(
837              "[API] Reusing cached validation verdict (passed=%s) for video_id=%s; "
838              "validator config unchanged ? skipping the validator suite. force=true
re-validates.",
839                  cached_validation.get("passed"),
840                  video_id,
841              )
842      else:
843          logger.info(f"Running validations for {req.video_path}...")
844          val_results, val_context = registry.run_all_with_context(local_video,
config)

```

```

845         failures = [r for r in val_results if not r.passed]
846         if not reused_validation:
847             # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
848             # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
849             db.set_validation_cache(
850                 video_id,
851                 val_fingerprint,
852                 not failures,
853                 [r.model_dump() for r in val_results],
854             )
855
856         if failures:
857             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
858             # status; it no longer destroys the video's prior good segments.
859             db.add_video(
860                 video_id,
861                 req.video_path,
862                 "failed",
863                 [r.model_dump() for r in val_results],
864             )
865             response = {
866                 "video_id": video_id,
867                 "status": "failed",
868                 "message": _validation_failure_message(failures),
869                 "results": [r.model_dump() for r in val_results],

```

81. assistant (2026-07-07T17:09:10.883Z)

The wiring looks correct. Let me run a quick smoke test to catch any import/logic errors before writing the full test suite.

82. user (2026-07-07T17:09:22.020Z)

```

SCHEMA_VERSION = 10
fingerprint stable across identical config: True (len 64 )
fingerprint changes on blur threshold change: True
fingerprint changes on sport change: True
validators in fingerprint: ['blur_check', 'format_check', 'pts_continuity_check',
'relevance_check', 'resolution_fps_check', 'scene_cut_check']
user_version = 10
validation_cache table present: True
get on empty: None
round-trip: True True True
upsert overwrite: True True True
cleared: True

```

83. assistant (2026-07-07T17:09:58.147Z)

The core works end-to-end. Now let me check the `Failure`/`Success` result contract so my test segmenter stubs match, then write the tests.

84. user (2026-07-07T17:09:59.265Z)

```

1      """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3      This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5      The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6      (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8      We use simple dataclasses so callers can use isinstance checks or structural pattern

```

```

matching.
9      This is intentionally small and incremental ? we start by applying it to the segmenter
10     and provider layers (the biggest sources of silent [] returns today).
11
12     The types are generic where it makes sense. Callers can still access .value on Success
13     or .message on Failure.
14
15     This module has no dependencies on the rest of the app.
16     """
17
18     from __future__ import annotations
19
20     from dataclasses import dataclass, field
21     from typing import Any, Generic, List, Optional, Tuple, TypeVar, Union
22
23     T = TypeVar("T")
24
25
26     @dataclass
27     class Success(Generic[T]):
28         """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30         value: T
31         warnings: List[str] = field(default_factory=list)
32
33         def is_success(self) -> bool:
34             return True
35
36         def unwrap(self) -> T:
37             """Return the value. Only safe to call if you already know it's a Success."""
38             return self.value
39
40
41     @dataclass
42     class Failure:
43         """Represents a failure with a human-readable message and optional details."""
44
45         message: str
46         details: Optional[dict[str, Any]] = None
47         is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
48
49         def is_success(self) -> bool:
50             return False
51
52         def unwrap(self) -> Any:
53             """Raise to make misuse obvious in code that assumes success."""
54             raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57     # Convenient alias for use in type hints: Result[list[dict], Failure]
58     # Result[ValueType] means Success[ValueType] | Failure
59     # (the Failure "error" type is fixed for this contract)
60     Result = Union[Success[T], Failure]
61
62
63     def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64         """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66         - ``Failure`` -> ``(None, failure)``
67         - ``Success(value=v)`` -> ``(v, None)``
68         - a bare value (legacy code that predates this contract and still returns a raw
list)
69         -> ``(value, None)``
70

```



```

71         This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72         that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73         on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74         (Success | Failure | legacy bare value)."""
75         if isinstance(result, Failure):
76             return None, result
77         value = result.value if hasattr(result, "value") else result
78         return value, None
79

```

85. assistant (2026-07-07T17:11:18.481Z)

``Failure(message)`` / ``Success(value=...)`` confirmed. Now let me write the integration + unit tests.

86. user (2026-07-07T17:11:19.015Z)

Updated task #3 status

87. user (2026-07-07T17:11:19.198Z)

Updated task #4 status

88. assistant (2026-07-07T17:12:08.561Z)

"""Validation-result cache keyed by video_id (task 4585aa00).

``video\_id`` is the MD5 of the file bytes (backend/utils/hashing.py), so an identical-content REUPLOAD keys the same cache row. When that content was already validated under the SAME validator configuration, ``\_execute\_processing`` reuses the stored verdict instead of re-running the (minutes-long on a heavy 4K clip) validator suite. This is distinct from the pre-existing whole-pipeline cache, which only short-circuits on a prior SUCCESSFUL segmentation ? so a reupload after a SEGMENTATION FAILURE used to re-pay the full validation from scratch (the 2026-07-07 CUJ).

These tests drive ``\_execute\_processing`` with a stubbed validator suite (a spy that COUNTS how many times it actually runs) + a stubbed segmenter, so the assertions are on validation-run count, not on real decoding. Cardinal guarantees exercised: same content + same config ? validators run ONCE across two uploads; a validation-config change forces a re-run; ``force=true`` bypasses; a cached FAIL is replayed without a re-decode; a different content never reuses another video's verdict.

"""

```
import logging
```

```

import backend.main as m
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
from backend.pipeline.validators.base import ValidationResult, registry
from backend.results import Failure

```

```
_ORCH_LOGGER = "backend.orchestration"
```

```

def _pass():
    return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]

```

```

def _fail():
    return [
        ValidationResult(
            validator_name="blur_check",
            passed=False,
            message="too blurry",
            details={},
        )
    ]

class _ValSpy:
    """Stands in for ``registry.run_all_with_context`` and counts real invocations.

    A cache HIT must NOT call this ? ``calls`` staying flat across a second upload is the proof
    that
    validation was skipped (the whole point of the cache)."""

    def __init__(self, results):
        self._results = results
        self.calls = 0

    def __call__(self, *a, **k):
        self.calls += 1
        return list(self._results), {}

class _FailSeg:
    """A segmenter whose detection FAILS (Failure result) ? reproduces "validation passed,
    detection
    failed ? no cached segments", the exact state the whole-pipeline cache can't short-
    circuit."""

    def __init__(self, db_, cfg_):
        pass

    def process_video(self, *a, **k):
        return Failure("detector crashed")

class _OkSeg:
    """A segmenter that writes one play row (never reached when validation fails first)."""

    def __init__(self, db_, cfg_):
        self.db = db_

    def process_video(self, vid, path, *a, **k):
        sid = self.db.add_play_segment(vid, 0.0, 5.0)
        return [
            {
                "id": sid,
                "start_time": 0.0,
                "end_time": 5.0,
                "duration": 5.0,
                "metadata": {},
            }
        ]

def _setup(tmp_path, monkeypatch, *, results, seg, cfg=None, content=b"identical-bytes"):
    """Wire the module globals + a real (temp-file) DB + stubbed validator/segmenter, and return
    ``(db, spy, video_path)``. The video file's BYTES fix its video_id, so writing the same
    bytes
    twice (a 'reupload') hits the same cache key."""
    db = Database(str(tmp_path / "t.db"))

```

```

monkeypatch.setattr(m, "db_instance", db)
monkeypatch.setattr(m, "global_config", cfg or AppConfig())
spy = _ValSpy(results)
monkeypatch.setattr(m.registry, "run_all_with_context", spy)
monkeypatch.setattr(m.segmenter_registry, "get", lambda name: seg)
video = tmp_path / "v.mp4"
video.write_bytes(content)
return db, spy, str(video)

```

```

def _process(video, **kw):
    return m._execute_processing(m.ProcessRequest(video_path=video, segmenter="motion", **kw))

```

--- the CUJ: reupload after a segmentation failure must NOT re-validate -----

```

def test_reupload_after_segmentation_failure_skips_validation(tmp_path, monkeypatch, caplog):
    """Validation PASSES but detection FAILS ? no segments, so the whole-pipeline cache misses
    on the
    reupload. The validation cache must still short-circuit: validators run exactly ONCE across
    the two
    uploads (the second reuses the cached PASS and goes straight to segmentation)."""
    db, spy, video = _setup(tmp_path, monkeypatch, results=_pass(), seg=_FailSeg)

    out1 = _process(video)
    assert out1["status"] == "failed" # detection failed
    assert spy.calls == 1 # validated once
    vid = out1["video_id"]
    cached = db.get_validation_cache(vid)
    assert cached is not None and cached["passed"] is True # verdict persisted
    assert db.query_play_segments(vid, {}) == [] # no segments ? whole-pipeline cache WILL miss

    with caplog.at_level(logging.INFO, logger=_ORCH_LOGGER):
        out2 = _process(video)
        assert out2["status"] == "failed"
        assert out2["video_id"] == vid # same content id
        assert spy.calls == 1 # validators did NOT re-run ? the cached verdict was reused
        assert any(
            "Reusing cached validation verdict" in r.getMessage() for r in caplog.records
        )

```

```

def test_cached_row_records_the_current_fingerprint(tmp_path, monkeypatch):
    """The persisted verdict is tagged with the config fingerprint it was computed under, so a
    later
    lookup can tell 'same config' from 'config changed'."""
    db, spy, video = _setup(tmp_path, monkeypatch, results=_pass(), seg=_FailSeg)
    out = _process(video)
    cached = db.get_validation_cache(out["video_id"])
    assert cached["fingerprint"] == registry.config_fingerprint(m.global_config)

```

--- guardrails: config change + force must both re-validate -----

```

def test_config_change_forces_revalidation(tmp_path, monkeypatch):
    """Changing a validation threshold flips the fingerprint, so the stale verdict is NOT reused
    ? the
    validators run again under the new config (never serve a PASS decided under different
    thresholds)."""
    db, spy, video = _setup(tmp_path, monkeypatch, results=_pass(), seg=_FailSeg)
    _process(video)

```

```

assert spy.calls == 1
m.global_config.validation.min_blur_threshold = 999.0 # different validator config
_process(video)
assert spy.calls == 2 # re-validated under the changed config

def test_sport_change_forces_revalidation(tmp_path, monkeypatch):
    """`sport` is not under `validation` but the relevance validator keys its court-color
profile
off it, so it is part of the fingerprint ? switching sport re-validates."""
    db, spy, video = _setup(tmp_path, monkeypatch, results=_pass(), seg=_FailSeg)
    _process(video)
    assert spy.calls == 1
    m.global_config.sport = "tennis"
    _process(video)
    assert spy.calls == 2

def test_force_bypasses_validation_cache(tmp_path, monkeypatch):
    """`force=true` must re-validate even with an identical cached verdict (mirrors how it
bypasses
the whole-pipeline segment cache)."""
    db, spy, video = _setup(tmp_path, monkeypatch, results=_pass(), seg=_FailSeg)
    _process(video)
    assert spy.calls == 1
    _process(video, force=True)
    assert spy.calls == 2 # forced re-validation despite the cache

def test_force_refreshes_the_cached_verdict(tmp_path, monkeypatch):
    """A forced re-validation re-writes the cache under the current fingerprint, so a subsequent
(non-forced) reupload reuses the FRESH verdict rather than re-decoding again."""
    db, spy, video = _setup(tmp_path, monkeypatch, results=_pass(), seg=_FailSeg)
    _process(video)
    _process(video, force=True)
    assert spy.calls == 2
    _process(video) # non-forced: should hit the refreshed cache
    assert spy.calls == 2

```

--- a cached FAILURE is reused too (no re-decode of a known-bad clip) -----

```

def test_cached_validation_failure_is_reused(tmp_path, monkeypatch):
    """A clip that FAILED validation caches its failure verdict; a reupload replays the SAME
failure
without re-decoding (the whole-pipeline cache never fires ? the video never became
'processed')."""
    db, spy, video = _setup(tmp_path, monkeypatch, results=_fail(), seg=_OkSeg)
    out1 = _process(video)
    assert out1["status"] == "failed" and "too blurry" in out1["message"]
    assert spy.calls == 1
    out2 = _process(video)
    assert out2["status"] == "failed"
    assert out2["message"] == out1["message"] # identical failure replayed
    assert spy.calls == 1 # not re-decoded

```

--- content-hash keying: a different video never reuses another's verdict -----

```

def test_different_content_does_not_reuse_verdict(tmp_path, monkeypatch):
    """Even with the same filename, DIFFERENT bytes ? a different video_id ? its own validation.

```

```

A
cached verdict never leaks across distinct content."""
db, spy, video = _setup(tmp_path, monkeypatch, results=_pass(), seg=_FailSeg)
_process(video)
assert spy.calls == 1
other = tmp_path / "other.mp4"
other.write_bytes(b"totally-different-bytes")
_process(str(other))
assert spy.calls == 2 # a different content hash never reuses another video's verdict

```

--- the fingerprint itself: stable, and sensitive to every verdict-affecting knob -----

```

def test_config_fingerprint_is_stable_and_config_sensitive():
    base = AppConfig()
    fp = registry.config_fingerprint(base)
    assert fp == registry.config_fingerprint(AppConfig()) # deterministic across instances
    assert len(fp) == 64 # sha256 hex digest

    c_blur = AppConfig()
    c_blur.validation.min_blur_threshold += 1
    c_fps = AppConfig()
    c_fps.validation.min_fps += 1
    c_res = AppConfig()
    c_res.validation.min_resolution_height += 1
    c_scene = AppConfig()
    c_scene.validation.max_scene_cuts_per_minute += 1
    c_pts = AppConfig()
    c_pts.validation.pts_max_gap_seconds += 1
    c_rel = AppConfig()
    c_rel.validation.relevance.court_pixels_min_ratio = 0.9
    c_sport = AppConfig()
    c_sport.sport = "tennis"

    fps = {
        registry.config_fingerprint(c)
        for c in (base, c_blur, c_fps, c_res, c_scene, c_pts, c_rel, c_sport)
    }
    # Every distinct config (incl. sport + each validation threshold) yields a distinct
    fingerprint.
    assert len(fps) == 8

def test_config_fingerprint_accepts_dict_config():
    """Robust to a dict config (not only a typed AppConfig) so the fingerprint never crashes the
    pipeline on a legacy caller."""
    typed = AppConfig()
    as_dict = typed.model_dump(mode="json")
    # Same content via dict vs typed still produces a valid 64-char digest (value need not match
    the
    # typed one ? dict lacks the model coercion ? but it must be stable + non-crashing).
    d_fp = registry.config_fingerprint(as_dict)
    assert isinstance(d_fp, str) and len(d_fp) == 64
    assert d_fp == registry.config_fingerprint(dict(as_dict))

```

89. user (2026-07-07T17:12:08.603Z)

File created successfully at: C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_validation_cache.py (file state is current in your context ? no need to Read it back)

90. assistant (2026-07-07T17:12:14.120Z)

Now let me run the new test file to validate the behavior before touching the schema-version tests:

91. user (2026-07-07T17:12:21.553Z)

..... [100%]
10 passed in 4.36s

92. assistant (2026-07-07T17:12:35.195Z)

All 10 new tests pass. Now let me bump the four test files that hardcode the head schema version (9?10). Starting with `test\_database.py`:

93. user (2026-07-07T17:12:37.685Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

94. user (2026-07-07T17:12:41.532Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

95. user (2026-07-07T17:12:47.590Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

96. user (2026-07-07T17:12:55.213Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

97. assistant (2026-07-07T17:12:57.602Z)

Now the other three files. Let me view the exact lines to make precise edits:

98. user (2026-07-07T17:13:00.921Z)

```
=== test_async_jobs.py 104-108, 144-150 ===
    # v9 = async compile (kind/params); v8 = compute-picker; v7 = UI locale; v6 = cost
    ledger;
    # jobs at v2/v3, user_models v5.
    assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
    cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
    assert {
        db = Database(path) # ladder should take it 1 ? 9
        with db._get_connection() as c:
            assert c.execute("PRAGMA user_version").fetchone()[0] == 9
            assert c.execute(
                "SELECT name FROM sqlite_master WHERE type='table' AND name='jobs'"
            ).fetchone()
            # Restore v5 personalization coverage (user_models table + user_id on
            candidate_segments).
=== test_job_worker_loop.py 339-343, 357-361 ===
    with db._get_connection() as conn:
        # Head of the migration ladder (bump when database.py adds a new `if version < N`).
        assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
```

```

db = Database(path)
with db._get_connection() as c:
    assert c.execute("PRAGMA user_version").fetchone()[0] == 9
=== test_locale_analytics.py 32-36, 49-53 ===
path = str(tmp_path / "fresh.db")
Database(path)
assert _user_version(path) == 9 # head: v7 added locale, v8 compute, v9 compile job
assert "locale" in _cols(path, "jobs")

Database(path) # re-open ? the ladder runs _migrate_to_v7 (+ v8, v9) up to the head
assert _user_version(path) == 9
assert "locale" in _cols(path, "jobs")
# The prior row survived the ALTER and back-filled to NULL (not dropped, not defaulted to a
value).

```

99. user (2026-07-07T17:13:10.123Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

100. user (2026-07-07T17:13:13.253Z)

```

102     db = Database(str(tmp_path / "fresh.db"))
103     with db._get_connection() as conn:
104         # v9 = async compile (kind/params); v8 = compute-picker; v7 = UI locale; v6 =
cost ledger;
105         # jobs at v2/v3, user_models v5.
106         assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
107         cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
108         assert {
109             "id",
110             "video_path",
111             "video_id",
112             "status",
113             "progress",
114             "error",
115             "result",
116         } <= cols
117         assert {"policy", "next_poll_at"} <= cols # v3 self-scheduling columns
118         assert {
119             "locale",
120             "compute",
121             "kind",
122             "params",
123         } <= cols # v7 locale, v8 compute, v9 compile job
124
125
126     def test_migration_upgrades_v1_db_in_place(tmp_path):
127         path = str(tmp_path / "v1.db")
128         conn = sqlite3.connect(path)
129         conn.execute(
130             "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
131             "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)"
132         )
133         # A genuine v1 DB also has candidate_segments (created in the version<1 block);
include it
134         # so the v4 personalization ALTER (which adds user_id to that table) has a table to
alter.
135         conn.execute(
136             "CREATE TABLE candidate_segments (id INTEGER PRIMARY KEY AUTOINCREMENT, "
137             "video_id TEXT NOT NULL, detector TEXT NOT NULL, start_time REAL NOT NULL, "
138             "end_time REAL NOT NULL, duration REAL NOT NULL, stats TEXT)"
139         )
140         conn.execute("PRAGMA user_version = 1")
141         conn.commit()

```

```

142     conn.close()
143
144     db = Database(path) # ladder should take it 1 ? 9
145     with db._get_connection() as c:
146         assert c.execute("PRAGMA user_version").fetchone()[0] == 9
147         assert c.execute(
148             "SELECT name FROM sqlite_master WHERE type='table' AND name='jobs'"
149             ).fetchone()

```

101. user (2026-07-07T17:13:18.146Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_async_jobs.py has been updated successfully. (file state is current in your context ? no need to Read it back)

102. user (2026-07-07T17:13:22.186Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_async_jobs.py has been updated successfully. (file state is current in your context ? no need to Read it back)

103. assistant (2026-07-07T17:13:24.945Z)

Now `test\_job\_worker\_loop.py` and `test\_locale\_analytics.py`:

104. user (2026-07-07T17:13:26.070Z)

```

336
337     def test_fresh_db_is_v9(tmp_path):
338         db = Database(str(tmp_path / "v.db"))
339         with db._get_connection() as conn:
340             # Head of the migration ladder (bump when database.py adds a new `if version <
341             N`).
342             assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
343
344     def test_legacy_db_upgrades_to_head(tmp_path):
345         # A pre-migration legacy DB (only `videos`, user_version 0) must run the FULL
346         ordered
347         # migration ladder up to the current head (v9) without error ? this exercises the
348         v0?v9
349         # path including the v1 `candidate_segments` CREATE that the later v4 ALTER depends
350         on.
351         path = str(tmp_path / "legacy.db")
352         conn = sqlite3.connect(path)
353         conn.execute(
354             "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
355             "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)"
356         )
357         conn.execute("PRAGMA user_version = 0")
358         conn.commit()
359         conn.close()
360         db = Database(path)
361         with db._get_connection() as c:
362             assert c.execute("PRAGMA user_version").fetchone()[0] == 9

```

105. user (2026-07-07T17:13:33.833Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_job_worker_loop.py has been updated successfully. (file state is current in your context ? no need to Read it back)

106. user (2026-07-07T17:13:37.836Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_job_worker_loop.py has been updated successfully. (file state is current in your context ? no need to Read it back)

107. user (2026-07-07T17:13:40.797Z)

```
30
31     def test_fresh_db_v7_has_jobs_locale_column(tmp_path):
32         path = str(tmp_path / "fresh.db")
33         Database(path)
34         assert _user_version(path) == 9 # head: v7 added locale, v8 compute, v9 compile job
35         assert "locale" in _cols(path, "jobs")
36
37
38     def test_v6_db_upgrades_to_v7_and_backfills_locale_null(tmp_path):
39         path = str(tmp_path / "v6.db")
40         db = Database(path) # born at v7
41         db.create_job("j-old", "/v.mp4") # a prior row, no locale
42         # Simulate a real v6 jobs table (no locale column, version pinned at 6).
43         conn = sqlite3.connect(path)
44         conn.execute("ALTER TABLE jobs DROP COLUMN locale")
45         conn.execute("PRAGMA user_version = 6")
46         conn.commit()
47         conn.close()
48         assert "locale" not in _cols(path, "jobs")
49
50         Database(path) # re-open ? the ladder runs _migrate_to_v7 (+ v8, v9) up to the head
51         assert _user_version(path) == 9
52         assert "locale" in _cols(path, "jobs")
53         # The prior row survived the ALTER and back-filled to NULL (not dropped, not
54         # defaulted to a value).
```

108. user (2026-07-07T17:13:44.975Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_locale_analytics.py has been updated successfully. (file state is current in your context ? no need to Read it back)

109. user (2026-07-07T17:13:48.804Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_locale_analytics.py has been updated successfully. (file state is current in your context ? no need to Read it back)

110. assistant (2026-07-07T17:14:04.281Z)

Now let me add a direct DB-layer round-trip test for the cache in `test\_database.py`:

111. user (2026-07-07T17:14:11.467Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

112. assistant (2026-07-07T17:14:14.356Z)

Now let me run the affected test files together to confirm the schema-version bumps and DB test pass:

113. user (2026-07-07T17:15:04.958Z)

```
..... [ 55%]
..... [100%]
===== warnings summary =====
```

```

tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
129 passed, 8 warnings in 46.79s

```

114. assistant (2026-07-07T17:15:11.546Z)

All 129 pass (the warnings are pre-existing Google-auth noise). Task #4 done. Now the pre-merge gate ? let me run ruff and mypy first (fast), matching CI (`ruff check .`, `mypy backend training`):

115. user (2026-07-07T17:15:12.057Z)

Updated task #4 status

116. user (2026-07-07T17:15:12.579Z)

Updated task #5 status

117. user (2026-07-07T17:15:15.663Z)

```

=== ruff version ===
ruff 0.15.16
=== ruff check . ===
176 |         if not os.path.exists(path):
177 |             rows.append({"label": label, "ok": False, "error": "not found: " + path});
continue
    |
    |
178 |         try:
179 |             rows.append(score_one(label, path, gts, mods, args.fps, args.frame_width))
    |
    |
E702 Multiple statements on one line (semicolon)
--> score_local.py:181:102
    |
179 |             rows.append(score_one(label, path, gts, mods, args.fps, args.frame_width))
180 |         except Exception as e:
181 |             rows.append({"label": label, "ok": False, "error": "{}:
{}".format(type(e).__name__, e)}); traceback.print_exc()
    |
    |
    ^
182 |     print_table(rows, len(gts), args.name, mods["tm"])
183 |     return 0
    |

```

Found 18 errors.
[*] 4 fixable with the `--fix` option.

118. user (2026-07-07T17:15:24.845Z)

```

=== mypy version ===

```

```
mypy 2.1.0 (compiled: yes)
=== mypy backend training ===
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:212: error: Item "None" of "Any | None" has no attribute
"model_dump" [union-attr]
backend\orchestration.py:834: error: Item "None" of "Any | None" has no attribute "get" [union-
attr]
backend\orchestration.py:839: error: Item "None" of "Any | None" has no attribute "get" [union-
attr]
Found 3 errors in 2 files (checked 149 source files)
```

119. assistant (2026-07-07T17:16:07.115Z)

`ruff` flags only the two untracked dev tools (`score\_local.py`/`score\_golden\_gate.py`), which aren't part of my change or the committed tree. But mypy found 3 real issues in my code. Let me confirm ruff is clean on my files, then fix the mypy union-attr errors.

120. user (2026-07-07T17:16:11.125Z)

```
=== ruff on MY changed files only ===
All checks passed!
=== confirm the 18 ruff errors are ALL in the two untracked dev tools ===
```

121. user (2026-07-07T17:16:19.373Z)

```
=== which files have the 18 ruff errors ===
--> score_local.py
=== are those files tracked in git? ===
?? score_golden_gate.py
?? score_local.py
```

122. assistant (2026-07-07T17:16:25.102Z)

All 18 ruff errors are in the two **untracked** dev tools (not my change, not in the committed tree CI checks). My files are ruff-clean. Now let me fix the 3 real mypy `union-attr` errors. First `base.py`:

123. user (2026-07-07T17:16:29.983Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\validators\base.py has been updated successfully. (file state is current in your context ? no need to Read it back)

124. assistant (2026-07-07T17:16:32.824Z)

Now `orchestration.py` ? restructure so mypy narrows `cached\_validation` via `is not None`:

125. user (2026-07-07T17:16:39.886Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

126. user (2026-07-07T17:17:00.054Z)

```
=== mypy backend training ===
```

```

backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\_init_.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\providers\_init_.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 149 source files

```

```

=== ruff on changed files ===
All checks passed!

```

127. assistant (2026-07-07T17:17:05.896Z)

mypy clean (149 files, the `note:` lines are pre-existing informational). Let me re-verify the cache tests after the refactor and inspect how the full suite runs coverage:

128. user (2026-07-07T17:17:23.431Z)

```

..... [100%]
10 passed in 8.57s
=== run_all_tests.py: how it invokes pytest/coverage ===
4:A single, discoverable entry point so you (or a reviewer / CI) can verify the
8:   python run_all_tests.py -v           # verbose (passthrough to pytest)
10:  python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
12:Anything after the script name is passed straight through to pytest.
18: package isn't installed (they use `pytest.importorskip`). To exercise them too:
20:- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
26:import subprocess
42:   cmd = [sys.executable, "-m", "pytest", *args]
51:   print(" Full test suite: " + " ".join(cmd))
55:   rc = subprocess.run(cmd).returncode
60:       + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit {rc}")
=== pytest.ini / setup.cfg / pyproject addopts ===

```

129. assistant (2026-07-07T17:17:30.232Z)

Let me find the exact CI coverage invocation to reproduce it faithfully:

130. user (2026-07-07T17:17:39.843Z)

```

.github/workflows/ci.yml:99:      run: python -m pip install pytest fastapi uvicorn pydantic
python-dotenv google-genai
.github/workflows/ci.yml:101:     run: python -m pytest tests/test_import_smoke.py -v
.github/workflows/ci.yml:132:     # actually run in CI instead of skipping via
pytest.importorskip.
.github/workflows/ci.yml:136:     # (pytest, pytest-cov, httpx for the fastapi TestClient,
ruff, mypy).
.github/workflows/ci.yml:139:     # pytest.importorskip ? run_all_tests.py prints which. The
coverage floor
.github/workflows/ci.yml:146:     run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
.github/workflows/ci.yml:168:     run: python -m pip install pytest fastapi uvicorn pydantic
python-dotenv google-genai numpy
.github/workflows/ci.yml:175:     python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
.github/workflows/nightly.yml:105:    run: python -m pip install pytest fastapi uvicorn
pydantic python-dotenv google-genai
.github/workflows/nightly.yml:107:    run: python -m pytest tests/test_import_smoke.py -v
.github/workflows/nightly.yml:136:    # run instead of skipping via pytest.importorskip.
.github/workflows/nightly.yml:141:    run: python run_all_tests.py --cov=backend
--cov=training --cov-report=term --cov-fail-under=70
.github/workflows/nightly.yml:159:    run: python -m pip install pytest fastapi uvicorn

```

```

pydantic python-dotenv google-genai numpy
.github/workflows/nightly.yml:163:         python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
=== fallback: search repo for cov-fail-under ===
./.github/workflows/ci.yml:146:         run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
./.github/workflows/nightly.yml:141:         run: python run_all_tests.py --cov=backend
--cov=training --cov-report=term --cov-fail-under=70
./scratch/quality_report.py:898:         "(&minus;&minus;cov-fail-under=70, calibrated
against the measured 72%). Combined with default-"

```

131. assistant (2026-07-07T17:17:47.220Z)

Exact CI command confirmed. Let me run the full suite with coverage in the background, and review my complete diff while it runs.

132. user (2026-07-07T17:17:50.390Z)

Command running in background with ID: bng92vr00. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\b3f086c7-9143-451e-8d2c-de20114d21c6\tasks\bng92vr00.output. You will be notified when it completes. To check interim output, use Read on that file path.

133. assistant (2026-07-07T17:17:52.854Z)

While the suite runs, let me review my full diff for correctness:

134. user (2026-07-07T17:17:55.708Z)

```

backend/infrastructure/database.py      | 34 ++++++-----
backend/infrastructure/database_media.py | 62 ++++++-----
backend/orchestration.py                | 59 ++++++-----
backend/pipeline/validators/base.py     | 51 ++++++-----
tests/test_async_jobs.py                | 10 +++--
tests/test_database.py                  | 59 ++++++-----
tests/test_job_worker_loop.py           | 8 +++--
tests/test_locale_analytics.py          | 8 +++--
8 files changed, 261 insertions(+), 30 deletions(-)
=== full diff of production code (not tests) ===
diff --git a/backend/infrastructure/database.py b/backend/infrastructure/database.py
index df6ea7f..78fa274 100644
--- a/backend/infrastructure/database.py
+++ b/backend/infrastructure/database.py
@@ -16,7 +16,7 @@ logger = logging.getLogger(__name__)
# auto-updating app: refuse to open a DB from a NEWER app (downgrade guard) and back the DB up
before
# an upgrade migration runs. `tests/test_database.py::test_schema_version_matches_ladder` pins
it to
# the ladder so adding a `_migrate_to_vN` without bumping this fails CI.
-SCHEMA_VERSION = 9
+SCHEMA_VERSION = 10

class SchemaTooNewError(RuntimeError):
@@ -146,7 +146,10 @@ class Database(DatabaseMediaMixin, DatabaseJobsMixin,
DatabaseUserModelsMixin):

    if version < 9:
        self._migrate_to_v9(cursor)
-
# future: if version < 10: cursor.execute("ALTER TABLE ..."); PRAGMA user_version =
10
+
+    if version < 10:
+        self._migrate_to_v10(cursor)

```

```

+         # future: if version < 11: cursor.execute("ALTER TABLE ..."); PRAGMA user_version =
11

        conn.commit()

@@ -465,6 +468,33 @@ class Database(DatabaseMediaMixin, DatabaseJobsMixin,
DatabaseUserModelsMixin):
    self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN params TEXT")
    cursor.execute("PRAGMA user_version = 9")

+    def _migrate_to_v10(self, cursor):
+        # v10: validation-result CACHE keyed by the content id. A video's id IS the MD5 of its
bytes
+        # (backend/utils/hashing.py::compute_video_id), so a REUPLOAD of identical content keys
the
+        # same row ? and when that content was ALREADY validated under the SAME validator
config, the
+        # pipeline reuses the stored verdict instead of re-decoding the clip through the whole
validator
+        # suite (minutes on a heavy 4K clip on the CPU-only control-plane; the 2026-07-07 CUJ
re-paid it
+        # on every retry after a segmentation failure). Columns:
+        #   fingerprint ? ValidationRegistry.config_fingerprint (the validation.* config +
sport +
+        #   validator set + a schema version); a lookup reuses the verdict ONLY
on a
+        #   fingerprint match, so a config/sport/validator change misses ? re-
validates.
+        #   passed      ? the overall verdict (1/0), so a cached FAIL short-circuits without a
decode.
+        #   results     ? the JSON ValidationResult list, replayed into the response + report.
+        # Standalone table (NO foreign key): the row is written right after validation runs ?
which can
+        # be BEFORE the videos UPSERT exists ? and is dropped explicitly by clear_video_data.
UPSERT on
+        # the video_id PK keeps ONE (latest) verdict per video. Additive-only ? nothing pre-
existing
+        # changes; an install that never reuploads behaves byte-identically to v9.
+        cursor.execute("""
+            CREATE TABLE IF NOT EXISTS validation_cache (
+                video_id TEXT PRIMARY KEY,
+                fingerprint TEXT NOT NULL,
+                passed INTEGER NOT NULL,
+                results TEXT NOT NULL,
+                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
+            )
+        """)
+        cursor.execute("PRAGMA user_version = 10")
+

        # Domain CRUD methods live in mixins so Database remains the public facade while
        # schema/migration/connection logic stays centralized in this module.
diff --git a/backend/infrastructure/database_media.py b/backend/infrastructure/database_media.py
index e58865c..66b1d58 100644
--- a/backend/infrastructure/database_media.py
+++ b/backend/infrastructure/database_media.py
@@ -39,6 +39,61 @@ class DatabaseMediaMixin:
    "Failed to add video",
    )

+    def set_validation_cache(
+        self,
+        video_id: str,
+        fingerprint: str,
+        passed: bool,

```

```

+         validation_results: List[Dict[str, Any]],
+     ) -> bool:
+         """Cache a video's validation verdict keyed by its content id (the MD5 ``video_id``),
so an
+         identical-bytes REUPLOAD can skip re-running the (minutes-long on heavy 4K) validator
suite.
+
+         ``fingerprint`` is ``ValidationRegistry.config_fingerprint(config)`` ? a hash of the
+         validation config + sport + validator set + a schema version. A reader
+         (``get_validation_cache``)
+         only reuses the verdict while that fingerprint still matches the CURRENT config, so a
config
+         change re-validates instead of serving a stale verdict. ``passed`` is the overall
verdict and
+         ``validation_results`` the full ValidationResult list (dumped) so a cached FAIL can
replay the
+         same message and a cached PASS carries the same results into the report/response.
UPSERT on the
+         ``video_id`` PK keeps ONE (latest) verdict per video: a re-validation under a NEW
config
+         overwrites the stale row rather than accumulating history."""
+         return self._execute_write(
+             "INSERT INTO validation_cache (video_id, fingerprint, passed, results) "
+             "VALUES (?, ?, ?, ?) "
+             "ON CONFLICT(video_id) DO UPDATE SET "
+             "fingerprint=excluded.fingerprint, passed=excluded.passed, "
+             "results=excluded.results, created_at=CURRENT_TIMESTAMP",
+             (video_id, fingerprint, 1 if passed else 0, json.dumps(validation_results)),
+             "Failed to write validation cache",
+         )
+
+     def get_validation_cache(self, video_id: str) -> Optional[Dict[str, Any]]:
+         """The cached validation verdict for ``video_id`` ? ``{fingerprint, passed, results}``
? or
+         ``None`` when nothing is cached.
+
+         The caller compares ``fingerprint`` against the CURRENT
+         ``ValidationRegistry.config_fingerprint(config)`` and reuses the verdict ONLY on a
match (a
+         config / sport / validator-set change flips the fingerprint ? re-validate). ``results``
is the
+         stored ValidationResult list (parsed from JSON), replayed so a cached PASS proceeds to
+         segmentation and a cached FAIL returns the same failure ? in both cases without re-
decoding the
+         video."""
+         with self._connect() as conn:
+             row = (
+                 conn.cursor()
+                 .execute(
+                     "SELECT fingerprint, passed, results FROM validation_cache "
+                     "WHERE video_id = ?",
+                     (video_id,),
+                 )
+                 .fetchone()
+             )
+             if not row:
+                 return None
+             d = dict(row)
+             d["passed"] = bool(d["passed"])
+             d["results"] = json.loads(d["results"] or "[]")
+             return d
+
+     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
+         """Return (max play_segment id, max candidate_segment id) for a video (0 if none).

```

```
@@ -304,6 +359,11 @@ class DatabaseMediaMixin:
```

```
    def clear_video_data(self, video_id: str):
        with self._connect() as conn:
            conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
            cur = conn.cursor()
            cur.execute("DELETE FROM videos WHERE id = ?", (video_id,))
            # Drop the cached validation verdict too ? the validation_cache row has no FK to
videos
            # (it can be written before the videos row exists), so an explicit clear must
remove it
            # here or the next upload would reuse a verdict for data the caller just wiped.
            cur.execute("DELETE FROM validation_cache WHERE video_id = ?", (video_id,))
            conn.commit()
```

```
diff --git a/backend/orchestration.py b/backend/orchestration.py
```

```
index 9b54905..7f2028a 100644
```

```
--- a/backend/orchestration.py
```

```
+++ b/backend/orchestration.py
```

```
@@ -33,7 +33,7 @@ from backend import storage
```

```
    from backend.config import AppConfig, StitchingConfig
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.watermark_i18n import localize_watermark
    -from backend.pipeline.validators.base import registry
    +from backend.pipeline.validators.base import ValidationResult, registry
    from backend.results import Failure
    from backend.utils.paths import output_base, safe_output_filename
```

```
@@ -797,11 +797,62 @@ def _execute_processing(
```

```
    )
    return response

    # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
    + # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report) ?
    + #    UNLESS this exact content already has a cached verdict under the current validator
config.
        notify("validating")
    - logger.info(f"Running validations for {req.video_path}...")
    - val_results, val_context = registry.run_all_with_context(local_video, config)
    + # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an identical-
content
    + # reupload keys the same row. When that content was ALREADY validated under the SAME
validator
    + # configuration, reuse the stored verdict instead of re-decoding the clip through the
whole
    + # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane, which the
    + # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation passed
but
    + # detection failed ? no cached segments, so the whole-pipeline cache above misses and
we land
    + # here). The fingerprint folds in the validation.* config + sport + validator set + a
schema
    + # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
    + # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors how it
bypasses
    + # the whole-pipeline cache). Composes with #513 (the worker skips its own re-validation
on a
    + # cloud dispatch), so a post-failure reupload reaches segmentation in ~0 validation
time.
    + val_fingerprint = registry.config_fingerprint(config)
    + cached_validation = None
    + if not req.force:
```



```

+         try:
+             cached_validation = db.get_validation_cache(video_id)
+         except Exception: # noqa: BLE001 ? a cache-read fault must degrade to a real
validation
+             logger.warning(
+                 "[API] validation-cache read failed for video_id=%s ? re-validating.",
+                 video_id,
+                 exc_info=True,
+             )
+         if (
+             cached_validation is not None
+             and cached_validation.get("fingerprint") == val_fingerprint
+         ):
+             # Replay the stored ValidationResult list; val_context stays {} (no fresh
ffprobe_meta ?
+             # the report's media block degrades gracefully to size-only, which is the
deliberate cost
+             # of skipping the decode). Segmentation reads neither, so it is unaffected.
+             reused_validation = True
+             val_results = [
+                 ValidationResult(**r) for r in cached_validation.get("results", [])
+             ]
+             logger.info(
+                 "[API] Reusing cached validation verdict (passed=%s) for video_id=%s; "
+                 "validator config unchanged ? skipping the validator suite. force=true re-
validates.",
+                 cached_validation.get("passed"),
+                 video_id,
+             )
+         else:
+             reused_validation = False
+             logger.info(f"Running validations for {req.video_path}...")
+             val_results, val_context = registry.run_all_with_context(local_video, config)
+             failures = [r for r in val_results if not r.passed]
+             if not reused_validation:
+                 # Persist the verdict keyed by (content, config) so the NEXT identical reupload
skips the
+                 # decode. Best-effort (the writer swallows + logs its own faults); a miss just re-
validates.
+                 db.set_validation_cache(
+                     video_id,
+                     val_fingerprint,
+                     not failures,
+                     [r.model_dump() for r in val_results],
+                 )

+             if failures:
+                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
diff --git a/backend/pipeline/validators/base.py b/backend/pipeline/validators/base.py
index 5530026..5916ef6 100644
--- a/backend/pipeline/validators/base.py
+++ b/backend/pipeline/validators/base.py
@@ -1,3 +1,5 @@
+import hashlib
+import json
+import logging
+from abc import ABC, abstractmethod
+from typing import Any, List
@@ -13,6 +15,13 @@ from backend.pipeline.validators.frame_sampler import (

+         logger = logging.getLogger(__name__)

+         ## Bumped when validator LOGIC (not config) changes the verdict for identical bytes+config ?
e.g. a
+         ## validator is rewritten, or the sampling arithmetic changes. It rides inside

```

```

config_fingerprint, so
+ # a bump invalidates every cached validation verdict (see backend/orchestration.py's validation
cache
+ # + backend/infrastructure/database.py v10). The validator SET and each validator's config-
derived
+ # thresholds are already folded into the fingerprint; this covers changes the config can't
express.
+_VALIDATION_FINGERPRINT_SCHEMA = 1
+

class ValidationResult(BaseModel):
    validator_name: str
@@ -176,6 +185,48 @@ class ValidationRegistry:
    results, _ = self.run_all_with_context(video_path, config)
    return results

+ def config_fingerprint(self, config: Any) -> str:
+     """Stable hash of EVERYTHING that determines a validation verdict for fixed video
bytes.
+
+     Powers the video_id-keyed validation-result cache (backend/orchestration.py): a
reupload of
+     identical content reuses its stored verdict ONLY while this fingerprint still matches,
so any
+     change that could flip a decision correctly forces a re-validation (never serve a stale
PASS/FAIL). What goes in ? and why each matters:
+
+     * the whole ``validation`` config block ? every threshold the validators read
      (min_blur_threshold / min_fps / min_resolution / max_scene_cuts_per_minute / pts gap
/
+     relevance court-color bounds). Hashing the full model (not a hand-picked subset)
means a
+     NEW validation knob is covered automatically.
+     * ``sport`` ? the relevance validator keys its court-color profile off ``config.sport``
(a
+     top-level field, NOT under ``validation``), so switching sports changes the verdict.
+     * the sorted validator NAMES ? adding/removing a validator changes which checks run.
+     * a schema version (:data:_VALIDATION_FINGERPRINT_SCHEMA) ? a manual escape hatch for
validator LOGIC changes the config can't express.
+
+     Accepts a typed ``AppConfig`` (attribute access) or a plain dict; ``default=str`` keeps
the
+     dump total on any exotic value. Deterministic: ``sort_keys`` fixes ordering."""
+     validation: Any = getattr(config, "validation", None)
+     if validation is None and isinstance(config, dict):
+         validation = config.get("validation")
+     validation_repr = (
+         validation.model_dump(mode="json")
+         if hasattr(validation, "model_dump")
+         else validation
+     )
+     sport: Any = getattr(config, "sport", None)
+     if sport is None and isinstance(config, dict):
+         sport = config.get("sport")
+     payload = {
+         "schema": _VALIDATION_FINGERPRINT_SCHEMA,
+         "validators": sorted(v.name for v in self._validators),
+         "sport": sport,
+         "validation": validation_repr,
+     }
+     blob = json.dumps(
+         payload, sort_keys=True, default=str, separators=(",", ":")
+     )
+     return hashlib.sha256(blob.encode("utf-8")).hexdigest()
+

```

```
# Shared global registry
registry = ValidationRegistry()
```

135. assistant (2026-07-07T17:18:26.269Z)

The diff is clean and self-consistent. Let me check the suite's progress:

136. user (2026-07-07T17:18:30.395Z)

(Bash completed with no output)

137. assistant (2026-07-07T17:18:37.092Z)

The suite is running (pytest buffers output until completion). I'll wait for it to finish rather than poll. Let me use the time to double-check one interaction I want to be certain about ? that a cache ****hit on the cloud-dispatch path**** still produces `model_dump()`-able results downstream in `_maybe_dispatch_remote`. Let me trace it with a targeted test add to the cloud test file's mental model ? actually, let me just verify by writing a quick focused check.

138. user (2026-07-07T17:18:48.496Z)

```
round-trip model_dump ok: True
passed attr intact: True
```

139. assistant (2026-07-07T17:19:04.415Z)

Round-trip confirmed ? a reused verdict is indistinguishable from a fresh one downstream. Now I'll wait for the full suite to complete.

140. user (2026-07-07T17:19:07.646Z)

waiting for background suite (will be notified on completion)

141. assistant (2026-07-07T17:19:18.539Z)

Let me use the wait productively ? scan for any test that processes the same content twice and might now hit the cache unexpectedly (the main behavioral-change risk):

142. user (2026-07-07T17:19:22.272Z)

```
=== tests that call _execute_processing / process twice or assert re-validation ===
36 tests/test_api_endpoints.py
36 tests/test_api_cloud_dispatch.py
27 tests/test_async_jobs.py
3 tests/test_validation_cache.py
3 tests/test_job_worker_loop.py
3 tests/test_edge_auth.py
2 tests/test_upload_gcs_streaming.py
2 tests/test_main_module_budget.py
2 tests/test_base.py
1 tests/test_submit_md5_cache.py
1 tests/test_job_worker_module_seam.py
0 tests/test_worker.py
0 tests/test_validators_base.py
0 tests/test_server_entrypoint.py
0 tests/test_profile_parity.py
```

143. assistant (2026-07-07T17:19:30.821Z)

`test_validators_base.py` exists (covers the registry) and a few suites reprocess content ? the full suite is the authoritative check. Let me check if it's finished:

144. user (2026-07-07T17:19:32.895Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

145. assistant (2026-07-07T17:19:50.694Z)

Implementation is complete and the fast gates are green (ruff clean on my files, mypy clean across 149 files, all targeted test files pass). The full suite with coverage is running in the background ? I'll report when it finishes.

Here's where things stand:

****What I built**** ? a validation-result cache keyed by `video\_id` (the MD5 content hash), so an identical-bytes reupload skips the validator suite when the content was already validated under the same config:

- ****Fingerprint**** (`ValidationRegistry.config\_fingerprint`, [base.py](backend/pipeline/validators/base.py)): SHA-256 over the full `validation.\*` config + `sport` (the relevance validator keys its court-color profile off it ? a field *outside* `validation`, which I caught by reading the validators) + the sorted validator-name set + a bumpable schema version. Hashing the whole `validation` block (not a hand-picked subset) means a new knob is covered automatically.
- ****Storage**** ? a dedicated `validation\_cache` table (migration ****v10****, [database.py](backend/infrastructure/database.py)) keyed by `video\_id`, with `set/get\_validation\_cache` + `clear\_video\_data` cleanup ([database_media.py](backend/infrastructure/database_media.py)). I chose a standalone table over extending the video record so there's exactly one read site and one write site ? no threading the fingerprint through `add\_video`'s four callers or the cloud-staging path.
- ****Wiring**** ([orchestration.py](backend/orchestration.py:800)): between the whole-pipeline segment cache and the validator run ? cache hit + fingerprint match ? replay stored verdict (PASS ? segment; FAIL ? return stored failure, no re-decode); `force=true` bypasses and refreshes; a config/sport/validator change misses and re-validates.

****Acceptance covered**** by tests/test_validation_cache.py (10 tests): the CUJ (reupload after segmentation failure validates once across two uploads, asserted via the reuse log line), config-change and sport-change force re-validation, `force` bypasses, cached FAIL is replayed, different content never reuses, plus fingerprint stability/sensitivity ? and a DB round-trip test in `test\_database.py`.

****Note on ruff:**** `ruff check .` reports 18 errors, but all are in `score\_local.py`/`score\_golden\_gate.py` ? pre-existing ****untracked**** dev tools not part of this change or the committed tree CI checks. My files are ruff-clean.

146. user (2026-07-07T17:20:34.373Z)

pls open a PR and send for review

147. assistant (2026-07-07T17:20:41.671Z)

Before opening the PR, I need the full-suite gate to be green. Let me check the background run's result:

148. user (2026-07-07T17:20:45.951Z)

```
=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q --cov=backend --cov=training --cov-report=term --cov-fail-under=70
obsreport (optional reporting dep): available
=====
..... [ 3%]
.....
```

149. assistant (2026-07-07T17:21:02.942Z)

The suite is at ~3% (large suite, ~1800 tests) ? I'll wait for it to finish before committing,

since opening a PR on an unverified gate would violate my pre-merge discipline. Let me use the time to verify branch safety and draft the PR body.

150. user (2026-07-07T17:21:06.394Z)

Exit code 128

=== branch + HEAD (re-verify per branching-safety) ===

fatal: not a git repository (or any of the parent directories): .git

151. assistant (2026-07-07T17:21:20.784Z)

The Bash cwd reset to the stub dir. Let me use `git -C` with the absolute repo path to be robust:

152. user (2026-07-07T17:21:24.855Z)

=== branch + HEAD ===

feat/validation-result-cache

ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker (#515)

=== commits on branch vs origin/master (empty = nothing committed yet) ===

=== changed files to stage (excluding untracked dev tools) ===

M backend/infrastructure/database.py

M backend/infrastructure/database_media.py

M backend/orchestration.py

M backend/pipeline/validators/base.py

M tests/test_async_jobs.py

M tests/test_database.py

M tests/test_job_worker_loop.py

M tests/test_locale_analytics.py

?? docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md

?? tests/test_validation_cache.py

153. assistant (2026-07-07T17:21:33.028Z)

Branch is clean (based on `origin/master` @ ef41473, nothing committed yet). Note: there's an untracked `RUNLOG\_...md` from a **sibling session** ? I'll stage only my 9 explicit files, never that or the dev tools. Let me draft the PR body while the suite finishes:

154. assistant (2026-07-07T17:21:56.763Z)

What & why

Adds a **validation-result** cache keyed by `video\_id` so a reupload of identical content does **not** re-run the validator suite when that content was already validated under the same validator configuration.

Today the only skip path in `\_execute\_processing` is the whole-pipeline cache, which fires only when the video is `status=="processed"` **and** has cached play segments (i.e. a prior **successful** segmentation). So after a **segmentation failure** (validation passed, detection failed ? no cached segments), a reupload of the same `video\_id` re-runs the full validator suite from scratch ? minutes on a heavy clip on the CPU-only control-plane (~6.5 min on the 5.3K 10-bit HEVC "Video 1"). This bit the owner repeatedly during the 2026-07-07 CUJ retries.

`video\_id` is the MD5 of the file bytes (`compute\_video\_id`), so identical content ? identical key. This PR reuses the stored verdict when the request's `video\_id` already has one computed under the **same validator-config fingerprint**: PASS ? straight to segmentation; FAIL ? return the stored failure ? in both cases **without re-decoding the video**.

Composes with #513 (worker already skips its own re-validation on cloud dispatch) and the shared-decode speedup (#512): together, a post-failure reupload reaches segmentation in **~0 validation time**.

Design

- **Config fingerprint** ? `ValidationRegistry.config_fingerprint(config)`` (``backend/pipeline/validators/base.py``): SHA-256 over
 - the **whole** ``validation`` config block (min_blur_threshold / min_fps / min_resolution / max_scene_cuts_per_minute / pts gap / relevance court-color bounds) ? hashing the full model, not a hand-picked subset, so a **new validation knob** is covered automatically;
 - **sport** ? the relevance validator keys its court-color profile off ``config.sport``, a top-level field **outside** ``validation``, so switching sports must invalidate;
 - the **sorted validator names** ? adding/removing a validator changes which checks run;
 - a **schema version** (``_VALIDATION_FINGERPRINT_SCHEMA``) ? manual escape hatch for validator **logic** changes the config can't express.
- **Storage** ? a dedicated ``validation_cache`` table (**migration v10**), keyed by ``video_id``, columns ``fingerprint`` / ``passed`` / ``results`` / ``created_at``. Chosen over extending the video record so there's exactly **one read site and one write site** ? no threading the fingerprint through ``add_video``'s four callers or the cloud-staging path. ``set/get_validation_cache`` in ``database_media.py``; ``clear_video_data`` drops the cache row too.
- **Wiring** (``backend/orchestration.py``, between the whole-pipeline cache and the validator run): fingerprint match ? replay the stored verdict; a config/sport/validator change misses ? re-validate + rewrite; **force=true** bypasses the reuse and refreshes the verdict (mirrors how it bypasses the pipeline cache). A cache-read fault degrades to a normal validation.

Guardrails honored: never serve a stale PASS/FAIL (any threshold/sport/validator change flips the fingerprint ? re-validate); content-hash keying means a different video (even same filename) never reuses another's verdict. On a reuse the observability report's media block degrades gracefully to size-only (no fresh `ffprobe_meta`) ? the deliberate cost of skipping the decode; segmentation reads neither, so it is unaffected.

Tests

New ``tests/test_validation_cache.py`` (10 tests), driving ``_execute_processing`` with a validator spy that counts real runs:

- **the CUJ** ? reupload after a segmentation failure validates **once** across two uploads (asserted via run count **and** the reuse log line);
- config change and sport change each **force re-validation**; ``force=true`` bypasses; a forced run **refreshes** the cache;
- a cached **FAIL** is replayed without a re-decode; **different content** never reuses a verdict;
- fingerprint is stable + sensitive to every verdict-affecting knob, and accepts a dict config.

Plus a DB round-trip/UPSERT/clear test in ``test_database.py``, and the schema-version-ladder asserts bumped 9?10 (``test_database.py``, ``test_async_jobs.py``, ``test_job_worker_loop.py``, ``test_locale_analytics.py``).

Verification (local, matching CI)

- **Full suite** (``python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70``): ``_`` (filled in below)
- **ruff** ``ruff check .`` (0.15.16): clean on all changed files. (Pre-existing 18 errors are only in the untracked dev tools ``score_local.py`` / ``score_golden_gate.py``, not in this change or the committed tree.)
- **mypy** ``mypy backend training`` (2.1.0): **Success**, no issues found in 149 source files.

Related: task "Speed up video validation" (#512/#514), #513.

155. user (2026-07-07T17:21:56.831Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\b3f086c7-9143-451e-8d2c-de20114d21c6\scratchpad\pr_body.md (file state is current in your context ? no need to Read it back)

156. assistant (2026-07-07T17:22:01.231Z)

PR body drafted. Let me check the suite's progress:

157. user (2026-07-07T17:22:04.736Z)

```
..... [ 31%]
..... [ 34%]
..... [ 38%]
..... [ 42%]
..... [ 46%]
.....
```

158. assistant (2026-07-07T17:22:13.902Z)

Suite is at ~46%, all green so far, no failures. It'll take a few more minutes. I'll hold on committing until it completes ? I won't open the PR on an unverified gate. I'll pick this up automatically when the run finishes and then commit ? push ? open the PR with the verified numbers filled in.

159. user (2026-07-07T17:25:22.444Z)

```
<task-notification>
<task-id>bng92vr00</task-id>
<tool-use-id>toolu_01Nc3nDFTndbtt96j8H9aESi</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users\avidu-Projects-badminton-highlight-indexer\b3f086c7-9143-451e-8d2c-de20114d21c6\tasks\bng92vr00.output</output-file>
<status>completed</status>
<summary>Background command "Run full test suite with coverage (CI command)" completed (exit code 0)</summary>
</task-notification>
```

160. assistant (2026-07-07T17:25:27.211Z)

The suite finished with exit code 0. Let me confirm the pass count and coverage before committing:

161. user (2026-07-07T17:25:30.203Z)

```
backend\utils\paths.py           44      3    93%
backend\utils\redact.py          16      0   100%
backend\utils\run_telemetry.py   126     12    90%
backend\utils\single_instance.py 43     11    74%
backend\utils\video.py           34      3    91%
backend\utils\workspace.py       49      0   100%
backend\worker\__init__.py        0      0   100%
backend\worker\__main__.py       319     29    91%
training\__init__.py             0      0   100%
training\gen0\__init__.py        0      0   100%
training\gen0\harness.py        130      9    93%
-----
TOTAL                           12451   1767    86%
Required test coverage of 70% reached. Total coverage: 85.81%
1853 passed, 1 skipped, 25 warnings in 443.64s (0:07:23)
=====
RESULT: PASS - all tests green
=====
```

162. assistant (2026-07-07T17:25:37.464Z)

Gate is fully green: **1853 passed, 1 skipped**, coverage **85.81%** (above the 70% floor, no regression vs. the ~85% baseline). Let me finalize the PR body with these numbers, then commit and push.

163. user (2026-07-07T17:25:40.281Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\b3f086c7-9143-451e-8d2c-de20114d21c6\scratchpad\pr_body.md has been updated successfully. (file state is current in your context ? no need to Read it back)

164. user (2026-07-07T17:25:43.237Z)

Updated task #5 status

165. assistant (2026-07-07T17:25:46.029Z)

Now re-verify the branch is still clean (concurrent-session safety), then stage my 9 explicit files and commit:

166. user (2026-07-07T17:25:52.954Z)

```
=== re-verify branch + base right before commit ===
feat/validation-result-cache
(above must be empty)
=== fetch to check base didn't move under me ===
1      0
```

167. assistant (2026-07-07T17:26:03.576Z)

origin/master moved forward by 1 commit since I branched (a sibling session merged something). I have no commits yet, only working-tree changes. Let me see what landed and whether it touches my files:

168. user (2026-07-07T17:26:08.288Z)

```
=== new commit(s) on origin/master since my base ef41473 ===
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
=== files that new commit touched ===
docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md
docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md
=== do they overlap my changed files? ===
```

169. assistant (2026-07-07T17:26:59.111Z)

The new commit `115b83d` (#517) is ****docs-only**** and fully disjoint from my files ? zero conflict. My commit will sit on top of `ef41473`; since the docs commit is only on master's side, `origin/master..HEAD` will list only my commit (no sibling work rides in), and the PR diff against master will be exactly my changes. I'll stage my 9 explicit files and commit (leaving the sibling's untracked RUNLOG and the dev tools alone):

170. user (2026-07-07T17:27:02.479Z)

```
warning: in the working copy of 'tests/test_validation_cache.py', LF will be replaced by CRLF
the next time Git touches it
=== staged (must be exactly my 9 files) ===
backend/infrastructure/database.py
backend/infrastructure/database_media.py
backend/orchestration.py
backend/pipeline/validators/base.py
tests/test_async_jobs.py
tests/test_database.py
tests/test_job_worker_loop.py
tests/test_locale_analytics.py
tests/test_validation_cache.py
```

171. assistant (2026-07-07T17:27:06.234Z)

Exactly my 9 files staged (the LF/CRLF notice is benign Windows line-ending normalization). Now commit:

172. user (2026-07-07T17:27:14.960Z)

```
[feat/validation-result-cache 4008742] feat(validation): cache validation verdict by video_id to
skip re-validation on reupload
 9 files changed, 522 insertions(+), 30 deletions(-)
 create mode 100644 tests/test_validation_cache.py
=== verify: ONLY my commit is ahead of origin/master ===
4008742 feat(validation): cache validation verdict by video_id to skip re-validation on reupload
```

173. assistant (2026-07-07T17:27:28.597Z)

Clean ? commit `4008742`, and `origin/master..HEAD` lists **only my commit** (no sibling work rode in). Now let me push and open the PR:

174. user (2026-07-07T17:27:32.469Z)

```
remote:
remote: Create a pull request for 'feat/validation-result-cache' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/feat/validation-
result-cache
remote:
branch 'feat/validation-result-cache' set up to track 'origin/feat/validation-result-cache'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      feat/validation-result-cache -> feat/validation-result-cache
```

175. assistant (2026-07-07T17:27:35.226Z)

Branch pushed. Now open the PR against `master` with the drafted body:

176. user (2026-07-07T17:27:41.738Z)

<https://github.com/Khelsutra/badminton-highlight-indexer/pull/518>

177. assistant (2026-07-07T17:28:02.931Z)

PR opened: **<https://github.com/Khelsutra/badminton-highlight-indexer/pull/518>**. Let me verify it's ready (not draft), that CI kicked off, and check for reviewers/CODEOWNERS to route it properly:

178. user (2026-07-07T17:28:09.064Z)

```
=== PR state ===
{"baseRefName":"master","headRefName":"feat/validation-result-cache","isDraft":false,"mergeable":
:"MERGEABLE","number":518,"reviewRequests":[],"state":"OPEN","url":"https://github.com/Khelsutra
/badminton-highlight-indexer/pull/518"}
=== CODEOWNERS present? ===
=== CI checks ===
Built wheel smoke (install + launch)    pending 0      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28885707630/job/85685221969
Golden fixtures cross-OS (macos-latest) pending 0      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28885707630/job/85685222021
Leak scan (PII / attribution guard)      pass    7s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28885707630/job/85685221917
Markdown link check (live docs) pass    10s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28885707630/job/85685221911
Ruff lint                                pass    11s      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28885707630/job/85685221942
Mypy (lenient baseline) pending 0      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28885707630/job/85685222022
Syntax sweep + import smoke (no GPU stack) pass    20s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28885707630/job/85685222026
Dependency license scan (no copyleft in runtime deps) pending 0
https://github.com/Khelsutra/badminton-highlight-
```

```
indexer/actions/runs/28885707630/job/85685221932
Full test suite (offline, mocked)           pending 0      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28885707630/job/85685222032
Golden fixtures cross-OS (ubuntu-latest)    pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28885707630/job/85685222024
Golden fixtures cross-OS (windows-latest)   pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28885707630/job/85685222086
```

179. assistant (2026-07-07T17:28:32.920Z)

PR **#518** is **OPEN**, not draft, **MERGEABLE** (clean against `master`). CI is running ? Ruff, leak scan, import smoke, and markdown-link checks already **pass**; full-suite/mypy/golden-fixtures still pending (I verified the full suite + mypy locally). There's no CODEOWNERS and no other collaborators, so there's no human reviewer to auto-request.

Let me record this checkpoint in memory (a PR was pushed), then I'll summarize:

180. assistant (2026-07-07T17:28:50.302Z)

```
---
name: pr518-validation-result-cache
description: "PR #518 OPEN ? validation-result cache keyed by video_id so a reupload skips re-
validation when config is unchanged (task_4585aa00)"
metadata:
  type: project
---
```

PR #518 OPEN (branch `feat/validation-result-cache`, base `master`, 2026-07-07):
<https://github.com/Khelsutra/badminton-highlight-indexer/pull/518> ? implements task_4585aa00
(the missing cache that made the CUJ retries re-pay validation; see `[[session-handoff-gpu-resident]]`).

Problem: the only skip path in `backend/orchestration.py::_execute_processing`` was the whole-pipeline cache (fires only on a prior **SUCCESSFUL** segmentation). After a **SEGMENTATION FAILURE** (validation passed, detection failed ? no cached segments), a reupload of the same `video_id`` re-ran the full validator suite (~6.5 min on the 5.3K 10-bit HEVC "Video 1" on the CPU-only control-plane).

Fix (dedicated table, one read site + one write site ? NOT extending the video record, to avoid threading the fingerprint through `add_video's` 4 callers + the cloud-staging path):

- `ValidationRegistry.config_fingerprint(config)`` (validators/base.py) ? SHA-256 of the **WHOLE** `validation.*`` block + `sport`` (relevance validator keys its court profile off it ? a field **OUTSIDE** `validation``) + sorted validator names + `_VALIDATION_FINGERPRINT_SCHEMA=1``. Reuse **ONLY** on match.
- `validation_cache`` table (**migration v10**; `SCHEMA_VERSION 9?10`), keyed by `video_id``; `set/get_validation_cache`` in `database_media.py`; `clear_video_data`` drops the cache row too (no FK ? written before the videos **UPSERT**).
- `_execute_processing``: between the pipeline cache and the validator run ? fingerprint match ? replay stored verdict (**PASS?**segment, **FAIL?**stored failure, no re-decode); `force=true`` bypasses + refreshes.

On reuse `val_context={}`` so the obs-report media block degrades to size-only (deliberate ? keeps it ~0-cost).

Composes with `[[pr513-worker-skip-revalidation]]` (#513) + `[[validation-shared-decode-pass]]` (#512):

post-failure reupload reaches segmentation in ~0 validation time.

```
**Gate (local, matched CI):** full suite **1853 passed / 1 skipped**, coverage **85.81%** (floor 70);
`ruff check .` clean on changed files (the 18 pre-existing errors are all in the UNTRACKED dev tools
`score_local.py`/`score_golden_gate.py`, not this change); `mypy backend training` clean (149 files).
Branch cut from `origin/master`; only my commit (4008742) is ahead ? the disjoint docs commit #517
(115b83d) landed on master mid-session but did NOT ride into my branch. **? Migration v10 must deploy
worker+control-plane together (same coupling as #507/#513).** "Send for review": no
CODEOWNERS/collaborators
to auto-request; the review mechanism is the billed `/code-review ultra 518`, which is OWNER-triggered
(I can't launch it). Files: orchestration.py, validators/base.py, infrastructure/database.py + database_media.py, tests/test_validation_cache.py (+ schema-version bumps in 3 other test files).
```

181. user (2026-07-07T17:28:50.358Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr518-validation-result-cache.md (file state is current in your context ? no need to Read it back)

182. user (2026-07-07T17:28:53.390Z)

```
1      # Memory index
2
3      - ? **Start here (GPU-resident rollout):** [session-handoff-gpu-resident.md](session-handoff-gpu-resident.md) ? #506 implemented, PR #507 open; next: on-box parity gate + 15-clip golden gate.
4
5      - [Actual repo location](actual-repo-location.md) ? primary working dir is an empty STUB; real code is under `khelsutra-guru/badminton-highlight-indexer`.
6      - [Efficient timeouts, retries & Windows gcloud SSH](efficient-timeouts-and-retries.md) ? no silent retry loops; plain `gcloud compute ssh` (plink/pscp reject `-o`, mishandle `~`); diagnose-first, run long work detached.
```

183. user (2026-07-07T17:28:57.854Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

184. assistant (2026-07-07T17:29:13.391Z)

Done. PR is open and ready for review.

[PR #518 ? feat(validation): cache validation verdict by video_id](https://github.com/Khelsutra/badminton-highlight-indexer/pull/518)

Status: OPEN · not draft · **MERGEABLE** (clean against `master`) · commit `4008742` (only my commit is ahead of `origin/master` ? the disjoint docs commit #517 that landed mid-session did not ride in).

CI so far: Ruff ?, leak scan ?, import smoke ?, markdown links ?; full-suite / mypy / golden-fixtures still running. Locally I reproduced the CI gate: **1853 passed / 1 skipped**, coverage 85.81% (floor 70), ruff + mypy clean.

On "send for review": the PR is up and marked ready, but I want to be straight with you about routing it:

- There's **no CODEOWNERS** and no other collaborators on the repo, so there's no human reviewer for me to auto-request (you're the author).
- Your actual review mechanism is the billed **ultrareview**, which is owner-triggered ? I can't launch it. To run it: **`/code-review ultra 518`**.

- If you'd like a pass **now**, I can run the local `/code-review`` skill on this diff myself and post/address findings before you spend on ultrareview ? just say the word.

One operational caveat for whoever merges/deploys: this adds ***DB migration v10*** (``validation_cache`` table). Like #507/#513, deploy the ***worker image and control-plane together*** ? v10 is additive and back-compatible, but keep them in lockstep.

The full rationale, design trade-offs (dedicated table vs. extending the video record), guardrails, and the verification record are in the PR description.

185. user (2026-07-07T17:32:02.267Z)

```
<command-message>engineering:code-review</command-message>
<command-name>/engineering:code-review</command-name>
<command-args>ultra 518</command-args>
```

186. user (2026-07-07T17:32:02.267Z)

Base directory for this skill: C:\Users\avidu\AppData\Roaming\Claude\local-agent-mode-sessions\87f748ce-e72b-44f3-8ef0-78e3212a99c3\b4180250-5146-4b86-9001-c9f3dd74258c\rpm\plugin_01FTLa86dhbVJ3HB1LdHdhN7\skills\code-review

/code-review

> If you see unfamiliar placeholders or need to check which tools are connected, see [CONNECTORS.md](../../CONNECTORS.md).

Review code changes with a structured lens on security, performance, correctness, and maintainability.

Usage

```
...
/code-review <PR URL or file path>
...
```

Review the provided code changes: @518

If no specific file or URL is provided, ask what to review.

How It Works

```
...
????????????????????????????????????????????????????????????????
?                                CODE REVIEW                                ?
????????????????????????????????????????????????????????????????
?  STANDALONE (always works)                                             ?
?  ? Paste a diff, PR URL, or point to files                            ?
?  ? Security audit (OWASP top 10, injection, auth)                     ?
?  ? Performance review (N+1, memory leaks, complexity)                 ?
?  ? Correctness (edge cases, error handling, race conditions)          ?
?  ? Style (naming, structure, readability)                             ?
?  ? Actionable suggestions with code examples                          ?
????????????????????????????????????????????????????????????????
?  SUPERCHARGED (when you connect your tools)                           ?
?  + Source control: Pull PR diff automatically                         ?
?  + Project tracker: Link findings to tickets                           ?
?  + Knowledge base: Check against team coding standards                 ?
????????????????????????????????????????????????????????????????
...
```

Review Dimensions

Security

- SQL injection, XSS, CSRF
- Authentication and authorization flaws
- Secrets or credentials in code
- Insecure deserialization
- Path traversal
- SSRF

Performance

- N+1 queries
- Unnecessary memory allocations
- Algorithmic complexity ($O(n^2)$ in hot paths)
- Missing database indexes
- Unbounded queries or loops
- Resource leaks

Correctness

- Edge cases (empty input, null, overflow)
- Race conditions and concurrency issues
- Error handling and propagation
- Off-by-one errors
- Type safety

Maintainability

- Naming clarity
- Single responsibility
- Duplication
- Test coverage
- Documentation for non-obvious logic

Output

```markdown

### Code Review: [PR title or file]

#### Summary

[1-2 sentence overview of the changes and overall quality]

#### Critical Issues

| # | File   | Line   | Issue         | Severity   |
|---|--------|--------|---------------|------------|
| 1 | [file] | [line] | [description] | ? Critical |

#### Suggestions

| # | File   | Line   | Suggestion    | Category    |
|---|--------|--------|---------------|-------------|
| 1 | [file] | [line] | [description] | Performance |

#### What Looks Good

- [Positive observations]

#### Verdict

[Approve / Request Changes / Needs Discussion]

```

If Connectors Available

If `**~source control**` is connected:

- Pull the PR diff automatically from the URL
- Check CI status and test results

If `**~project tracker**` is connected:

- Link findings to related tickets
- Verify the PR addresses the stated requirements

If ****~knowledge base**** is connected:

- Check changes against team coding standards and style guides

Tips

1. ****Provide context**** ? "This is a hot path" or "This handles PII" helps me focus.
2. ****Specify concerns**** ? "Focus on security" narrows the review.
3. ****Include tests**** ? I'll check test coverage and quality too.

187. assistant (2026-07-07T17:34:03.026Z)

This invokes the engineering plugin's standalone review skill (a self-review I perform directly) ? distinct from the repo's billed cloud `/code-review ultra``, which I can't launch. I'll review PR #518 rigorously and adversarially, despite having authored it.

Let me pull the exact pushed diff so I review what's actually on the PR, not my memory of it:

188. user (2026-07-07T17:34:07.972Z)

=== PR #518 diff (pushed HEAD vs merge-base with master) ===

```
backend/infrastructure/database.py      | 34 +++-
backend/infrastructure/database_media.py | 62 ++++++-
backend/orchestration.py                | 59 +++++-
backend/pipeline/validators/base.py     | 51 ++++++
tests/test_async_jobs.py               | 10 +-
tests/test_database.py                 | 59 +++++-
tests/test_job_worker_loop.py          | 8 +-
tests/test_locale_analytics.py         | 8 +-
tests/test_validation_cache.py          | 261 ++++++
9 files changed, 522 insertions(+), 30 deletions(-)
```

=== CI status now ===

```
Built wheel smoke (install + launch)    pass    1m6s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28885707630/job/85685221969
Dependency license scan (no copyleft in runtime deps) pass    33s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28885707630/job/85685221932
Full test suite (offline, mocked)        pass    2m51s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28885707630/job/85685222032
Golden fixtures cross-OS (macos-latest)  pass    20s     https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28885707630/job/85685222021
Golden fixtures cross-OS (ubuntu-latest) pass    22s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28885707630/job/85685222024
Golden fixtures cross-OS (windows-latest) pass    1m1s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28885707630/job/85685222086
Leak scan (PII / attribution guard)       pass    7s      https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28885707630/job/85685221917
Markdown link check (live docs)          pass    10s     https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28885707630/job/85685221911
Mypy (lenient baseline)                  pass    32s     https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28885707630/job/85685222022
Ruff lint                                pass    11s     https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28885707630/job/85685221942
Syntax sweep + import smoke (no GPU stack) pass    20s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28885707630/job/85685222026
```

189. assistant (2026-07-07T17:34:21.452Z)

All ****11 CI checks pass**** (full suite, mypy, ruff, golden-fixtures cross-OS on

mac/ubuntu/windows, wheel smoke, license scan). Now the adversarial pass over the diff. Let me re-read the two most logic-dense regions with fresh eyes:

190. user (2026-07-07T17:34:22.347Z)

```
799
800      # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
801      #    UNLESS this exact content already has a cached verdict under the current
validator config.
802      notify("validating")
803      # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
804      # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
805      # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
806      # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
807      # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
808      # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
809      # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
810      # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
811      # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
812      # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
813      # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
814      val_fingerprint = registry.config_fingerprint(config)
815      cached_validation = None
816      if not req.force:
817          try:
818              cached_validation = db.get_validation_cache(video_id)
819          except Exception: # noqa: BLE001 ? a cache-read fault must degrade to a
real validation
820              logger.warning(
821                  "[API] validation-cache read failed for video_id=%s ? re-
validating.",
822                  video_id,
823                  exc_info=True,
824              )
825      if (
826          cached_validation is not None
827          and cached_validation.get("fingerprint") == val_fingerprint
828      ):
829          # Replay the stored ValidationResult list; val_context stays {} (no fresh
ffprobe_meta ?
830          # the report's media block degrades gracefully to size-only, which is the
deliberate cost
831          # of skipping the decode). Segmentation reads neither, so it is unaffected.
832          reused_validation = True
833          val_results = [
834              ValidationResult(**r) for r in cached_validation.get("results", [])
835          ]
836          logger.info(
837              "[API] Reusing cached validation verdict (passed=%s) for video_id=%s; "
838              "validator config unchanged ? skipping the validator suite. force=true
re-validates.",
839              cached_validation.get("passed"),
840              video_id,
```

```

841         )
842     else:
843         reused_validation = False
844         logger.info(f"Running validations for {req.video_path}...")
845         val_results, val_context = registry.run_all_with_context(local_video,
config)
846         failures = [r for r in val_results if not r.passed]
847         if not reused_validation:
848             # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
849             # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
850             db.set_validation_cache(
851                 video_id,
852                 val_fingerprint,
853                 not failures,
854                 [r.model_dump() for r in val_results],

```

191. assistant (2026-07-07T17:34:31.508Z)

Confirmed the one substantive finding: the `ValidationResult(**r)` `**reconstruction` (line 833) sits outside the `try/except` (which only guards the `get` at 818). If a stored `results` dict is loadable JSON but can't rebuild a `ValidationResult` ? e.g. the model gains a required field without a fingerprint-schema bump ? it raises uncaught and **fails the job** instead of degrading to a fresh validation. A cache should be a pure speedup, never a new failure mode. Let me widen the guard:

192. user (2026-07-07T17:34:42.500Z)

The file `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\orchestration.py` has been updated successfully. (file state is current in your context ? no need to Read it back)

193. assistant (2026-07-07T17:35:02.978Z)

Now a test proving the hardening ? a poisoned cache row (matching fingerprint, un-reconstructable results) must degrade to re-validation, not crash:

194. user (2026-07-07T17:35:10.024Z)

The file `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_validation_cache.py` has been updated successfully. (file state is current in your context ? no need to Read it back)

195. user (2026-07-07T17:35:25.878Z)

```

=== validation-cache tests (incl. new hardening test) ===
..... [100%]
11 passed in 3.77s
=== ruff + mypy on orchestration ===
All checks passed!
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 1 source file

```

196. assistant (2026-07-07T17:35:32.008Z)

11/11 cache tests pass, ruff + mypy clean. Let me run the process-path integration suites to confirm the refactor didn't regress anything, then commit the review fix:

197. user (2026-07-07T17:37:00.295Z)

```
tests/test_api_endpoints.py: 10 warnings
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
156 passed, 18 warnings in 84.15s (0:01:24)
```

198. assistant (2026-07-07T17:37:10.317Z)

156 passed ? the refactor is regression-free. Let me commit the self-review fix and update the PR:

199. user (2026-07-07T17:37:18.399Z)

```
=== branch ===
feat/validation-result-cache
warning: in the working copy of 'tests/test_validation_cache.py', LF will be replaced by CRLF
the next time Git touches it
=== staged ===
backend/orchestration.py
tests/test_validation_cache.py
[feat/validation-result-cache da717e7] harden(validation-cache): degrade to re-validation on any
cache read/replay fault
 2 files changed, 47 insertions(+), 23 deletions(-)
=== commits ahead of origin/master (mine only) ===
da717e7 harden(validation-cache): degrade to re-validation on any cache read/replay fault
4008742 feat(validation): cache validation verdict by video_id to skip re-validation on reupload
```

200. assistant (2026-07-07T17:37:20.956Z)

Two commits, both mine. Let me push to update the PR:

201. user (2026-07-07T17:37:25.320Z)

```
To https://github.com/Khelsutra/badminton-highlight-indexer.git
4008742..da717e7 feat/validation-result-cache -> feat/validation-result-cache
```

202. assistant (2026-07-07T17:37:58.638Z)

Pushed (`da717e7`) ? PR #518 updated. Let me post the review summary to the PR for the record, then give you the full report:

203. user (2026-07-07T17:38:10.854Z)

<https://github.com/Khelsutra/badminton-highlight-indexer/pull/518#issuecomment-4906744210>

204. assistant (2026-07-07T17:38:30.208Z)

Review complete and posted to the PR. Here's the report:

Code Review: [PR #518 ? validation-result cache by video_id](<https://github.com/Khelsutra/badminton-highlight-indexer/pull/518>)

Summary

Adds a `video\_id`-keyed validation-result cache so an identical-content reupload skips the (minutes-long) validator suite when the validator config is unchanged. The change is well-scoped, thoroughly tested, and CI-green. My adversarial pass found **one real robustness gap**, which I fixed and pushed (`da717e7`).

Critical Issues

#	File	Line	Issue	Severity
1	[orchestration.py](backend/orchestration.py:833)	833	(pre-fix) `ValidationResult(**r)` replay sat outside the cache `try/except` ? a JSON-loadable but schema-incompatible cached row would raise and **fail the job** instead of degrading to fresh validation	?
2	[base.py](backend/pipeline/validators/base.py:200)	~200	Fingerprint hashes the *sport name*, not the sport profile's relevance-config contents (a fallback path only). Same-name profile code change wouldn't invalidate	?
3	[orchestration.py](backend/orchestration.py:802)	802	`notify("validating")` fires on a cache hit before the skip	?
4	[orchestration.py](backend/orchestration.py:829)	~829	On reuse `val_context={}` ? obs-report media block degrades to size-only	?

Findings

#	File	Line	Finding	Category	Status
1	[orchestration.py](backend/orchestration.py:833)	833	(pre-fix) `ValidationResult(**r)` replay sat outside the cache `try/except` ? a JSON-loadable but schema-incompatible cached row would raise and **fail the job** instead of degrading to fresh validation	Correctness/robustness	*** Fixed `da717e7` ? guard widened; poisoned-row test added
2	[base.py](backend/pipeline/validators/base.py:200)	~200	Fingerprint hashes the *sport name*, not the sport profile's relevance-config contents (a fallback path only). Same-name profile code change wouldn't invalidate	Edge case	Accepted ? schema-version escape hatch covers it
3	[orchestration.py](backend/orchestration.py:802)	802	`notify("validating")` fires on a cache hit before the skip	Cosmetic	Accepted ? reuse is sub-ms
4	[orchestration.py](backend/orchestration.py:829)	~829	On reuse `val_context={}` ? obs-report media block degrades to size-only	By-design	Accepted ? deliberate, keeps reuse ~0-cost

What Looks Good

- **Content-hash keying** ? `video\_id` is the MD5, so a different video (even same filename) never reuses another's verdict; a dedicated test proves it.
- **Fingerprint captures the full validation surface** ? the whole `validation.\*` block (so a new knob is covered automatically) *plus* `sport` (caught by reading `relevance.py`, which keys its court profile off it ? a field outside `validation`) *plus* the validator set *plus* a bumpable schema version.
- **Dedicated-table design** minimizes blast radius ? one read site, one write site; no threading the fingerprint through `add\_video`'s four callers or the cloud-staging path.
- **Security clean** ? parameterized SQL, no secrets, no PII in the cache, no path use of `video\_id`.
- **Guardrails enforced by tests** ? config change *and* sport change force re-validation; `force=true` bypasses + refreshes; cached FAIL replays without re-decode.
- **Migration v10** follows the established `CREATE ... IF NOT EXISTS` re-runnable pattern; additive-only.

Verdict

*** Approve *** ? after the applied hardening (`da717e7`). CI on `4008742` was **11/11 green**;
it's re-running on `da717e7` (small error-handling refinement + test). Locally: full suite
1853p/1s @ 85.81% , process-path suites **156p** , cache suite **11p** all green post-fix.

Note: this is my own change, so treat this as a rigorous self-review, not independent sign-off. For the billed independent pass, run `code-review ultra 518` (owner-triggered ? I can't launch it). One deploy reminder stands: **migration v10 ? deploy worker + control-plane together**.

205. user (2026-07-07T17:38:32.311Z)

pls merge if no blockers found in the review

206. assistant (2026-07-07T17:38:50.571Z)

Before merging, I need to verify the pre-merge gate holds on the *final* commit (`da717e7`), and check that `master` hasn't moved in a way that conflicts (a sibling session merged PR #519, and my change adds migration **v10** ? I must confirm no schema collision). Let me check:

207. user (2026-07-07T17:38:59.342Z)

```
115b83d..71c88ef master -> origin/master
=== new commits on origin/master since my base ef41473 ===
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
```

```
=== did anything on master touch migrations / schema version / validation_cache? ===
=== PR mergeable + CI on latest commit da717e7 ===
{"headRefOid":"da717e784258ef79969ddf12a63b692d0a09beec","mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE"}
```

208. assistant (2026-07-07T17:39:08.063Z)

Both new master commits (#517, #519) are **docs-only** ? no schema/migration changes, so my v10 doesn't collide, and they're disjoint from my code. PR is MERGEABLE but `UNSTABLE` (checks still settling). Let me see exactly which checks are pending/failing on `da717e7`:

209. user (2026-07-07T17:39:10.632Z)

```
Built wheel smoke (install + launch)    pass    1m1s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218766
Dependency license scan (no copleft in runtime deps)    pass    37s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218651
Golden fixtures cross-OS (macos-latest)    pass    20s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218686
Golden fixtures cross-OS (windows-latest)    pass    1m17s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218662
Leak scan (PII / attribution guard)    pass    6s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218774
Markdown link check (live docs)    pass    7s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218677
Mypy (lenient baseline)    pass    32s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218685
Ruff lint    pass    10s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218707
Syntax sweep + import smoke (no GPU stack)    pass    22s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218694
Full test suite (offline, mocked)    pending 0    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218700
Golden fixtures cross-OS (ubuntu-latest)    pass    21s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218663
```

210. assistant (2026-07-07T17:39:31.041Z)

Everything on `da717e7` passes except the **full test suite**, which is still running (that's the `UNSTABLE` state). Per my pre-merge discipline I won't merge until the full suite is green. Let me wait for it (it took ~3 min in the prior run):

211. user (2026-07-07T17:40:35.733Z)

```
Mypy (lenient baseline)    pass    32s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218685
Ruff lint    pass    10s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218707
Syntax sweep + import smoke (no GPU stack)    pass    22s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218694
Built wheel smoke (install + launch)    pass    1m1s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218766
Dependency license scan (no copleft in runtime deps)    pass    37s
https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218651
Full test suite (offline, mocked)    pass    2m54s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218700
Golden fixtures cross-OS (macos-latest)    pass    20s    https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28886293622/job/85687218686
```

```
Golden fixtures cross-OS (ubuntu-latest)      pass    21s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28886293622/job/85687218663
Golden fixtures cross-OS (windows-latest)     pass    1m17s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28886293622/job/85687218662
Leak scan (PII / attribution guard)           pass    6s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28886293622/job/85687218774
Markdown link check (live docs) pass         7s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28886293622/job/85687218677
Mypy (lenient baseline) pass                 32s      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28886293622/job/85687218685
Ruff lint      pass    10s      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28886293622/job/85687218707
Syntax sweep + import smoke (no GPU stack)    pass    22s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28886293622/job/85687218694
```

212. assistant (2026-07-07T17:41:01.639Z)

All ****11 CI checks pass**** on `da717e7` (full suite 2m54s, ruff, mypy, coverage-gated suite, golden fixtures cross-OS).

****Pre-merge gate ? all green, no blockers:****

- ? Full test suite: CI green on `da717e7`; locally 1853p/1s
- ? Ruff + Mypy: CI green
- ? Coverage: 85.81% ? 70 floor (CI enforces `--cov-fail-under=70`)
- ? Review finding resolved (hardening in `da717e7`)
- ? Mergeable; master's new commits (#517/#519) are docs-only and disjoint ? clean squash, no sibling work rides in

The user authorized merge if no blockers ? there are none. Let me do a final branch-safety re-check and squash-merge:

213. user (2026-07-07T17:41:05.205Z)

```
=== FINAL check: only MY commits are ahead of current origin/master (71c88ef) ===
da717e7 harden(validation-cache): degrade to re-validation on any cache read/replay fault
4008742 feat(validation): cache validation verdict by video_id to skip re-validation on reupload
--- (must be exactly my 2 commits; no sibling commit) ---
```

214. assistant (2026-07-07T17:41:08.316Z)

Confirmed ? exactly my 2 commits, no sibling work. Squash-merging:

215. user (2026-07-07T17:41:19.653Z)

(Bash completed with no output)

216. assistant (2026-07-07T17:41:24.479Z)

Let me verify the merge landed:

217. user (2026-07-07T17:41:31.008Z)

```
=== PR state ===
{"mergeCommit":{"oid":"699d85bc0abd6e35d3196a4566dcec23f649394e"},"mergedAt":"2026-07-
07T17:41:17Z","state":"MERGED"}
=== origin/master tip after merge ===
71c88ef..699d85b master -> origin/master
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
```

=== validation_cache migration present on master? ===

3

218. assistant (2026-07-07T17:41:42.503Z)

****Merged.**** PR #518 is MERGED (squash commit `699d85b` on master, at 2026-07-07T17:41), remote branch auto-deleted, and migration v10 is on master. Let me sync my local checkout to master and clean up the local branch:

219. user (2026-07-07T17:41:47.027Z)

```
(use "git pull" to update your local branch)
Switched to branch 'master'
  create mode 100644 docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_validation-latency_2026-07-07.md
  create mode 100644 tests/test_validation_cache.py
=== master tip ===
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
=== delete merged local feature branch ===
      'refs/remotes/origin/feat/validation-result-cache', but not yet merged to HEAD
Deleted branch feat/validation-result-cache (was da717e7).
```

220. assistant (2026-07-07T17:41:54.217Z)

Local `master` is at my merged commit `699d85b`, feature branch cleaned up. Let me update memory to reflect the merge:

221. user (2026-07-07T17:41:58.306Z)

```
<tool_use_error>String to replace not found in file.
String: ---
name: pr518-validation-result-cache
description: "PR #518 OPEN ? validation-result cache keyed by video_id so a reupload skips re-
validation when config is unchanged (task_4585aa00)"
metadata:
  type: project
---
```

****PR #518 OPEN**** (branch `feat/validation-result-cache`, base `master`, 2026-07-07):
<https://github.com/Khelsutra/badminton-highlight-indexer/pull/518> ? implements task_4585aa00
(note: Edit also tried swapping \uXXXX escapes and their characters; neither form matched, so the mismatch is likely elsewhere in old_string. Re-read the file and copy the exact surrounding text.)</tool_use_error>

222. assistant (2026-07-07T17:42:03.507Z)

Let me re-read the memory file (a sibling session may have touched it):

223. user (2026-07-07T17:42:04.303Z)

```
1      ---
2      name: pr518-validation-result-cache
3      description: "PR #518 OPEN ? validation-result cache keyed by video_id so a reupload
skips re-validation when config is unchanged (task_4585aa00)"
4      metadata:
5          node_type: memory
6          type: project
7          originSessionId: b3f086c7-9143-451e-8d2c-de20114d21c6
8      ---
9
10     **PR #518 OPEN** (branch `feat/validation-result-cache`, base `master`, 2026-07-07):
11     https://github.com/Khelsutra/badminton-highlight-indexer/pull/518 ? implements
task_4585aa00
```

12 (the missing cache that made the CUJ retries re-pay validation; see [[session-handoff-gpu-resident]]).

224. user (2026-07-07T17:42:11.662Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr518-validation-result-cache.md has been updated successfully. (file state is current in your context ? no need to Read it back)

225. user (2026-07-07T17:42:17.843Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr518-validation-result-cache.md has been updated successfully. (file state is current in your context ? no need to Read it back)

226. assistant (2026-07-07T17:42:24.555Z)

Now the MEMORY.md index line ? let me read its current state (a sibling edited it) and update:

227. user (2026-07-07T17:42:25.834Z)

12:- [PR #518 validation-result cache](pr518-validation-result-cache.md) ? reupload skips re-validation when config unchanged (video_id-keyed cache, migration v10); OPEN, gate green.

228. user (2026-07-07T17:42:31.286Z)

<tool_use_error>File has been modified since read, either by the user or by a linter. Read it again before attempting to write it.</tool_use_error>

229. user (2026-07-07T17:42:36.904Z)

11 - [compute=local hosted guardrail](compute-local-hosted-guardrail.md) ? the "weights not found" trap fix: #511 + #169 + revendor #516 merged AND **DEPLOYED LIVE** (control-plane rev `00018-qvz`, image `sha256:8045f54e?`; rollback ? rev `00017-qbd`).
12 - [Validation shared decode pass](validation-shared-decode-pass.md) ? validators now share ONE decode pass (**PR #512 MERGED**, squash 20862d7); pure grab()-through is SLOWER than seeks ? use forward-hybrid. **CLOSED 2026-07-07: DEPLOY_REPORT + PR #519 MERGED (squash 71c88ef)** (real-L4 A/B 169?70 s = 2.43x; off-box 1.95?3.23x; zero decision drift). True-4K sub-second follow-up = issue #514.
13 - [PR #518 validation-result cache](pr518-validation-result-cache.md) ? reupload skips re-validation when config unchanged (video_id-keyed cache, migration v10); OPEN, gate green.

230. user (2026-07-07T17:42:42.653Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

231. assistant (2026-07-07T17:42:52.052Z)

Let me update the memory body's closing note (it still says OPEN + review-pending) to reflect the self-review and merge:

232. user (2026-07-07T17:42:52.925Z)

30 **Composes with [[pr513-worker-skip-revalidation]] (#513) + [[validation-shared-decode-pass]] (#512):**
31 post-failure reupload reaches segmentation in ~0 validation time.
32
33 **Gate (local, matched CI):** full suite **1853 passed / 1 skipped**, coverage **85.81%** (floor 70);
34 `ruff check .` clean on changed files (the 18 pre-existing errors are all in the UNTRACKED dev tools

```
35     `score_local.py`/`score_golden_gate.py`, not this change); `mypy backend training` clean
(149 files).
36     Branch cut from `origin/master`; only my commit (4008742) is ahead ? the disjoint docs
commit #517
37     (115b83d) landed on master mid-session but did NOT ride into my branch. **? Migration
v10 must deploy
38     worker+control-plane together (same coupling as #507/#513).** "Send for review": no
CODEOWNERS/collaborators
39     to auto-request; the review mechanism is the billed `/code-review ultra 518`, which is
OWNER-triggered
40     (I can't launch it). Files: orchestration.py, validators/base.py,
infrastructure/database.py +
41     database_media.py, tests/test_validation_cache.py (+ schema-version bumps in 3 other
test files).
```

233. user (2026-07-07T17:43:04.405Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr518-validation-result-cache.md has been updated successfully. (file state is current in your context ? no need to Read it back)

234. assistant (2026-07-07T17:43:23.312Z)

Done ? PR #518 is merged and everything is cleaned up.

Merged: [feat(validation): cache validation verdict by video_id (#518)](<https://github.com/Khelsutra/badminton>)

****No blockers found**** ? squash-merged as `699d85b` on `master` at 17:41 UTC.

****Pre-merge gate (verified on the final commit `da717e7`, not just the description):****

- ? CI ****11/11 green**** ? full test suite, ruff, mypy, coverage-gated suite, golden fixtures on mac/ubuntu/windows, wheel smoke, license/leak scans
- ? Local full suite 1853p/1s @ ****85.81%**** coverage (? 70 floor, no regression)
- ? The one self-review finding (unguarded cache replay) fixed ****before**** merge, with a regression test
- ? Clean squash ? the two docs commits that landed on `master` mid-session (#517, #519) are disjoint and did not ride into my change

****Post-merge cleanup:****

- Remote + local `feat/validation-result-cache` branches deleted
- Local `master` fast-forwarded to `699d85b`
- Memory updated (PR #518 ? MERGED, with the self-review note)

****One operational reminder for deploy:**** this ships ****DB migration v10**** (`validation\_cache` table). It's additive/back-compatible, but keep the ****worker image and control-plane in lockstep**** on the rollout ? same coupling as #507/#513. After deploy, the CUJ payoff lands: a post-segmentation-failure reupload of the same clip now reaches segmentation in ~0 validation time (composing with #512's faster validators and #513's worker-side skip).

One caveat worth stating plainly: the independent billed ****`/code-review ultra 518`**** was ****not**** run ? that's owner-triggered and I can't launch it. What I did was a rigorous ****self****-review of my own change, so if you want a second, independent pass, that command is still available (though the PR is now merged, so it'd be post-merge).